

Оглавление

ГЛАВА 1. ТЕОРЕТИЧЕСКАЯ ЧАСТЬ	2
1.1. ВРЕМЕННЫЕ РЯДЫ И ПОСТАНОВКА ЗАДАЧИ	2
1.1.1. Введение	2
1.1.2. Постановка задачи	3
1.1.3. Метрики качества прогноза	3
1.2. КЛАССИЧЕСКИЕ МОДЕЛИ ВРЕМЕННЫХ РЯДОВ	4
1.2.1. Модели AR, MA, ARIMA, SARIMA	4
1.2.2. ACF, PACF и выбор порядка модели	6
1.2.3. ETS	6
1.2.4. Модель Prophet	8
1.3. НЕЙРОСЕТЕВЫЕ МОДЕЛИ ДЛЯ ВРЕМЕННЫХ РЯДОВ	9
1.3.1. Модель HELLformer	9
1.3.2. Модель FEDformer	12
ГЛАВА 2. ПРАКТИЧЕСКАЯ ЧАСТЬ	16
2.1. ОПИСАНИЕ ПАРАМЕТРОВ И АРХИТЕКТУР МОДЕЛЕЙ С ПРАКТИЧЕСКОЙ ТОЧКИ ЗРЕНИЯ	16
2.1.1. ARIMA - один статистический блок ARIMA(p,d,q) на исходном ряде.	16
2.1.2. ARIMA-auto	16
2.1.3. ETS	17
2.1.4. Prophet с аддитивной моделью и без встроенных календарных сезонностей	17
2.1.5. HELLformer - нейросетевой блок	17
2.2. ПРИМЕНЕНИЕ И ДИАГНОСТИКА МОДЕЛЕЙ НА СИНТЕТИЧЕСКИХ РЯДАХ	18
2.2.1. Используемые профили (ряды для испытаний)	18
2.2.2. Обучение моделей, подбор параметров ARIMA	21
2.2.3. Результаты обучения моделей на синтетических рядах	27
2.3. ТЕСТИРОВАНИЕ МОДЕЛЕЙ НА НАБОРЕ ДАННЫХ M3	32
2.3.1. Протокол обучения и тестирования	32
2.3.2. Количественные результаты	33
2.3.3. Качественный анализ прогнозов на отдельных рядах	34
2.3.4. Анализ разложения HELLformer (роль базовой компоненты и остатка)	35
2.3.5. Вывод по эксперименту M3	38
2.4. ТЕСТИРОВАНИЕ МОДЕЛЕЙ НА НАБОРЕ ДАННЫХ M4	39
2.4.1. Протокол эксперимента	39
2.4.2. Количественные результаты и их интерпретация	40
2.4.3. Качественный анализ (примеры рядов quarterly)	40
2.4.4. Вывод по тестированию на M4	42
ЗАКЛЮЧЕНИЕ	43
СПИСОК ЛИТЕРАТУРЫ	44

Глава 1. Теоретическая часть

1.1. Временные ряды и постановка задачи

1.1.1. Введение

Временной ряд — это последовательность случайных величин, упорядоченных по времени:

$$\{X_t\}_{t \in T'}$$

где T — множество индексов времени. Чаще всего:

- **дискретное время:** $T = \mathbb{Z}$ или $T = \{1, 2, \dots\}$;
- **непрерывное время:** $T = \mathbb{R}$ или $T = [0, \infty)$.

Важно: во временных рядах наблюдения почти всегда **зависимы** (в отличие от классической выборки i.i.d.). Эту зависимость мы описываем через распределения совместных векторов $(X_{t_1}, \dots, X_{t_k})$, ковариации, спектры и т.п.

Процесс $\{X_t\}$ называется **строго стационарным** (или **сильно стационарным**), если для любых $k \in \mathbb{N}$, любых моментов времени $t_1, \dots, t_k \in T$ и любого сдвига h , такого что $t_i + h \in T$, выполняется равенство распределений:

$$(X_{t_1}, \dots, X_{t_k}) =_d (X_{t_1+h}, \dots, X_{t_k+h}).$$

То есть **все конечномерные распределения** инвариантны к сдвигу.

Это определение максимально «сильное»: оно говорит не только о средних/дисперсиях/ковариациях, а вообще обо всей вероятностной структуре.

Если процесс строго стационарен, то:

1. Все одномерные распределения одинаковы:

$$X_t =_d X_{t+h} \quad \forall t, h.$$

2. Если существуют математические ожидания, то они постоянны:

$$\mathbb{E}[X_t] = \mu \quad \forall t.$$

3. Если существуют вторые моменты, то дисперсия постоянна:

$$\text{Var}(X_t) = \sigma^2 \quad \forall t.$$

4. Если существуют ковариации, то ковариационная функция зависит только от лага:

$$\text{Cov}(X_t, X_{t+h}) = \gamma(h),$$

где $\gamma(h)$ — автоковариационная функция.

Процесс $\{X_t\}$ называется **слабо стационарным** (или **стационарным в широком смысле**, second-order stationary), если:

1. $\mathbb{E}[X_t] = \mu$ не зависит от t ;
2. $\text{Var}(X_t) < \infty$ и не зависит от t ;
3. $\text{Cov}(X_t, X_{t+h}) = \gamma(h)$ зависит только от лага h , а не от t .

Это определение уже «про моменты», а не про распределения.

Замечание. Если процесс строго стационарен и $\mathbb{E}X_t^2 < \infty$, то он **слабо стационарен**.

Если процесс гауссовский, то его конечномерные распределения полностью определяются средним и ковариационной матрицей, тогда слабая стационарность эквивалентна строгой стационарности

Для слабо стационарного процесса определяют:

- автоковариацию:

$$\gamma(h) = \text{Cov}(X_t, X_{t+h})$$

- автокорреляцию:

$$\rho(h) = \frac{\gamma(h)}{\gamma(0)}, \quad \gamma(0) = \text{Var}(X_t).$$

Свойства:

1. $\gamma(-h) = \gamma(h)$ (симметрия);
2. $|\rho(h)| \leq 1$;
3. Матрица $[\gamma(t_i - t_j)]_{i,j}$ неотрицательно определена (важное структурное ограничение).

Один из самых распространённых способов получить стационарность — перейти к разностям:

$$\Delta X_t = X_t - X_{t-1}.$$

Например, если X_t — случайное блуждание, то $\Delta X_t = \varepsilon_t$ (белый шум), который стационарен.

Это приводит к идеям **интегрированности** и моделям **ARIMA**, где исходный ряд может быть нестационарным, но после d разностей становится стационарным.

1.1.2. Постановка задачи

В настоящей работе рассматривается задача краткосрочного прогноза унивариативных временных рядов, то есть последовательностей наблюдений одной случайной величины, упорядоченных во времени.

Пусть задан временной ряд наблюдений $\{X_t\}_{t=1}^T$.

Задаются два параметра: длина исторического окна L и горизонт прогноза H . В момент времени t (для $t \geq L$) модель получает на вход вектор последних L наблюдений

$$X_t^{(\text{in})} = (X_{t-L+1}, \dots, X_t),$$

и должна сформировать прогноз на следующие H шагов вперёд в виде вектора

$$\hat{X}_t^{(\text{out})} = (\hat{X}_{t+1}, \dots, \hat{X}_{t+H}).$$

Таким образом, требуется построить модель f , реализующую отображение

$$\hat{X}_t^{(\text{out})} = f(X_t^{(\text{in})}), \text{ то есть } (\hat{X}_{t+1}, \dots, \hat{X}_{t+H}) = f(X_{t-L+1}, \dots, X_t),$$

и минимизирующую выбранную функцию ошибки на тестовых данных.

1.1.3. Метрики качества прогноза

Для количественной оценки качества прогнозов часто используют метрики ошибок в исходном масштабе ряда, а также относительные (процентные) ошибки. В данной работе применяются следующие показатели.

MSE (mean squared error) — среднеквадратичная ошибка:

$$\text{MSE} = \frac{1}{N} \sum_{i=1}^N (X_i - \hat{X}_i)^2$$

За счёт возведения отклонений в квадрат эта метрика существенно сильнее штрафует большие промахи.

RMSE (root mean squared error) — корень из среднеквадратичной ошибки:

$$\text{RMSE} = \sqrt{\text{MSE}} = \sqrt{\frac{1}{N} \sum_{i=1}^N (X_i - \hat{X}_i)^2}$$

Поскольку RMSE измеряется в тех же единицах, что и X_t , её удобнее интерпретировать на практике.

MAPE (mean absolute percentage error) — средняя абсолютная процентная ошибка:

$$\text{MAPE} = \frac{100\%}{N} \sum_{i=1}^N \left| \frac{X_i - \hat{X}_i}{X_i} \right|$$

Метрика показывает среднюю относительную ошибку в процентах, однако плохо определена и может быть неустойчивой при $X_i \approx 0$.

sMAPE (symmetric mean absolute percentage error) — симметричная средняя абсолютная процентная ошибка:

$$\text{sMAPE} = \frac{100\%}{N} \sum_{i=1}^N \frac{2|X_i - \hat{X}_i|}{|X_i| + |\hat{X}_i|}$$

В отличие от MAPE, sMAPE симметрична относительно фактических и прогнозных значений и, как правило, менее чувствительна к случаям, когда значения ряда близки к нулю.

1.2. Классические модели временных рядов

1.2.1. Модели AR, MA, ARIMA, SARIMA

AR(p) (autoregressive) задаётся рекуррентным соотношением

$$X_t = c + \sum_{i=1}^p \varphi_i X_{t-i} + \varepsilon_t, t \in \mathbb{Z},$$

где $c \in \mathbb{R}$ — константа (дрейф), $\varphi_1, \dots, \varphi_p \in \mathbb{R}$ — параметры, а $\{\varepsilon_t\}$ — белый шум, обычно предполагают

$$\mathbb{E}\varepsilon_t = 0, \quad \text{Var}(\varepsilon_t) = \sigma^2, \quad \text{Cov}(\varepsilon_t, \varepsilon_{t-h}) = 0 \quad (h \neq 0).$$

Введём оператор сдвига (lag operator) B , $BX_t = X_{t-1}$, тогда AR(p) записывается как

$$\varphi(B)X_t = c + \varepsilon_t, \quad \varphi(B) = 1 - \varphi_1 B - \dots - \varphi_p B^p$$

AR(p) называется (слабо) стационарной, если существует решение со стационарной ковариационной структурой. Достаточное и стандартное условие:

$$\varphi(z) \neq 0 \text{ для всех } |z| \leq 1,$$

то есть все корни характеристического полинома $\varphi(z)$ лежат **вне** единичного круга.

При стационарности можно выразить среднее:

$$\mu = \mathbb{E}X_t = \frac{c}{1 - \sum_{i=1}^p \varphi_i} \quad (\text{если } 1 - \sum_{i=1}^p \varphi_i \neq 0).$$

MA(q) (moving average) определяется как

$$X_t = \mu + \varepsilon_t + \sum_{j=1}^q \theta_j \varepsilon_{t-j},$$

где $\mu \in \mathbb{R}$, $\theta_1, \dots, \theta_q \in \mathbb{R}$, $\{\varepsilon_t\}$ — белый шум.

$$X_t = \mu + \theta(B)\varepsilon_t, \quad \theta(B) = 1 + \theta_1 B + \dots + \theta_q B^q.$$

MA(q) всегда (слабо) стационарна при $\text{Var}(\varepsilon_t) < \infty$, потому что это конечная линейная комбинация белого шума.

Чтобы модель имела единственное представление и можно было выражать шум через наблюдения, вводят условие обратимости:

$$\theta(z) \neq 0 \text{ для всех } |z| \leq 1$$

ARMA(p,q) - комбинация AR и MA:

$$X_t = c + \sum_{i=1}^p \varphi_i X_{t-i} + \varepsilon_t + \sum_{j=1}^q \theta_j \varepsilon_{t-j}.$$

В операторной форме:

$$\varphi(B)X_t = c + \theta(B)\varepsilon_t.$$

Стационарность определяется условиями на $\varphi(B)$ (корни $\varphi(z)$ вне единичного круга), обратимость — условиями на $\theta(B)$.

Модели ARMA предполагают (после центрирования) стационарность. Если $\{X_t\}$ нестационарен из-за тренда типа интегрированности, вводят разностный оператор:

$$\Delta X_t = X_t - X_{t-1} = (1 - B)X_t, \quad \Delta^d X_t = (1 - B)^d X_t$$

Процесс $\{X_t\}$ следует модели **ARIMA(p,d,q)**, если разностный ряд

$$Y_t := \Delta^d X_t$$

подчиняется ARMA(p,q):

$$\varphi(B)Y_t = c + \theta(B)\varepsilon_t.$$

Эквивалентная запись:

$$\varphi(B)(1 - B)^d X_t = c + \theta(B)\varepsilon_t.$$

Интерпретация параметра d

- $d = 0$: ARMA(p,q) на самом X_t .
- $d = 1$: стационаризуем первые разности.
- $d = 2$: стационаризуем вторые разности и т.д.

В ARIMA часто отдельно выделяют **дрейф** (линейный тренд в уровнях), который проявляется как константа в уравнении для разностей.

Если, помимо тренда, в данных присутствует сезонность с периодом s (например, $s = 12$ для месячных данных с годовой сезонностью), используют сезонное разностирование и сезонные AR/MA-компоненты:

$$\Delta_s X_t = X_t - X_{t-s} = (1 - B^s)X_t, \quad \Delta_s^D X_t = (1 - B^s)^D X_t$$

Процесс $\{X_t\}$ удовлетворяет модели **SARIMA(p, d, q) × (P, D, Q)_s**, если после обычного и сезонного разностирования получаем стационарный ARMA с сезонными полиномами:

$$\Phi(B^s) \varphi(B) (1 - B)^d (1 - B^s)^D X_t = c + \Theta(B^s) \theta(B) \varepsilon_t,$$

где

$$\begin{aligned} \varphi(B) &= 1 - \varphi_1 B - \dots - \varphi_p B^p, & \theta(B) &= 1 + \theta_1 B + \dots + \theta_q B^q, \\ \Phi(B^s) &= 1 - \Phi_1 B^s - \dots - \Phi_P B^{Ps}, & \Theta(B^s) &= 1 + \Theta_1 B^s + \dots + \Theta_Q B^{Qs}. \end{aligned}$$

Смысл параметров

- p, q : несезонные AR и MA порядки,
- d : порядок обычного разностирования,
- P, Q : сезонные AR и MA порядки,
- D : порядок сезонного разностирования,
- s : длина сезона (период сезонности).

Условия стационарности/обратимости

В стандартной постановке требуют, чтобы:

- несезонный AR-полином $\varphi(z)$ не имел корней в $|z| \leq 1$;
- сезонный AR-полином $\Phi(z)$ (в переменной $z = B^s$) также не имел корней в $|z| \leq 1$;

- аналогично для МА-полиномов $\theta(z)$ и $\Theta(z)$ — условия обратимости. Разностные множители $(1 - B)^d(1 - B^s)^D$ специально «встроены» для устранения (соответственно) трендовой и сезонной нестационарности.

1.2.2. ACF, PACF и выбор порядка модели

Для слабо стационарного временного ряда $\{X_t\}$ (то есть при $\mathbb{E}X_t = \mu$, $\text{Var}(X_t) < \infty$) вводятся функции, описывающие линейную зависимость между значениями ряда на разных лагах.

Автоковариационная функция (ACVF) определяется как

$$\gamma(h) = \text{Cov}(X_t, X_{t+h}), \quad h \in \mathbb{Z}$$

и зависит только от лага h . Нормируя $\gamma(h)$ на дисперсию $\gamma(0)$, получают **автокорреляционную функцию (ACF)**:

$$\rho(h) = \frac{\gamma(h)}{\gamma(0)} = \frac{\text{Cov}(X_t, X_{t+h})}{\text{Var}(X_t)}, \quad h \in \mathbb{Z}$$

В практических задачах обычно рассматривают $\rho(h)$ для $h \geq 0$; при этом $\rho(0) = 1$ и $\rho(-h) = \rho(h)$.

Частичная автокорреляционная функция (PACF) на лаге h измеряет линейную связь между X_t и X_{t-h} после исключения влияния промежуточных лагов $X_{t-1}, \dots, X_{t-h+1}$. Формально $\text{PACF}(h)$ — это коэффициент при X_{t-h} в линейной проекции X_t на пространство, натянутое на $\{X_{t-1}, \dots, X_{t-h}\}$:

$$X_t = a_h + \sum_{j=1}^h \phi_{h,j} X_{t-j} + u_{t,h},$$

где $u_{t,h}$ ортогонален всем $X_{t-1}, \dots, X_{t-h}$. Тогда

$$\alpha(h) := \text{PACF}(h) = \phi_{h,h}.$$

Эквивалентно, $\alpha(h) = \text{Corr}(r_t, r_{t-h})$, где r_t и r_{t-h} — остатки регрессий X_t и X_{t-h} на $X_{t-1}, \dots, X_{t-h+1}$.

ACF и PACF используются для идентификации порядков AR/MA-компонент:

- для стационарного **AR(p)** PACF «обрывается» после лага p (идеально: $\alpha(h) = 0$ при $h > p$), тогда как ACF обычно убывает постепенно;
- для **MA(q)** наоборот, ACF «обрывается» после лага q (идеально: $\rho(h) = 0$ при $h > q$), а PACF убывает постепенно.
- для смешанных **ARMA(p,q)** обе функции, как правило, затухают без резкого обрыва.

1.2.3. ETS

Модели класса **ETS (Error–Trend–Seasonal)** описывают временной ряд $\{X_t\}$ через *компоненты состояния* (уровень, тренд, сезонность) и *уравнения обновления* этих компонент во времени.

В отличие от ARIMA-подхода, где стационарность достигается разностями, ETS задаёт явную декомпозицию ряда и динамику компонент, а прогноз получается, как условное математическое ожидание будущих значений при заданном текущем состоянии.

Пусть сезонный период равен $s \in \mathbb{N}$ (например, $s = 12$).

Обозначим

- ℓ_t — **уровень** (level)
- b_t — **тренд** (trend)
- s_t — **сезонная компонента** (seasonal)

В каждый момент времени модель формирует однопериодный прогноз (в терминах состояния)

$$\hat{X}_{t|t-1} := \mathbb{E}(X_t | \mathcal{F}_{t-1}),$$

где $\mathcal{F}_{t-1}$ — информация, доступная до момента t .

Далее вводится **ошибка** (инновация):

$$\varepsilon_t := \frac{X_t - \hat{X}_{t|t-1}}{\sigma_t},$$

или в более простом варианте $\varepsilon_t := X_t - \hat{X}_{t|t-1}$

Конкретная нормировка зависит от того, является ли ошибка аддитивной или мультипликативной. В большинстве классических ETS предполагают, что $\{\varepsilon_t\}$ — белый шум с нулевым средним и постоянной дисперсией (часто $\varepsilon_t \sim \mathcal{N}(0,1)$ или $\mathcal{N}(0, \sigma^2)$ в зависимости от записи).

ETS-классификация задаётся тройкой букв ETS(E, T, S), где каждая из компонент может быть:

- **A** (additive) — аддитивная,
- **M** (multiplicative) — мультипликативная,

В аддитивной постановке наблюдение представляется как сумма компонент и ошибки; в мультипликативной — как произведение (что естественно, когда амплитуда сезонности растёт вместе с уровнем).

Пусть $\alpha, \beta, \gamma \in (0,1)$ — коэффициенты сглаживания (скорости адаптации), тогда:

- **Аддитивная ETS – ETS(A, A, A)**

Одношаговый прогноз задаётся

$$\hat{X}_{t|t-1} = \ell_{t-1} + b_{t-1} + s_{t-s}.$$

Ошибка (инновация) в аддитивной модели:

$$e_t = X_t - \hat{X}_{t|t-1}.$$

Уравнения обновления состояния:

$$\begin{aligned}\ell_t &= \ell_{t-1} + b_{t-1} + \alpha e_t, \\ b_t &= b_{t-1} + \beta e_t, \\ s_t &= s_{t-s} + \gamma e_t.\end{aligned}$$

То есть уровень, тренд и сезонность корректируются пропорционально текущей ошибке прогноза. Прогноз на горизонт $h \geq 1$ имеет вид

$$\hat{X}_{t+h|t} = \ell_t + hb_t + s_{t+h-s \cdot k}$$

где индекс сезонности берётся по модулю периода s (формально $k = \left\lfloor \frac{h}{s} \right\rfloor$, чтобы $t + h - sk$ указывал на соответствующий «сезонный класс»).

- **Мультипликативная ETS – ETS(M, A, M)**

Если вариабельность ряда пропорциональна уровню, применяют мультипликативные формы. Для ETS(M, A, M) обычно используют прогноз

$$\hat{X}_{t|t-1} = (\ell_{t-1} + b_{t-1}) s_{t-s}$$

и нормированную ошибку

$$e_t = \frac{X_t - \hat{X}_{t|t-1}}{\hat{X}_{t|t-1}}$$

Уравнения обновления:

$$\begin{aligned}\ell_t &= (\ell_{t-1} + b_{t-1}) (1 + \alpha e_t) \\ b_t &= b_{t-1} + \beta (\ell_{t-1} + b_{t-1}) e_t \\ s_t &= s_{t-s} (1 + \gamma e_t)\end{aligned}$$

Здесь корректировки также зависят от ошибки, но обновление происходит мультипликативно для уровня и сезонности. Прогноз тогда записывается как:

$$\hat{X}_{t+h|t} = (\ell_t + h b_t) s_{t+h-s \cdot k}.$$

Во многих рядах линейный тренд не сохраняется бесконечно, поэтому используют **демпфирование** с параметром $\phi \in (0,1)$. Тогда вклад тренда в прогноз ограничивается:

$$\hat{X}_{t+h|t} = \ell_t + \left(\sum_{j=1}^h \phi^j \right) b_t + (\text{сезонность}),$$

а в уравнениях состояния b_t обновляется аналогично, но прогнозный вклад тренда «затухает». Эта конструкция лежит в основе моделей с обозначением $T = \text{Ad}(\text{additive damped})$ и часто улучшает долгосрочные прогнозы.

1.2.4. Модель Prophet

Prophet задаёт наблюдение как сумму детерминированных компонент и случайной ошибки:

$$X_t = m(t) + s(t) + h(t) + \varepsilon_t, \quad t = 1, \dots, T,$$

где

- $m(t)$ — тренд (рост/спад) с возможными точками перелома,
- $s(t)$ — сезонность (годовая/недельная/дневная и т.п.),
- $h(t)$ — эффекты событий/праздников/регрессоров,
- ε_t — шум (обычно предполагают $\varepsilon_t \sim \mathcal{N}(0, \sigma^2)$, либо близкую к этому постановку).

Компонента тренда $m(t)$

Выбирается набор K точек перелома $\tau_1 < \dots < \tau_K$ (обычно из внутренних точек диапазона времени).

Вводится индикаторный вектор признаков

$$\mathbf{a}(t) = (a_1(t), \dots, a_K(t))^T, \quad a_k(t) = \mathbf{1}\{t \geq \tau_k\}.$$

Тогда тренд задают как

$$m(t) = (k + \mathbf{a}(t)^T \boldsymbol{\delta})t + (b + \mathbf{a}(t)^T \boldsymbol{\gamma}),$$

где k — базовый наклон, b — базовый сдвиг, $\boldsymbol{\delta} = (\delta_1, \dots, \delta_K)^T$ — изменения наклона в точках перелома.

Чтобы обеспечить непрерывность тренда при изменении наклона, $\boldsymbol{\gamma}$ берут как детерминированную функцию от $\boldsymbol{\delta}$ и $\{\tau_k\}$. Стандартный способ записи:

$$m(t) = kt + b + \sum_{k=1}^K \delta_k (t - \tau_k)_+, \quad (x)_+ = \max(x, 0),$$

что автоматически даёт непрерывную кусочно-линейную функцию.

Если предполагается уровень насыщения (ёмкость рынка и т.п.), задают «capacity» $C(t) > 0$ и используют

$$m(t) = \frac{C(t)}{1 + \exp(-g(t))}$$

где латентная линейная функция $g(t)$ также кусочно-линейная:

$$g(t) = kt + b + \sum_{k=1}^K \delta_k (t - \tau_k)_+.$$

В таком виде тренд ограничен сверху $C(t)$ и допускает изменения скорости роста.

Сезонность $s(t)$: ряд Фурье

Сезонные компоненты моделируются как линейная регрессия на гармониках (Fourier series). Для сезона с периодом P (например, $P = 7$ дней, $P = 365.25$ дней) и порядком N вводят признаки:

$$\mathbf{z}_P(t) = \left(\cos \frac{2\pi t}{P}, \sin \frac{2\pi t}{P}, \dots, \cos \frac{2\pi Nt}{P}, \sin \frac{2\pi Nt}{P} \right)^\top.$$

Тогда сезонность:

$$s_P(t) = \mathbf{z}_P(t)^\top \boldsymbol{\beta}_P.$$

Если сезонностей несколько (например, недельная и годовая), то

$$s(t) = \sum_{P \in \mathcal{P}} s_P(t) = \sum_{P \in \mathcal{P}} \mathbf{z}_P(t)^\top \boldsymbol{\beta}_P.$$

Фактически это линейная модель по набору синусоидальных базисов.

Эффекты праздников и регрессоров $h(t)$

Пусть есть набор событий (праздников) $\mathcal{E}$. Для каждого события $e \in \mathcal{E}$ строят бинарные признаки, активные в дни события и/или в окне вокруг него (lead/lag).

Обозначим вектор этих признаков $\mathbf{r}(t) \in \mathbb{R}^J$, тогда

$$h(t) = \mathbf{r}(t)^\top \boldsymbol{\kappa},$$

где $\boldsymbol{\kappa} \in \mathbb{R}^J$ — коэффициенты эффектов событий. Аналогично можно добавлять и другие внешние регрессоры.

Соберём все регрессионные признаки, кроме шума, в один вектор:

$$\mathbf{f}(t) = [t, (t - \tau_1)_+, \dots, (t - \tau_K)_+, \mathbf{z}_{P_1}(t), \dots, \mathbf{z}_{P_M}(t), \mathbf{r}(t)]^\top,$$

а параметры — в вектор $\boldsymbol{\theta}$, тогда Prophet можно рассматривать как модель

$$X_t = (\text{возможная нелинейность тренда}) + \mathbf{f}(t)^\top \boldsymbol{\theta} + \varepsilon_t,$$

где нелинейность появляется при логистическом тренде (через $C(t)/(1 + \exp(-g(t)))$) при линейном тренде всё сводится к линейной регрессии по базисам $(t - \tau_k)_+$, синусам/косинусам и индикаторам событий.

1.3. Нейросетевые модели для временных рядов

Нейросетевые модели в последние годы показали высокую эффективность в задачах прогнозирования временных рядов, особенно на больших и сложных наборах данных. В отличие от классических линейных моделей (ARIMA, ETS), нейросети способны автоматически извлекать нелинейные зависимости и сложные взаимодействия между наблюдениями.

В данной работе рассматриваются, в частности, архитектуры **Hellformer**, **FEDformer**, которые относятся к современным специализированным моделям для временных рядов.

1.3.1. Модель Hellformer

Рассматривается унивариативный временной ряд наблюдений $\{X_t\}_{t=1}^T$. Для фиксированных длин окна L и (при необходимости) горизонта H на шаге t формируется входной вектор

$$\mathbf{x}_t = (X_{t-L+1}, \dots, X_t) \in \mathbb{R}^L,$$

а модель строит прогноз $\hat{\mathbf{x}}_t^{(\text{out})} = (\hat{X}_{t+1}, \dots, \hat{X}_{t+H})$.

1) *Decomposition layer: дифференцируемая декомпозиция Holt–Winters*

Ключевая особенность Helformer — предварительная **декомпозиция** входной последовательности, направленная на явное выделение уровня и сезонности перед вниманием. Вводятся латентные компоненты **уровня** L_t и **сезонности** S_t , а также «очищенный» (десезонализированный/нормализованный) сигнал Y_t . В работе используется мультипликативная форма сглаживания (коэффициенты $\alpha, \gamma \in (0,1)$):

$$L_t = \alpha \frac{X_t}{S_t} + (1 - \alpha) L_{t-1},$$

$$S_{t+m} = \gamma \frac{Y_t}{L_{t+1}} + (1 - \gamma) S_t,$$

$$Y_t = \frac{X_t}{S_t L_t}$$

Здесь m — сезонный лаг (период сезонности).

α, γ рассматриваются как параметры слоя и оптимизируются **совместно** с весами нейросети (то есть блок декомпозиции включён в общую схему обучения).

Практически это означает, что из окна $\mathbf{x}_t$ строится последовательность $\mathbf{y}_t = (Y_{t-L+1}, \dots, Y_t)$, которая далее подаётся в attention-блоки.

2) *Encoder-only Transformer: самовнимание над $\mathbf{y}_t$*

Helformer использует **только encoder** (без decoder), что уменьшает вычислительную сложность по сравнению с «полным» Transformer. Пусть после (возможного) линейного отображения/эмбединга получена матрица признаков

$$Z_t \in \mathbb{R}^{L \times d_{\text{model}}},$$

соответствующая $\mathbf{y}_t$. Самовнимание задаётся стандартно через матрицы запросов, ключей и значений

$$Q = Z_t W_Q, \quad K = Z_t W_K, \quad V = Z_t W_V, \quad Q, K, V \in \mathbb{R}^{L \times d},$$

и внимание скалярного произведения

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d}}\right)V.$$

Мультиголовость реализуется параллельными проекциями:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) W_O$$

$$\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$$

Тем самым модель учится выделять релевантные зависимости между позициями окна (включая дальние лаги) уже на «сглаженном/десезонализированном» сигнале.

3) *Add & Norm: остаточные связи и нормализация*

Как и в Transformer, после каждого крупного подблока применяется остаточное соединение и нормализация:

$$\tilde{Z} = \text{LayerNorm}(Z + \text{Dropout}(\text{MultiHead}(\cdot))),$$

что стабилизирует обучение и улучшает распространение градиента на глубине.

4) *Замена FFN на LSTM: явная временная рекуррентность*

В классическом Transformer после attention стоит позиционно-независимый FFN. В Helformer его заменяют на **LSTM-слой**, чтобы усилить учёт последовательной динамики. Пусть на вход LSTM подаётся последовательность векторов $\tilde{z}_{t,1}, \dots, \tilde{z}_{t,L}$ (строки $\tilde{Z}$), тогда рекуррентно вычисляется скрытое состояние h_ℓ и память c_ℓ (для $\ell = 1, \dots, L$):

$$\begin{aligned} i_\ell &= \sigma(W_i \tilde{z}_{t,\ell} + U_i h_{\ell-1} + b_i), \\ f_\ell &= \sigma(W_f \tilde{z}_{t,\ell} + U_f h_{\ell-1} + b_f), \\ o_\ell &= \sigma(W_o \tilde{z}_{t,\ell} + U_o h_{\ell-1} + b_o), \\ \tilde{c}_\ell &= \tanh(W_c \tilde{z}_{t,\ell} + U_c h_{\ell-1} + b_c), \\ c_\ell &= f_\ell \odot c_{\ell-1} + i_\ell \odot \tilde{c}_\ell, \\ h_\ell &= o_\ell \odot \tanh(c_\ell). \end{aligned}$$

Тем самым attention отвечает за **глобальные зависимости**, а LSTM — за **последовательную временную динамику** в пределах окна.

5) *Выходной слой: прогноз будущего значения(ий)*

Полученное представление (например, последнее состояние h_L или агрегация по $\{h_\ell\}$) подаётся в полносвязный слой. Для одношагового прогноза:

$$\hat{X}_{t+1} = w^\top h_L + b.$$

Для прогноза с горизонтом H обычно используют векторный выход:

$$(\hat{X}_{t+1}, \dots, \hat{X}_{t+H}) = W h_L + b, \quad W \in \mathbb{R}^{H \times d_h}$$

На рисунке 1.3.1.1 показана архитектура Helformer, в которой блоки (**Series Decomposition**) $\rightarrow$ (**Multi-Head Attention**) $\rightarrow$ (**LSTM**) $\rightarrow$ (**Dense**) находятся внутри encoder-структуры, а обучение оптимизирует одновременно параметры сглаживания и веса сети.

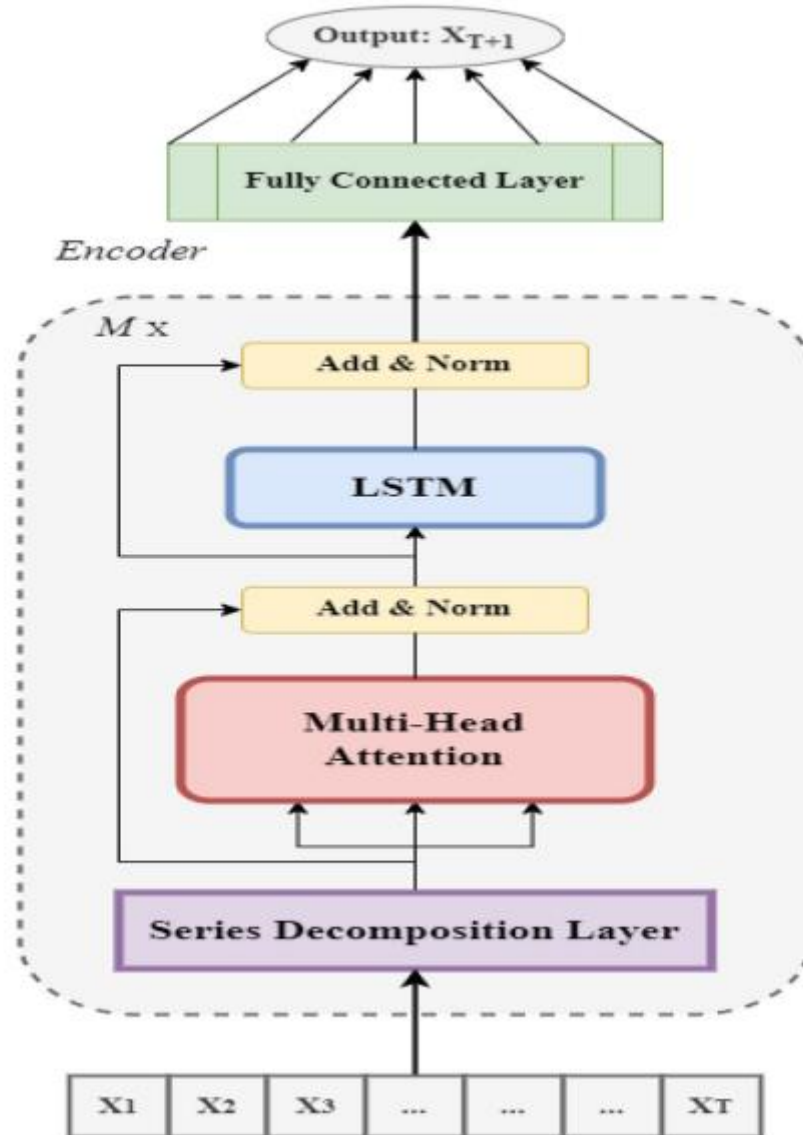


Рисунок 1.3.1.1. Архитектура Helformer с блоками (Series Decomposition) → (Multi-Head Attention) → (LSTM) → (Dense).

1.3.2. Модель FEDformer

Рассматривается унвариативный временной ряд $\{X_t\}_{t=1}^T$. Для заданных длины окна L и горизонта прогноза H на шаге t формируется вход

$$\mathbf{x}_t = (X_{t-L+1}, \dots, X_t) \in \mathbb{R}^L,$$

и требуется предсказать

$$\hat{\mathbf{x}}_t^{(\text{out})} = (\hat{X}_{t+1}, \dots, \hat{X}_{t+H}).$$

FEDformer реализует encoder–decoder архитектуру, в которой i на каждом слое выполняется сезонно-трендовая декомпозиция, а i блоки внимания заменены частотными операторами (Fourier / Wavelet), что обеспечивает глобальный обзор зависимостей и снижает вычислительную сложность по сравнению с $O(L^2)$ self-attention.

Общая структура FEDformer показана на рисунке 1.3.2.1.

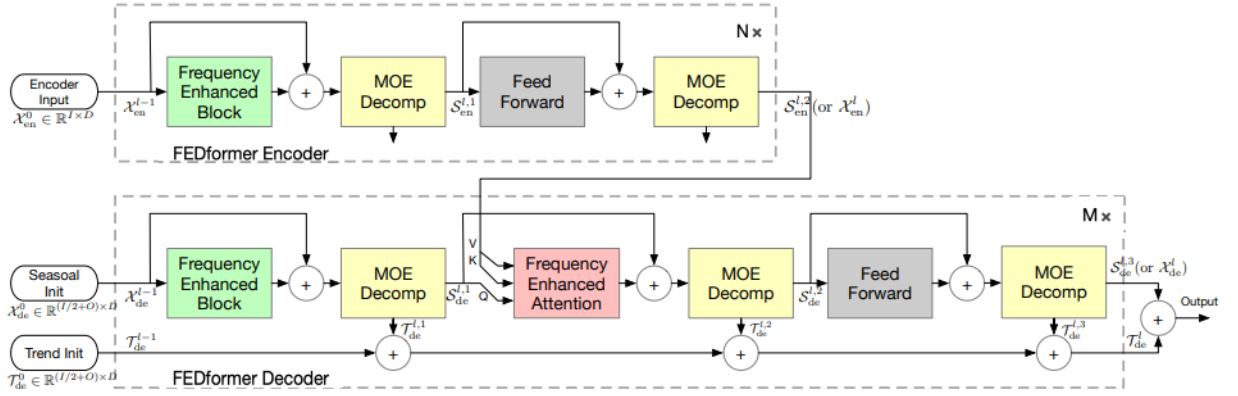


Рисунок 1.3.2.1. Общая структура FEDformer.

1) Глубокая декомпозиция (MOEDecom)

Внутренний блок MOEDecomп раскладывает входную последовательность скрытых представлений $U \in \mathbb{R}^{L \times D}$ на сезонную и трендовую части:

$$(U^{\text{seas}}, U^{\text{trend}}) = \text{MOEDecom}(U), \quad U^{\text{seas}} = U - U^{\text{trend}}.$$

Тренд U^{trend} строится как смесь нескольких сглаживаний (moving average) с различными окнами:

$$U^{\text{trend}} = \sum_{k \in \mathcal{K}} \pi_k(U) \text{MA}_k(U), \quad \pi(U) = \text{softmax}(G(U)).$$

Здесь $\{\text{MA}_k\}$ — набор усредняющих фильтров, $G(\cdot)$ — обучаемый «gating»-модуль, выбирающий веса смеси. Такая декомпозиция выполняется в каждом слое и стабилизирует обучение: encoder преимущественно передаёт сезонно-остаточную компоненту, а decoder накапливает тренд.

2) Частотные блоки: Fourier Enhanced Block и Fourier Enhanced Attention

В FEDformer-f стандартные self-/cross-attention заменены на операции в частотной области с выбором ограниченного числа мод $M \ll L$.

Fourier Enhanced Block (FEB-f) для входа $x \in \mathbb{R}^{L \times D}$:

$$\begin{aligned} q &= xW, \quad Q = \mathcal{F}(q), \\ \tilde{Q} &= \text{Select}(Q) \in \mathbb{C}^{M \times D}, \\ \tilde{Y}_{m,:} &= \tilde{Q}_{m,:} R_{:,m}, \quad R \in \mathbb{C}^{D \times D \times M}, \\ \text{FEB-f}(x) &= \mathcal{F}^{-1}(\text{padding}(\tilde{Y})) \end{aligned}$$

То есть: DFT → выбор M мод → обучаемый частотный оператор → обратное DFT.

На рисунке 1.3.2.2 показана структура блока FEB-f.

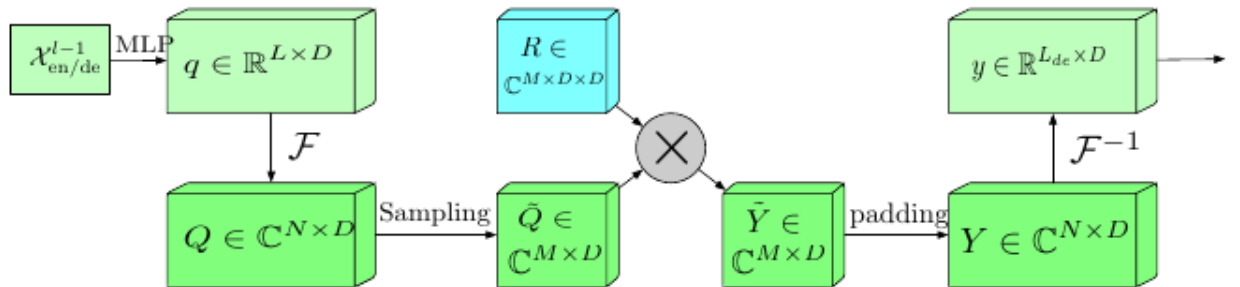


Рисунок 1.3.2.2. Структура блока с частотным усилением и преобразованием Фурье (FEB-f).

Fourier Enhanced Attention (FEA-f) реализует cross-взаимодействие в частотной области. Для q, k, v :

$$\begin{aligned}\tilde{Q} &= \text{Select}(\mathcal{F}(q)), & \tilde{K} &= \text{Select}(\mathcal{F}(k)), & \tilde{V} &= \text{Select}(\mathcal{F}(v)), \\ \tilde{Y} &= \sigma(\tilde{Q}\tilde{K}^\top) \tilde{V}, & \text{FEA-f}(q, k, v) &= \mathcal{F}^{-1}(\text{Padding}(\tilde{Y})).\end{aligned}$$

Функция $\sigma(\cdot)$ — нелинейность (в т.ч. softmax), применяемая к матрице подобия в частотной области.

На рисунке 1.3.2.3 показана структура блока FEA-f.

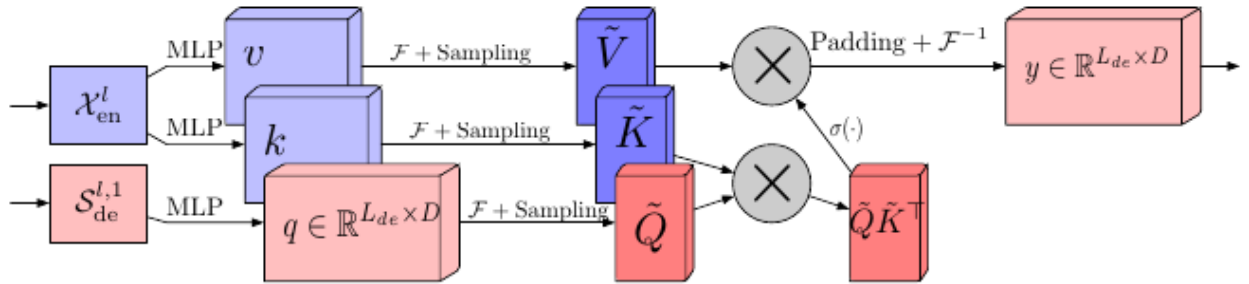


Рисунок 1.3.2.3. Структура блока FEA – f.

Wavelet-ветка (FEDformer-w)

В FEDformer-w преобразование Фурье заменяется многошкальной вейвлет-декомпозицией: вход x раскладывается на низко- и высокочастотные компоненты на нескольких масштабах, затем к компонентам применяются аналоги FEB/FEA, после чего выполняется реконструкция (inverse wavelet). Это даёт локализацию и по времени, и по частоте (рисунок 1.3.2.4).

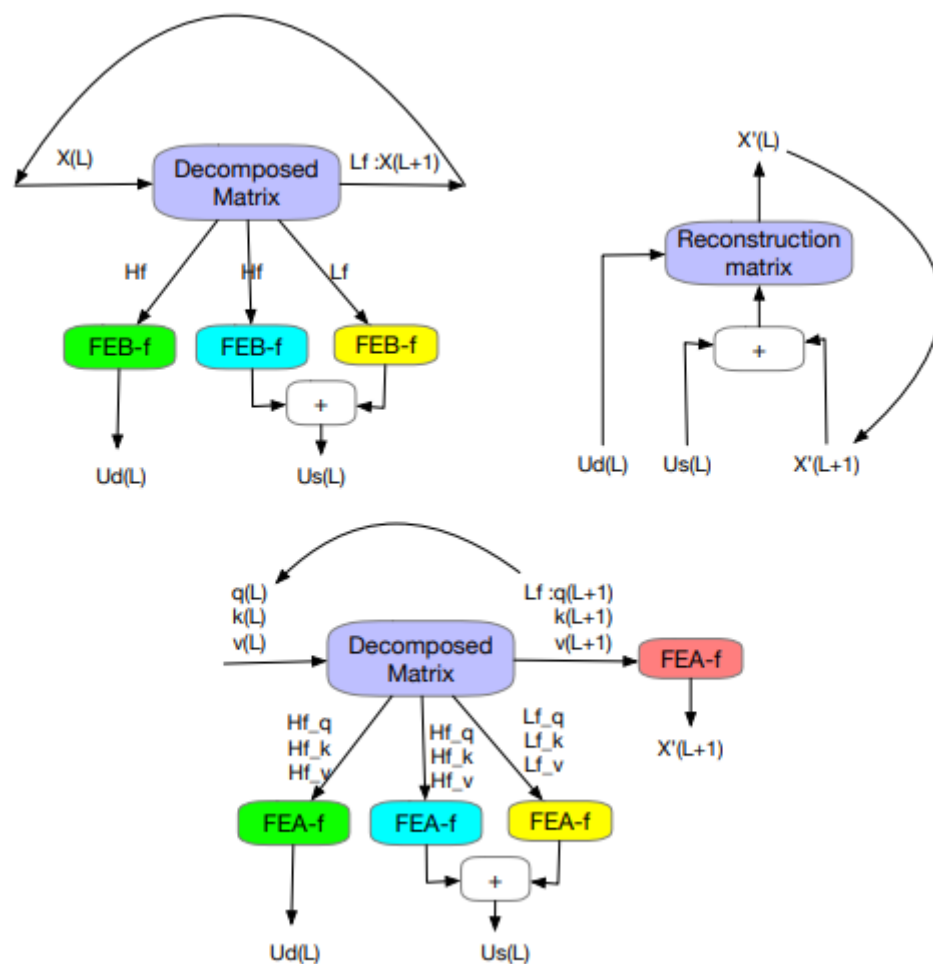


Рисунок 1.3.2.4. Этап декомпозиции в вейвлетном частотно-усиленном блоке, реконструкции вейвлетного блока, общий для модулей **FEB-w** и **FEA-w** и декомпозиции в вейвлетном частотно-усиленном модуле кросс-взаимодействия.

Глава 2. Практическая часть

2.1. Описание параметров и архитектур моделей с практической точки зрения

Для выполнения практической части были реализованы следующие классические статистические методы и архитектуры:

2.1.1. ARIMA - один статистический блок ARIMA(p,d,q) на исходном ряде.

Параметры архитектуры:

$order = (p, d, q)$, по умолчанию (1,1,1).

- В *statsmodels.ARIMA*:
 - $trend = t$ при $d > 0$, иначе "c" (дрейф через линейный тренд при дифференцировании).
 - $enforce_stationarity = False, enforce_invertibility = False$.
 - $maxiter = 200$.

Примечание:

В *ARIMA*, а также в *ARIMA* с автоподбором была специально добавлена защита от вырожденного прогноза:

- Если ряд слишком короткий (< 6) $\rightarrow$ линейная экстраполяция.
- Если прогноз "плоский" ($std \approx 0$), но у последних разностей ряда std не нулевая $\rightarrow$ линейная экстраполяция (борьба с деградацией/сдвигом к константе).
- При ошибках: перебор альтернативных порядков ((0,1,1), (1,1,0), (0,1,0), (1,0,1)), затем линейная экстраполяция.

2.1.2. ARIMA-auto

Модель с автоподбором - ARIMA-блок, но порядок (p,q) выбирается эвристикой по ACF/PACF.

Параметры автоподбора:

- $d = 1$ (по умолчанию), ограничения: $max_p = 3, max_q = 3$.
- $max_lag = \min(24, N//2)$.
- Порог значимости: $crit = 1.96 / \sqrt{N}$; выбирается **максимальный** значимый лаг $PACF \rightarrow p, ACF \rightarrow q$.
- Если данных мало ($N < 10$) или ACF/PACF недоступны $\rightarrow fallback (1, d, 0)$.

Логика защиты от отказа подбора:

- Если автоподбор падает $\rightarrow$ мини grid-search по AIC в окрестности: $d \in \{0,1\}, p, q \in \{0,1,2\}$, затем ARIMA с лучшим AIC.

- По аналогии с ARIMA: защита от «плоского» прогноза $\rightarrow$ линейная экстраполяция.

2.1.3. ETS

Параметры архитектуры:

- *trend* = *add* (по умолчанию).
- Сезонность включается **только если** *seasonal_periods* > 1, иначе *seasonal* = *None*.
- Обучение: *fit(optimized = True, use_brute = True)*.

Откат: если ряд короткий (< 4) или ошибка обучения $\rightarrow$ повтор последнего значения.

2.1.4. Prophet с аддитивной моделью и без встроенных календарных сезонностей

Параметры архитектуры:

- *seasonality_mode* = *additive*.
- *yearly_seasonality* = *False*, *weekly_seasonality* = *False*, *daily_seasonality* = *False*.
- Индекс времени создаётся искусственно: старт 2000 – 01 – 01, частота *freq* = "D" по умолчанию.

Откат: при ошибке или слишком коротком ряде → повтор последнего.

2.1.5. *HelFormer* - нейросетевой блок

Self-Attention + *LSTM*, прогноз 1 шаг (многошаговый строится итеративно).

Параметры архитектуры:

$lookback = L$ (запоминающее окно), по умолчанию 30.

B TensorFlow/Keras:

$$\begin{aligned} & \text{Input}(L, 1) \rightarrow (\times \text{num_blocks}) [\text{LayerNormalization}(\text{eps} = 1e^{-6}) \\ & \quad + \text{MultiHeadAttention}(\text{num_heads} = \text{num_heads}, \\ & \quad \text{key_dim} = \text{head_size}, \text{dropout} = \text{dropout}) + \text{residual}] \\ & \rightarrow \text{LSTM}(\text{units} = \text{units}, \text{activation} = \text{mish}) \rightarrow \text{Dense}(1). \end{aligned}$$

Параметры по умолчанию:

- $num_blocks = 4$,
- $num_heads = 4$,
- $head_size = 56$,
- $dropout \approx 0.1127$,
- $units = 25$.

Параметры обучения:

- *optimizer = Adam(lr = helformer_{lr})*
- *loss = MSE*
- *epochs*
- *batch_size*
- *verbose*

Окна: $X[t - L:t] \rightarrow y[t]$ (одношаговое), обучение с нуля для каждого ряда.

Предобработка (опционально, `helformer_use_hw = True`):

B statsmodels.ExponentialSmoothing:

- $trend = hw_trend$
- $seasonal = hw_seasonal$
- $seasonal\ periods = P$ если $P > 1$,

- обучение $fit(optimized = True, use_brute = True)$.

Сеть обучается на $ratio = \frac{y}{baseline}$ (защита: $|baseline| < 1e^{-8} \rightarrow 1.0$),
финальный прогноз: $forecast = ratio_pred * baseline_forecast$.
 Если выбран "mul", но есть $y \leq 0 \rightarrow$ переключение проблемных компонент на "add".
Нормализация:
 MinMax по train (если $max - min \approx 0 \rightarrow scale = 1$).

Примечание/защиты:

Режимы прогноза (helformer_forecast_mode): test_windows (teacher forcing), rollout (recursive), rollout_full (полная траектория, берутся последние H).

Короткие ряды: левый padding; в M4 дополнительно авто-уменьшение lookback, чтобы было ≥ 8 обучающих окон.

Если HW baseline не строится $\rightarrow$ работа по исходному ряду; NaN/Inf предсказания $\rightarrow$ подстановка последнего значения.

В тестах:

M3-конфиг: epochs=100, batch_size=32, lr $\approx$ 3.846e-4, num_blocks=4, num_heads=4, lookback=30, use_hw=True.

M4-конфиг: epochs=100, batch_size=64, lr=0.009, num_blocks=8, num_heads=8, lookback=30, use_hw=True, forecast_mode=rollout_full

2.2. Применение и диагностика моделей на синтетических рядах

Диагностика и разработка моделей велась на синтетических рядах. Синтетический ряд строится, как аддитивная модель:

$$y(t) = 10.0 + (trend_{slope} \cdot t) + season(t) + \varepsilon_t, \quad \varepsilon_t \sim N(0, noise_std)$$

Сезонность включалась только если $season_period > 1$ и $season_amp > 0$:

$$season(t) = season_amp \cdot \sin\left(\frac{2\pi t}{season_period}\right)$$

2.2.1. Используемые профили (ряды для испытаний)

- flat_low_noise (рисунок 2.2.1)
 - trend_slope = 0.0
 - season_period = None
 - season_amp = 0.0
 - noise_std = 0.1

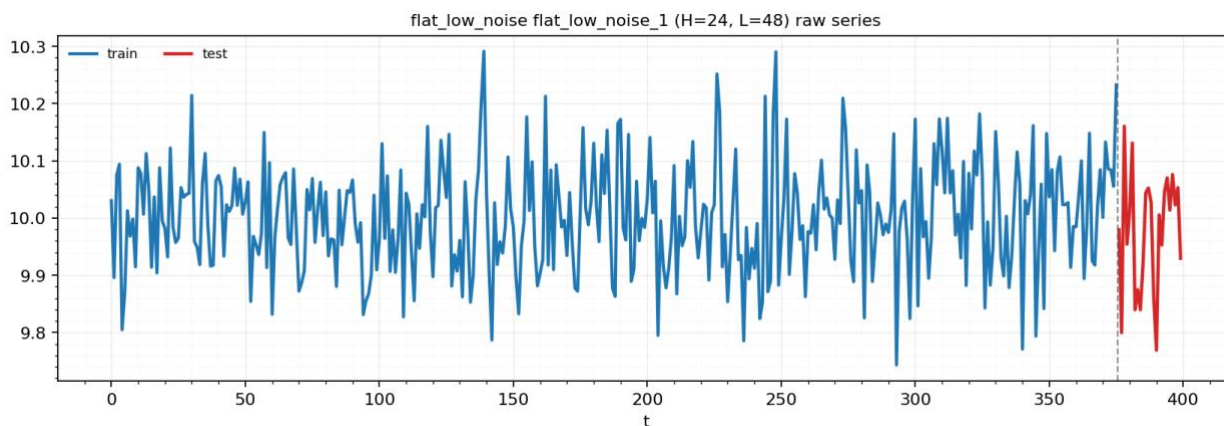


Рисунок 2.2.1. Синтетический ряд *flat_low_noise_1*, разделенный на *test* и *train*.

- *trend_only* (рисунок 2.2.2)
 - *trend_slope* = 0.02
 - *season_period* = None
 - *season_amp* = 0.0
 - *noise_std* = 0.1

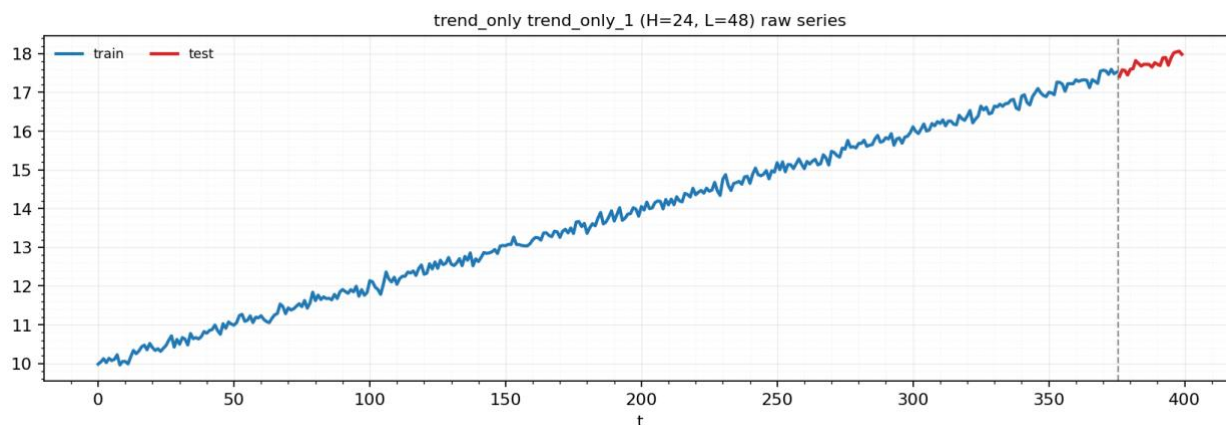


Рисунок 2.2.2. Синтетический ряд *trend_only_1*, разделенный на *test* и *train*.

- *season_only* (рисунок 2.2.3)
 - *trend_slope* = 0.0
 - *season_period* = 24
 - *season_amp* = 1.0
 - *noise_std* = 0.1

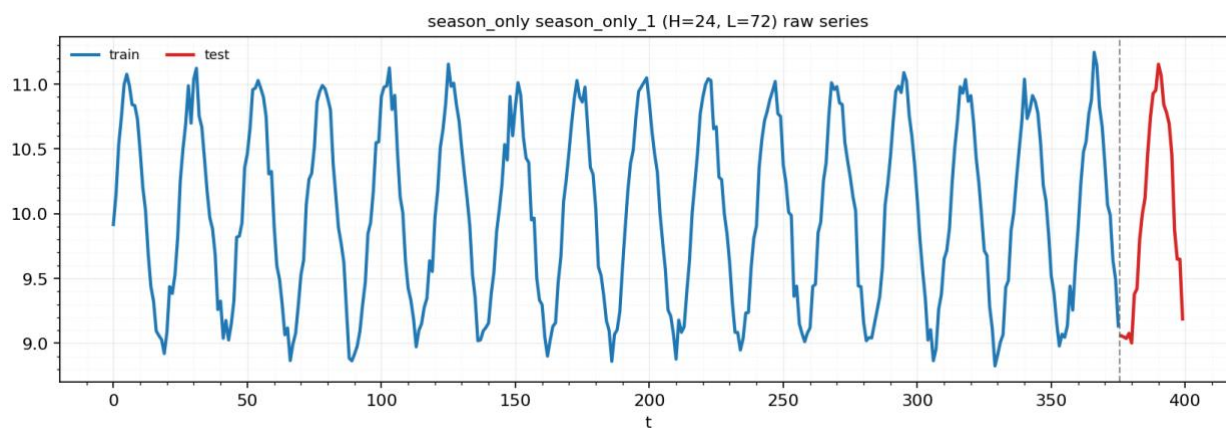


Рисунок 2.2.3. Синтетический ряд *season_only_1*, разделенный на *test* и *train*.

- *trend_season* (рисунок 2.2.4)
 - *trend_slope* = 0.02
 - *season_period* = 24
 - *season_amp* = 1.0
 - *noise_std* = 0.1

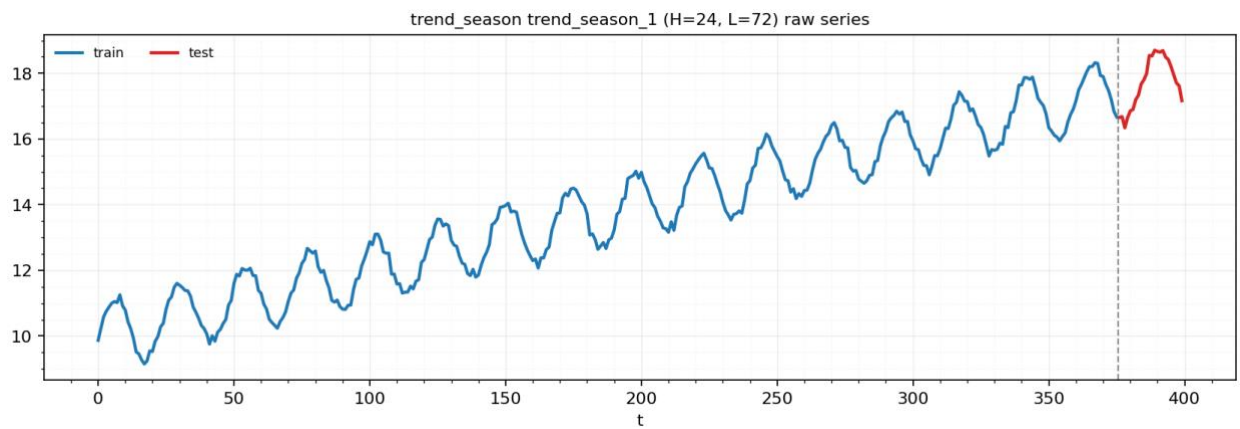


Рисунок 2.2.4. Синтетический ряд *trend_season_1*, разделенный на *test* и *train*.

- *trend_season_high_noise* (рисунок 2.2.5)
 - $\text{trend_slope} = 0.02$
 - $\text{season_period} = 24$
 - $\text{season_amp} = 1.0$
 - $\text{noise_std} = 0.5$

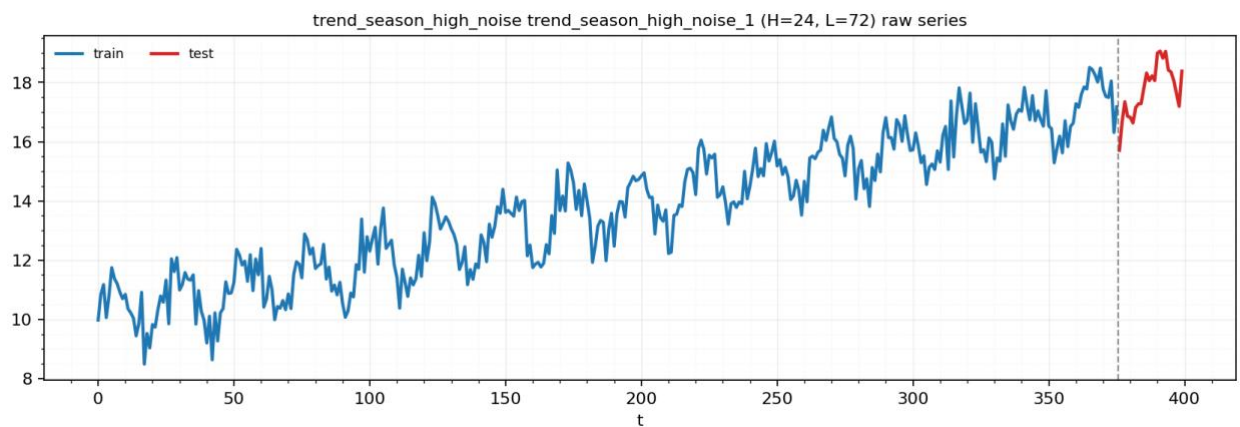


Рисунок 2.2.5. Синтетический ряд *trend_season_high_noise_1*, разделенный на *test* и *train*.

- *season_high_noise* (рисунок 2.2.6)
 - $\text{trend_slope} = 0.0$
 - $\text{season_period} = 24$
 - $\text{season_amp} = 1.0$
 - $\text{noise_std} = 0.5$

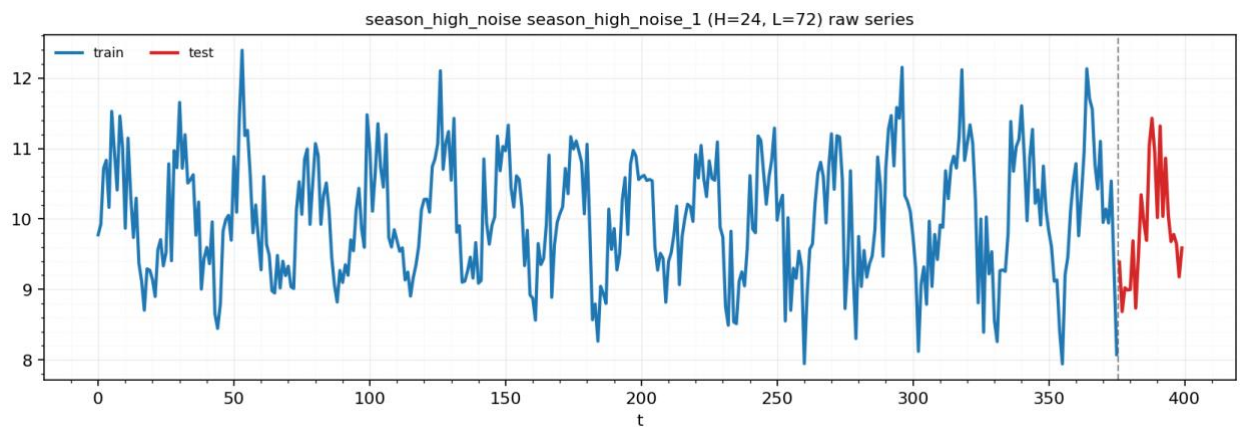


Рисунок 2.2.6. Синтетический ряд *season_high_noise_1*, разделенный на *test* и *train*.

2.2.2. Обучение моделей, подбор параметров ARIMA

Модели были обучены со следующими параметрами:

- горизонт: $horizon = 24$
- $L = best_L(y_{tr}, horizon, per)$, где $per = season_period$ или 1.
- $epochs = 8$
- $batch_size = 64$
- $lr = 5e^{-4}$
- $weight_decay = 1e^{-4}$
- $clip = 1.0$

Для каждого ряда были построены графики ACF / PACF, чтобы рассчитать порядки моделей *ARIMA* для этих рядов. За основу было взято правило:

- $d = 1$ (фиксировано),
- ищем значимые лаги по ACF/PACF только до 24 ($max_lag = 24$),
- ограничиваем $p \leq 3, q \leq 3$,
- p берём по *PACF*, q по *ACF* (по максимальному значимому лагу).

Итак, порядки моделей *ARIMA* для ряда:

- ***flat_low_noise_1*** (рисунок 2.2.7):

ACF/PACF почти весь в границах $\rightarrow$ ряд близок к белому шуму вокруг уровня $\rightarrow$ ACF/PACF в пределах значимости на лагах $1 \dots 24 \rightarrow p = 0, q = 0 \rightarrow \mathbf{ARIMA(0,1,0)}$

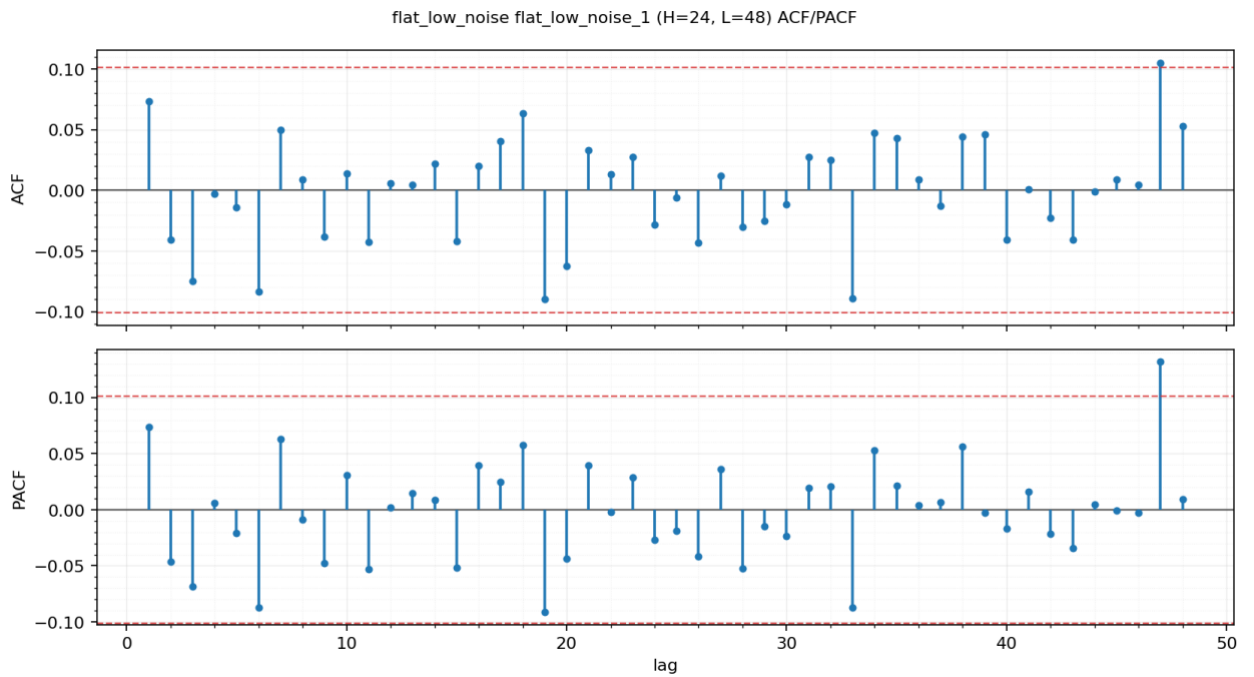


Рисунок 2.2.7. График ACF/PACF (*flat_low_noise_1*) для подбора параметров ARIMA.

- **trend_only_1 (рисунок 2.2.14):**

ACF долго остаётся высокой (медленное убывание) → сильная нестационарность, оправдывает $d = 1$; *PACF* выделяет короткую структуру на малых лагах → *PACF*: сильный пик на лаге 1, дальше около нуля → $p \approx 1$; *ACF* “длинная” из-за тренда, но по введенной эвристике q не обязателен → **ARIMA(1,1,0)**

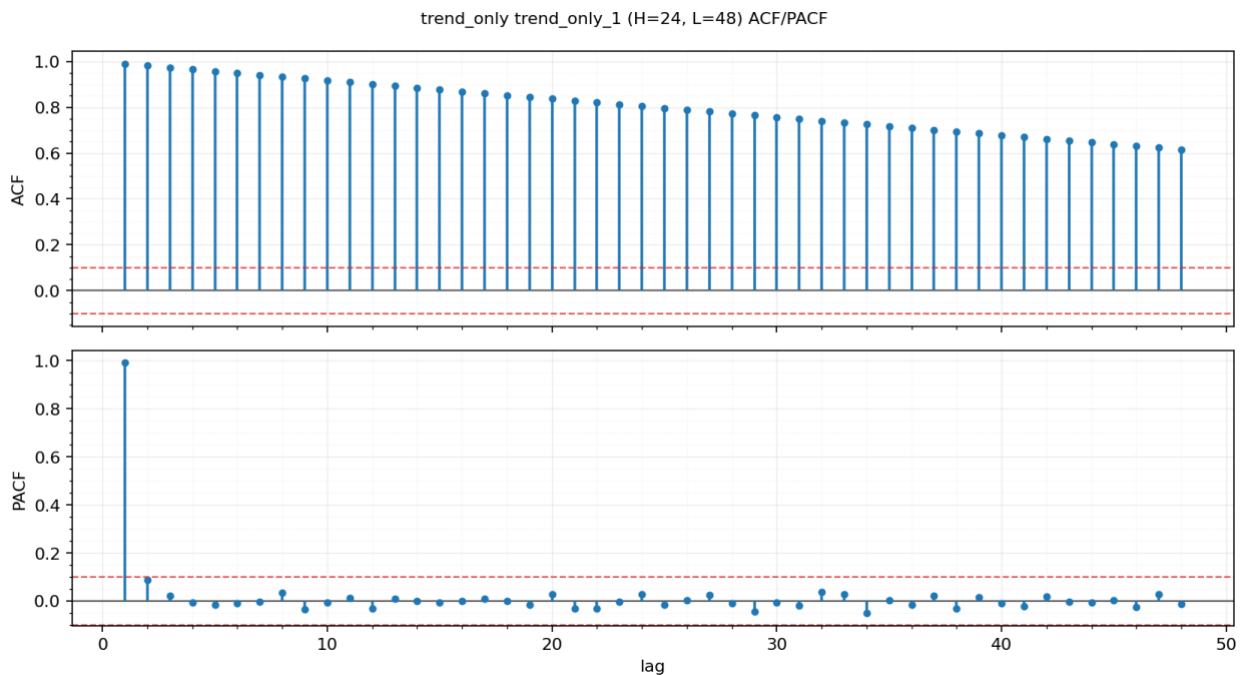


Рисунок 2.2.8. График ACF/PACF (*trend_only_1*) для подбора параметров ARIMA.

- *season_only_1* (рисунок 2.2.9):

ACF выраженно периодична → сезонность с периодом около 24; PACF имеет сильный вклад на малых лагах и слабее — на сезонных → PACF тоже имеет значимые лаги $> 3 \rightarrow p = 3 \rightarrow \text{ARIMA}(3,1,3)$

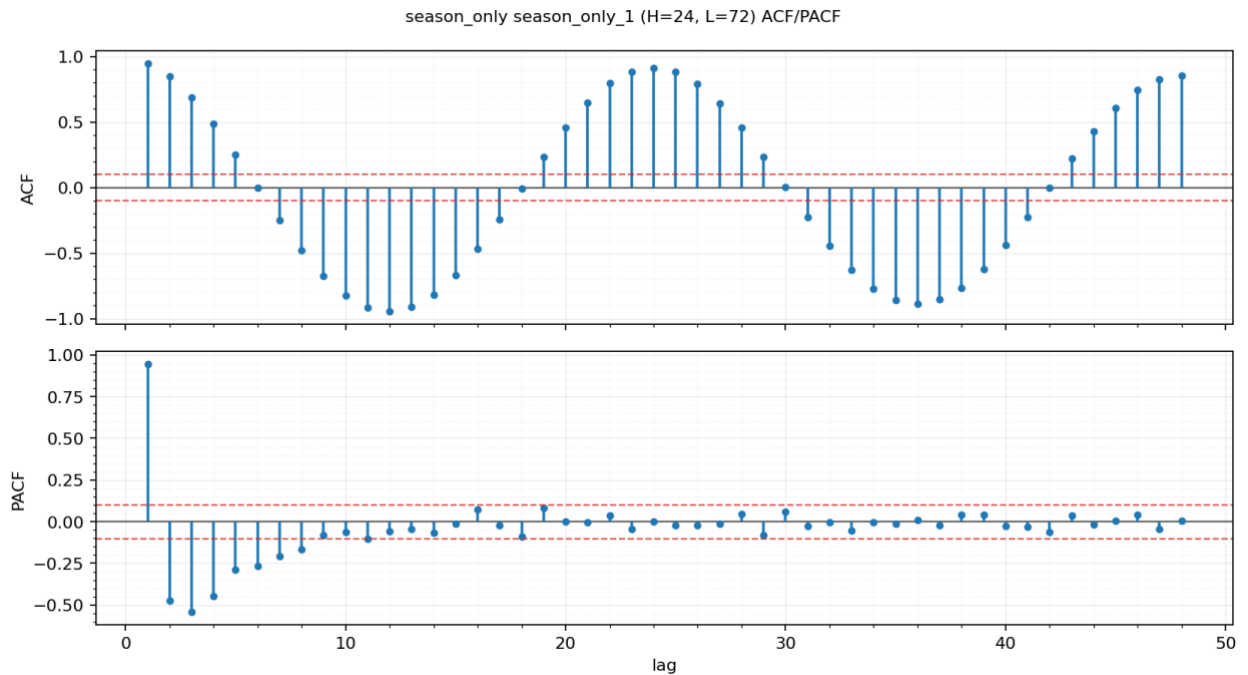


Рисунок 2.2.9. График ACF/PACF (*season_only_1*) для подбора параметров ARIMA.

- *season_high_noise_1* (рисунок 2.2.10)

тот же сезонный рисунок, но пики размыты шумом (амплитуда корреляций ниже, больше случайных колебаний) → та же сезонность, но шумнее → всё равно есть значимые лаги $> 3 \rightarrow p=q=3 \rightarrow \text{ARIMA}(3,1,3)$

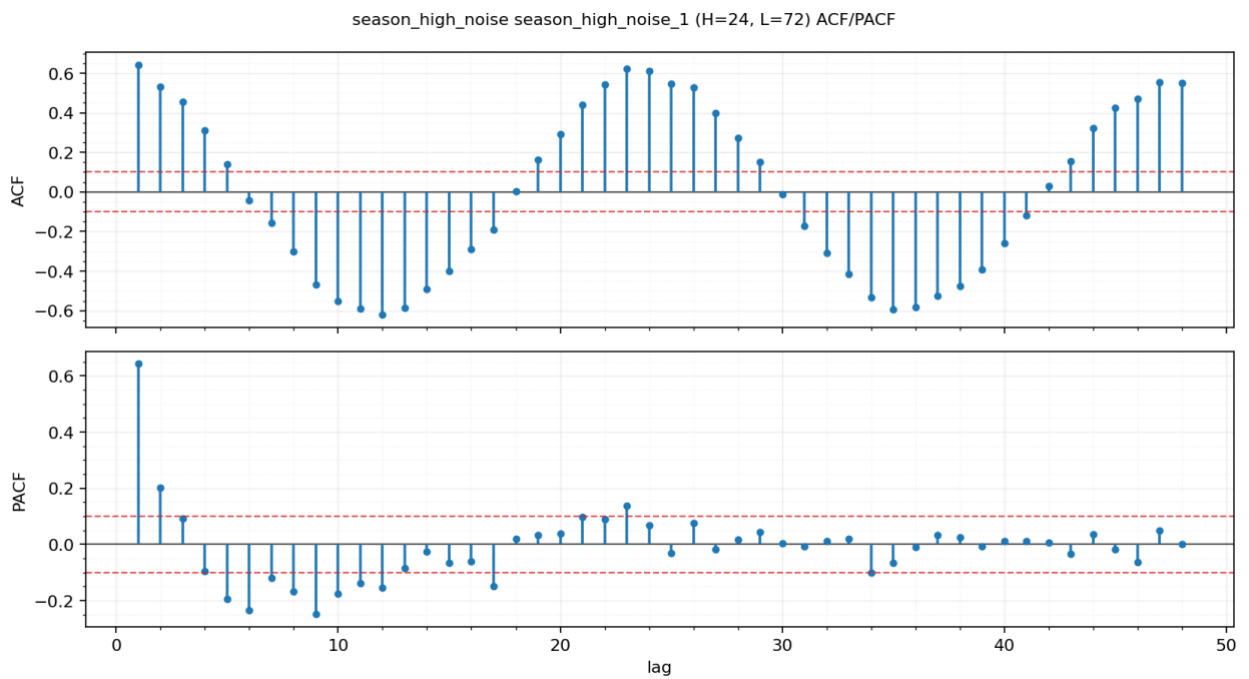


Рисунок 2.2.10. График ACF/PACF (*season_high_noise_1*) для подбора параметров ARIMA.

- *trend_season / trend_season_high_noise* (рисунок 2.2.11 и 2.2.12):

смесь двух эффектов: высокий фон ACF из-за тренда и периодическая структура из-за сезонности; при *high_noise* периодичность менее контрастна → смешение тренда и сезонности даёт значимые лаги по ACF/PACF до 24 → **ARIMA(3,1,3)**

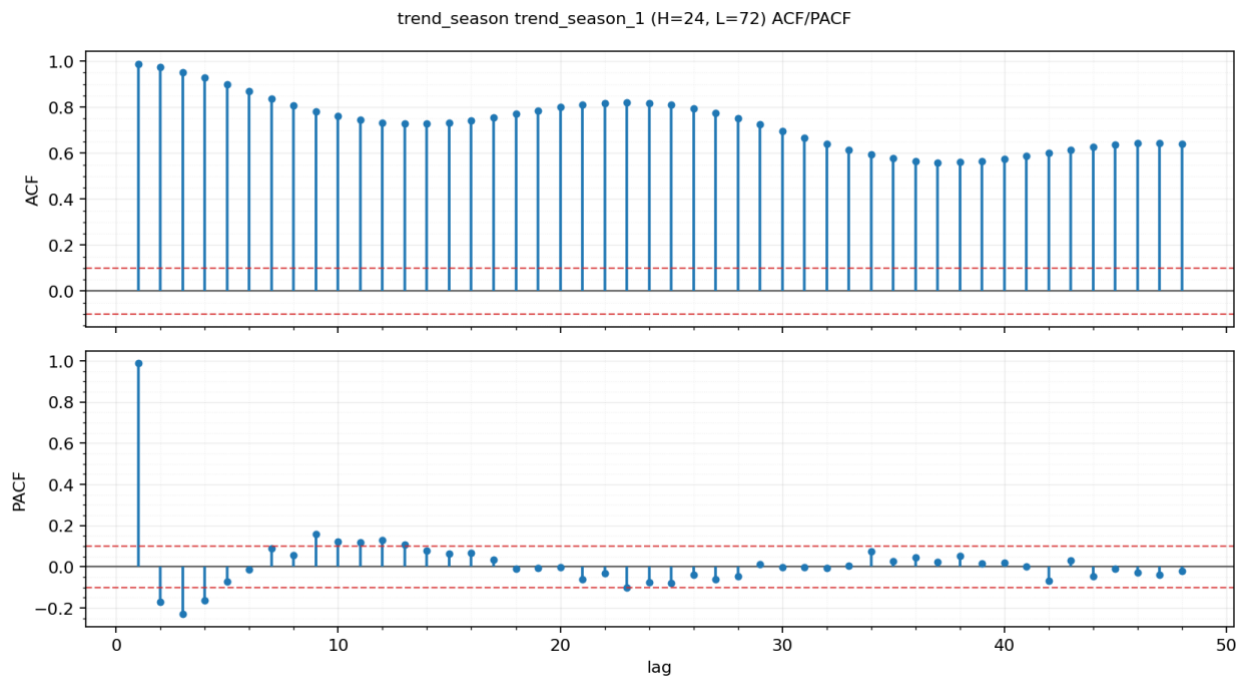


Рисунок 2.2.11. График ACF/PACF (*trend_season_1*) для подбора параметров ARIMA.

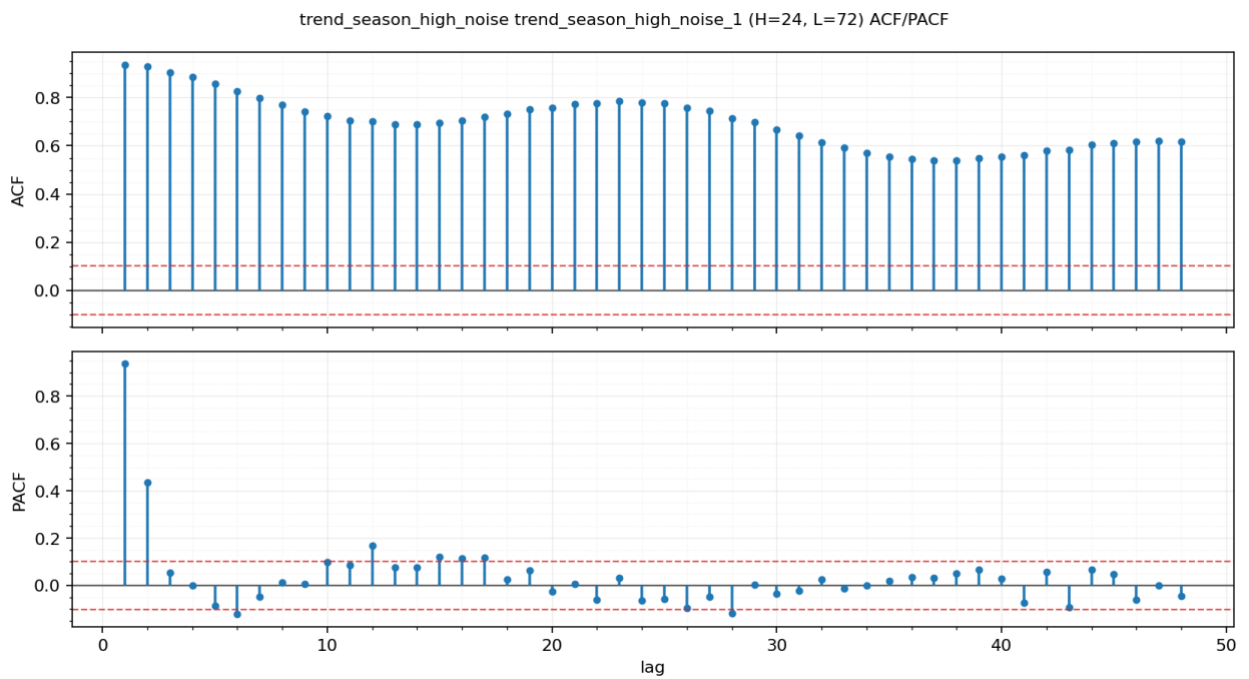


Рисунок 2.2.12. График ACF/PACF (*trend_season_high_noise_1*) для подбора параметров ARIMA.

Так как первый слой сети *Helformer* представляет собой *Holt-Winters* декомпозицию, было принято решение протестировать выходные значения данного слоя на предмет валидного разложения на тренд, сезонность и остатки. Результаты декомпозиции и предсказания до применения трансформерного слоя для рядов *trend_season_1* и *trend_season_high_noise_1* приведены на рисунках 2.2.13 и 2.2.14 соответственно.

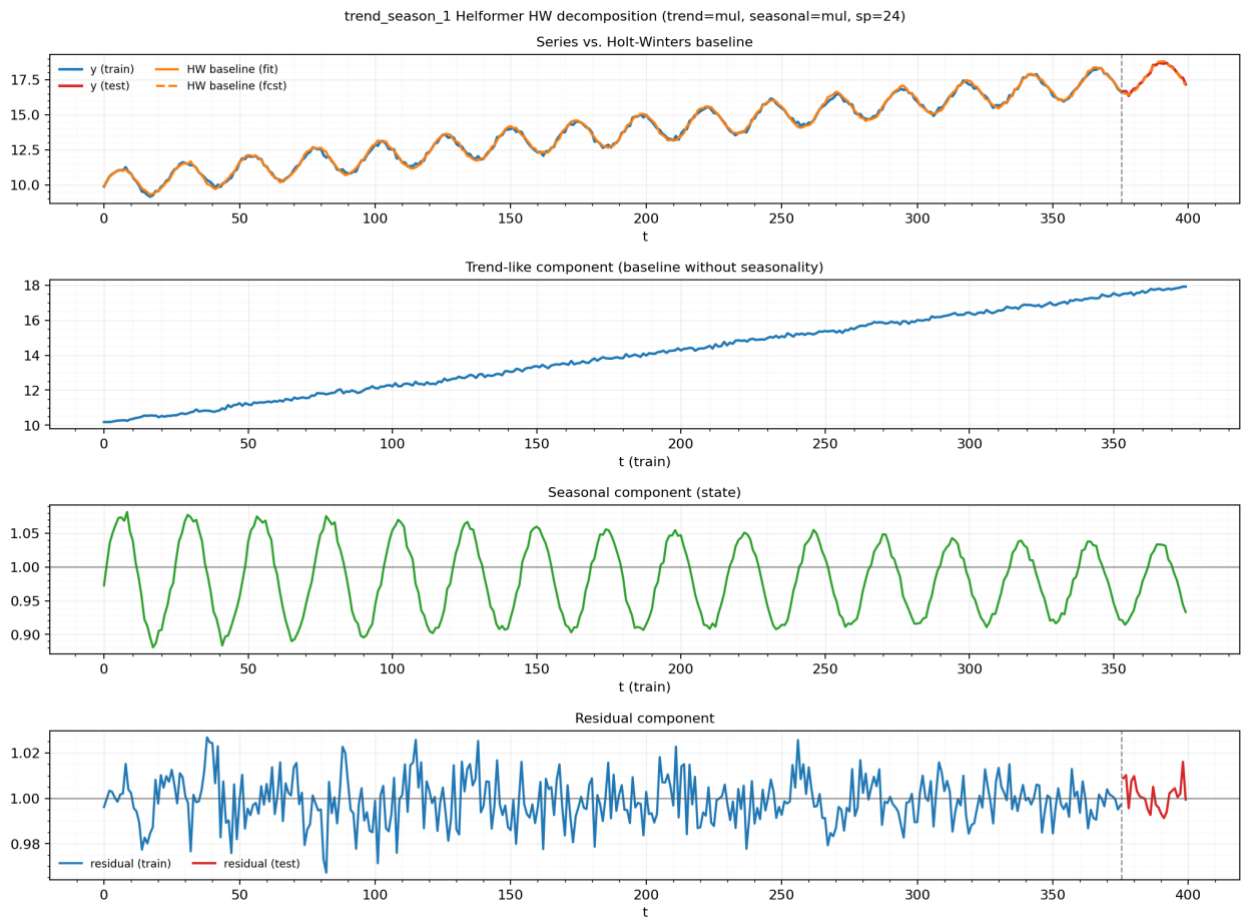


Рисунок 2.2.13. Графики компонент *Holt-Winters* декомпозиции для синтетического ряда *trend_season_1*.

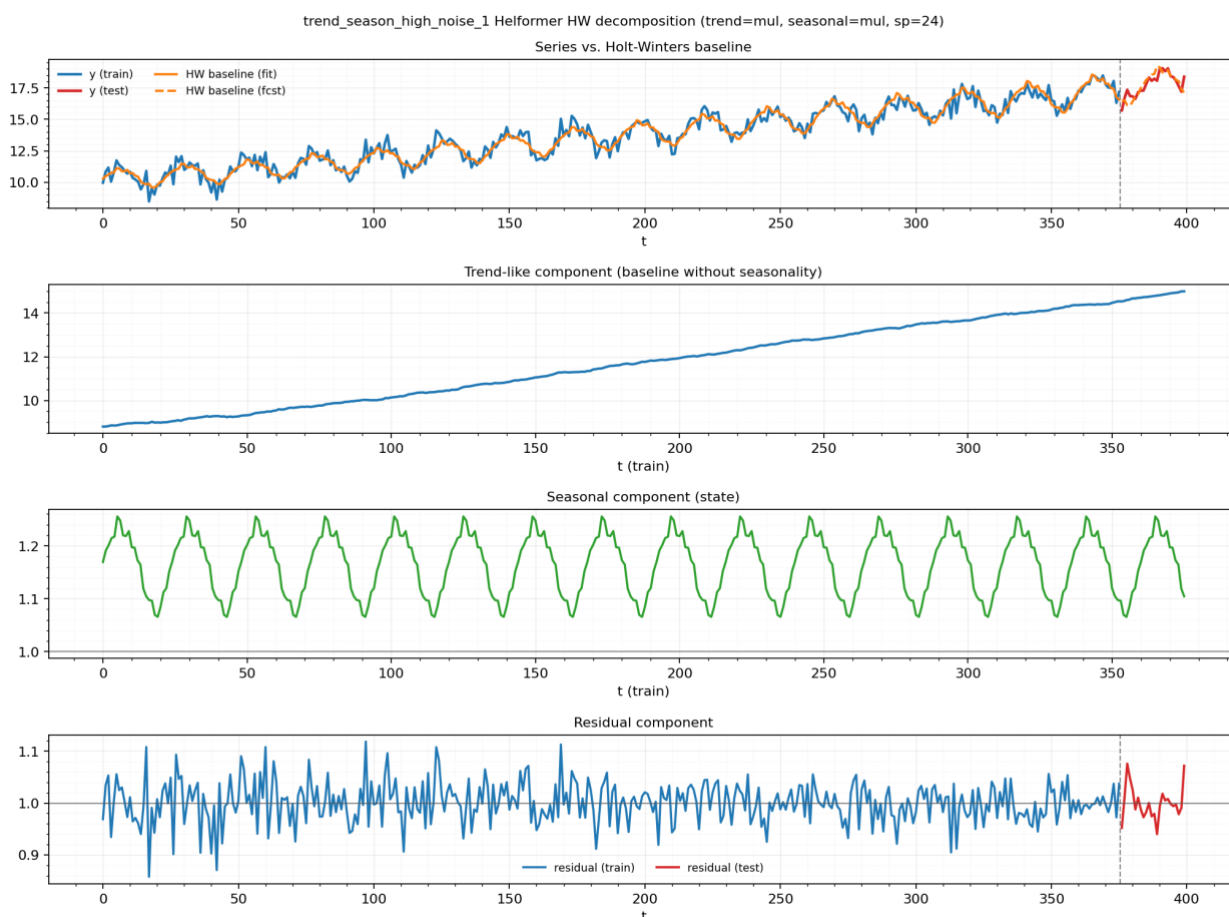


Рисунок 2.2.14. Графики компонент *Holt-Winters* декомпозиции для синтетического ряда *trend_season_high_noise_1*.

2.2.3. Результаты обучения моделей на синтетических рядах

Каждая из перечисленных выше моделей была обучена на тренировочных частях данных для каждого отдельного ряда, после чего итеративным путем были получены предсказания для каждого соответствующего синтетического ряда и модели.

Увеличенные графики предсказаний моделей *ARIMA* *auto-ARIMA*, *ETS*, *Prophet* и *Helformer* для рядов *flat_low_noise*, *season_high_noise*, *season_only*, *trend_only*, *trend_season* и *trend_season_high_noise*, а также соответствующие метрики качества приведены на рисунках 2.2.15 - 2.2.20 и в таблицах 2.2.1- 2.2.6 соответственно.

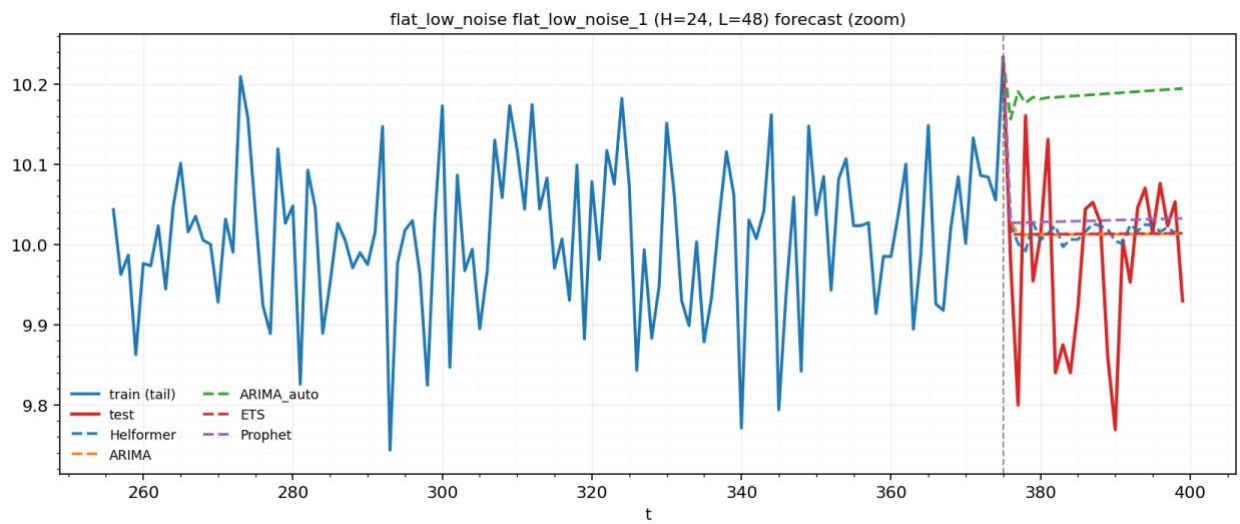


Рисунок 2.2.15. Предсказания реализованных гибридных и классических моделей на синтетическом ряду *flat_low_noise_1*.

Таблица 2.2.1. Результат по метрикам на синтетическом ряду *flat_low_noise_1*.

<i>Модель</i>	<i>sMAPE</i>	<i>MAPE</i>	<i>RMSE</i>	<i>MSE</i>
<i>ARIMA</i>	0.836555	0.840505	0.108148	0.011696
<i>ARIMA_auto</i>	2.088459	2.115946	0.233604	0.054571
<i>ETS</i>	0.829870	0.833827	0.107988	0.011661
<i>Prophet</i>	0.862866	0.868193	0.114499	0.013110
<i>Helformer</i>	0.824628	0.828383	0.107447	0.011545

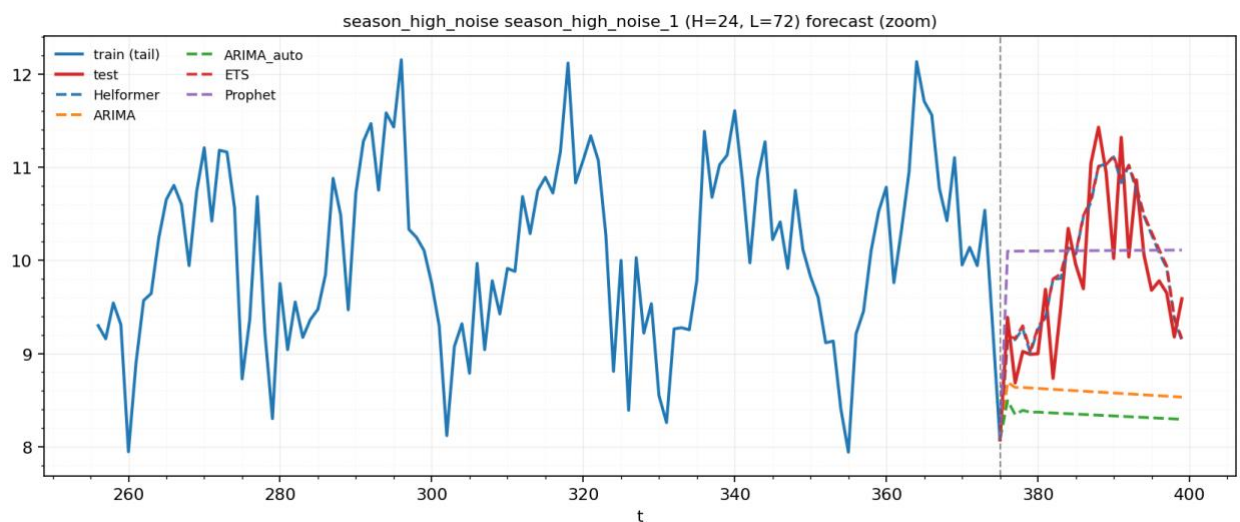


Рисунок 2.2.16. Предсказания реализованных гибридных и классических моделей на синтетическом ряду *season_high_noise_1*.

Таблица 2.2.2. Результат по метрикам на синтетическом ряду *season_high_noise_1*.

<i>Модель</i>	<i>sMAPE</i>	<i>MAPE</i>	<i>RMSE</i>	<i>MSE</i>
<i>ARIMA</i>	13.407610	12.312916	1.491867	2.225666
<i>ARIMA auto</i>	16.304866	14.836266	1.705509	2.908761
<i>ETS</i>	4.129768	4.230026	0.506289	0.256329
<i>Prophet</i>	6.947497	7.103944	0.811763	0.658960
<i>Helformer</i>	4.112161	4.204635	0.503587	0.253600

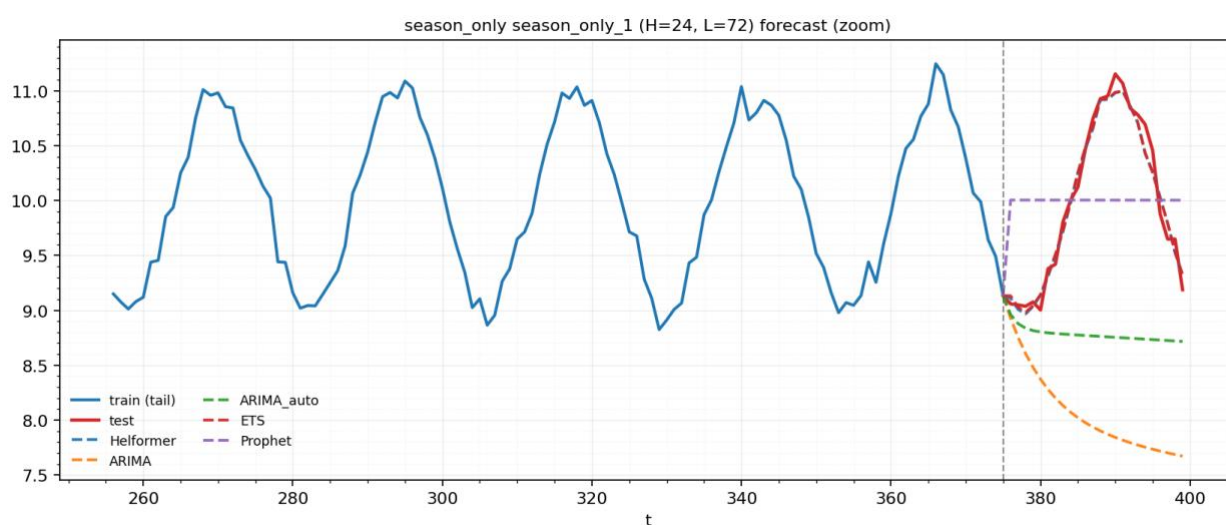


Рисунок 2.2.17. Предсказания реализованных гибридных и классических моделей на синтетическом ряду *season_only_1*.

Таблица 2.2.3. Результат по метрикам на синтетическом ряду *season_only_1*.

<i>Модель</i>	<i>sMAPE</i>	<i>MAPE</i>	<i>RMSE</i>	<i>MSE</i>
<i>ARIMA</i>	21.622234	19.084701	2.216728	4.913881
<i>ARIMA auto</i>	12.909242	11.875963	1.462063	2.137629
<i>ETS</i>	0.901212	0.900314	0.110802	0.012277
<i>Prophet</i>	6.675052	6.698519	0.744003	0.553540
<i>Helformer</i>	0.928345	0.927072	0.111921	0.012526

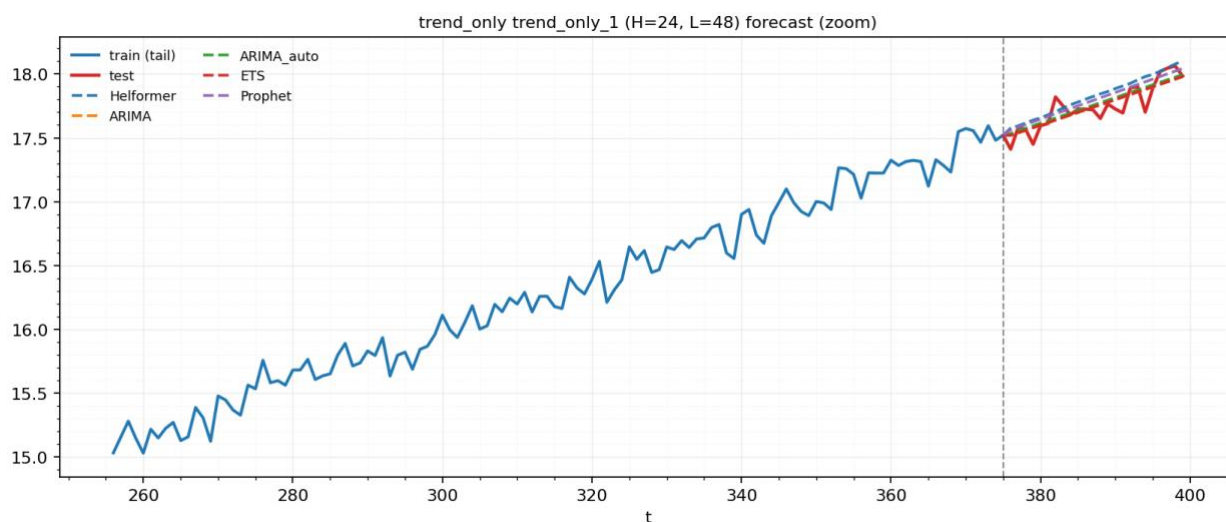


Рисунок 2.2.18. Предсказания реализованных гибридных и классических моделей на синтетическом ряду *trend_only_1*.

Таблица 2.2.4. Результат по метрикам на синтетическом ряду *trend_only_1*.

<i>Модель</i>	<i>sMAPE</i>	<i>MAPE</i>	<i>RMSE</i>	<i>MSE</i>
<i>ARIMA</i>	0.361398	0.361489	0.085027	0.007230
<i>ARIMA_auto</i>	0.372839	0.373262	0.086320	0.007451
<i>ETS</i>	0.360341	0.360446	0.084873	0.007203
<i>Prophet</i>	0.445540	0.446827	0.100227	0.010045
<i>Helformer</i>	0.532853	0.534919	0.118749	0.014101

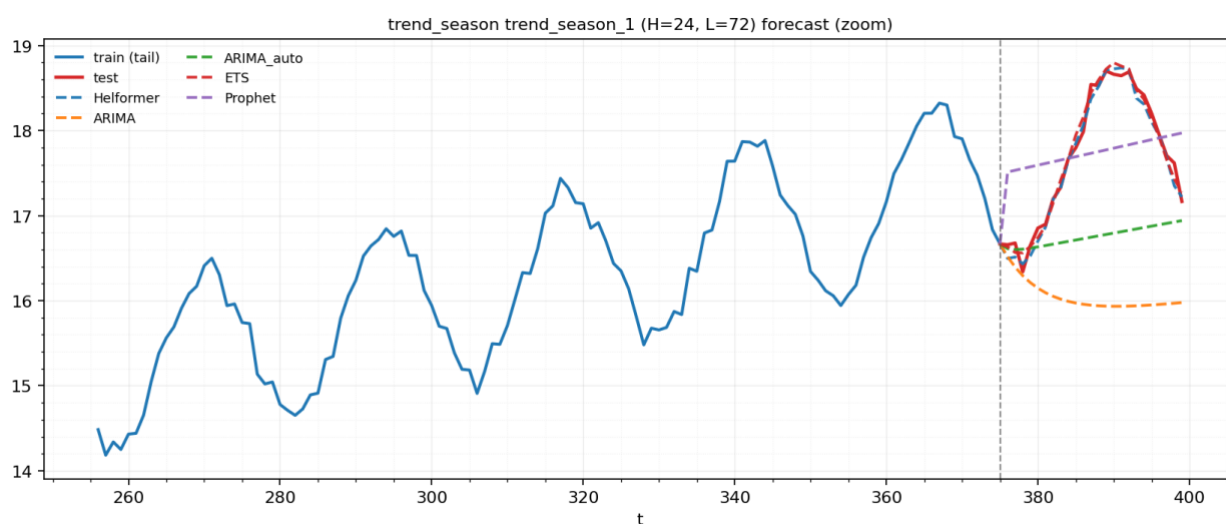


Рисунок 2.2.19. Предсказания реализованных гибридных и классических моделей на синтетическом ряду *trend_season_1*.

Таблица 2.2.5. Результат по метрикам на синтетическом ряду *trend_season_1*.

<i>Модель</i>	<i>sMAPE</i>	<i>MAPE</i>	<i>RMSE</i>	<i>MSE</i>
<i>ARIMA</i>	9.877591	9.297825	1.904548	3.627304
<i>ARIMA_auto</i>	5.653039	5.434191	1.193169	1.423652
<i>ETS</i>	0.465354	0.464001	0.104773	0.010977
<i>Prophet</i>	3.399143	3.410362	0.681593	0.464569
<i>Helformer</i>	0.455592	0.456106	0.100235	0.010047

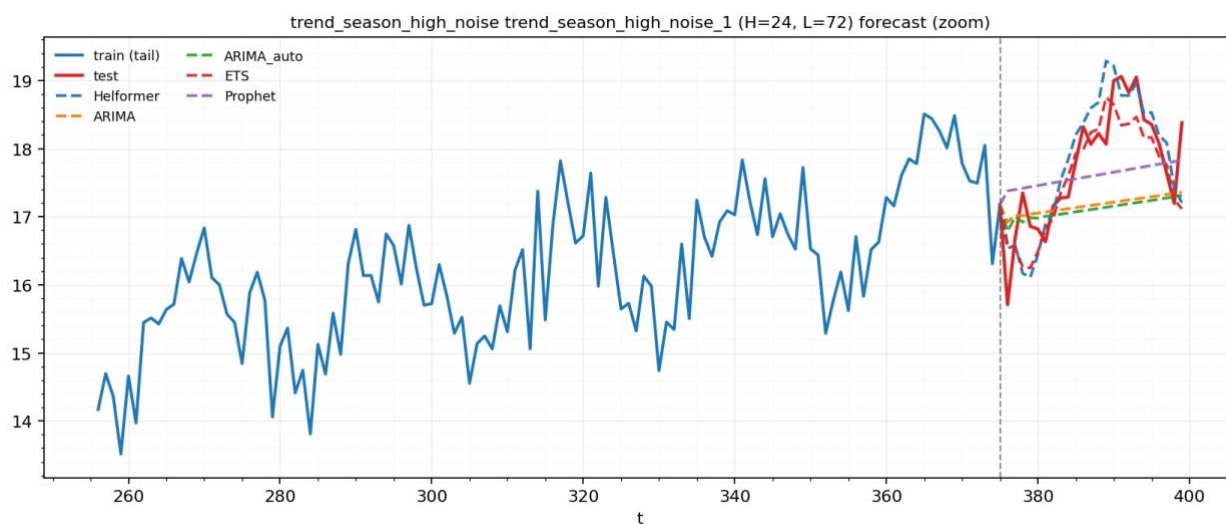


Рисунок 2.2.20. Предсказания реализованных гибридных и классических моделей на синтетическом ряду *trend_season_high_noise_1*

Таблица 2.2.6. Результат по метрикам на синтетическом ряду *trend_season_high_noise_1*.

<i>Модель</i>	<i>sMAPE</i>	<i>MAPE</i>	<i>RMSE</i>	<i>MSE</i>
<i>ARIMA</i>	4.534487	4.418706	0.977143	0.954809
<i>ARIMA_auto</i>	4.696072	4.564086	1.009541	1.019174
<i>ETS</i>	2.362437	2.362076	0.547851	0.300141
<i>Prophet</i>	3.758528	3.742476	0.785856	0.617569
<i>Helformer</i>	2.224664	2.204243	0.508991	0.259071

В среднем по всем синтетическим профилям наиболее стабильное качество дают **ETS** и **Helformer**: **ETS** чуть лучше по относительным метрикам (*sMAPE/MAPE*), а **Helformer** - по абсолютным (*RMSE/MSE*); при этом на рядах с сезонностью и на шумных профилях обе модели удерживают низкую ошибку (например, *season_only_1*: *sMAPE* $\approx$ 0.90 – 0.93; *trend_season_high_noise_1*: *sMAPE* $\approx$ 2.22 – 2.36). **ARIMA** и **ARIMA_auto** без явной сезонной части заметно деградируют на чисто сезонных и тренд+сезон рядах (на *season_only_1* вплоть до *sMAPE* = 21.62 и *RMSE* = 2.22; на *trend_season_1* *sMAPE* = 5.65 – 9.88 и *RMSE* = 1.19 – 1.90), тогда как **ETS** и **Helformer** сохраняют приемлемое качество даже при высоком шуме.

2.3. Тестирование моделей на наборе данных М3

Набор данных **М3** содержит временные ряды различных частот и традиционно используется как эталон для сравнения методов прогнозирования. В рамках данной работы эксперимент проведён на трёх поднаборках М3: **Monthly**, **Quarterly** и **Yearly**. Для снижения вычислительных затрат и обеспечения сопоставимости результатов из каждой категории была сформирована подвыборка из **42 случайно выбранных рядов**.

2.3.1 Протокол обучения и тестирования

Для каждого ряда использовалась стандартная схема «обучение–прогноз»:

- исходный ряд длины (T) делился на обучающую и тестовую части, где тест — это последние (H) точек;
- модели обучались на первых (T-H) наблюдениях и строили прогноз на горизонт (H).

Горизонты прогнозирования соответствуют настройкам эксперимента:

- **Monthly:** ($H=18$),
- **Quarterly:** ($H=8$),
- **Yearly:** ($H=6$).

Для нейросетевого компонента в Helformer использовалось фиксированное окно истории (**L=30**) (последние 30 точек обучающей части), что позволяет унифицировать вход модели и контролировать сложность обучения на коротких рядах.

Оценка качества выполнялась по метрикам *sMAPE*, *MAPE*, *RMSE*, *MSE*, вычисляемым на тестовом горизонте для каждого ряда отдельно. Далее значения агрегировались усреднением:

- **по категории** (среднее по 42 рядам в Monthly / Quarterly / Yearly),
- **общее** (среднее по всем 126 рядам трёх категорий).

Логика Helformer в рамках М3-эксперимента следующая: сначала подбирается и обучается базовая модель Holt–Winters (ETS) и получают её сглаженная аппроксимация и прогноз. Далее ряд приводится к остаточной компоненте (в мультипликативной постановке — через отношение ряда к базовой части; в аддитивной - через разность). Нейросетевой блок обучается предсказывать именно остаток на горизонте (H), после чего итоговый прогноз получается сборкой базового прогноза и прогноза остатка.

2.3.2 Количественные результаты

По усреднённым метрикам из были получены таблицы 2.3.1-2.3.4:

<i>Модель</i>	<i>sMAPE</i>	<i>MAPE</i>	<i>RMSE</i>	<i>MSE</i>
<i>Helformer</i>	13.634000	13.923000	813.717000	1781900.964000
<i>ARIMA</i>	14.828000	15.225000	836.691000	1841615.850000
<i>ARIMA auto</i>	17.490000	20.958000	983.953000	2651395.667000
<i>ETS</i>	16.861000	15.467000	851.408000	1866762.888000
<i>Prophet</i>	15.338000	15.145000	815.847000	1397285.337000

Таблица 2.3.1 — Средние метрики на M3 Monthly (42 ряда)

<i>Модель</i>	<i>sMAPE</i>	<i>MAPE</i>	<i>RMSE</i>	<i>MSE</i>
<i>Helformer</i>	9.070000	10.318000	509.114000	443652.950000
<i>ARIMA</i>	10.545000	12.661000	601.907000	677521.025000
<i>ARIMA auto</i>	11.348000	13.729000	648.372000	808340.859000
<i>ETS</i>	9.399000	10.727000	501.330000	412553.043000
<i>Prophet</i>	12.578000	15.065000	695.885000	765442.116000

Таблица 2.3.2 — Средние метрики на M3 Quarterly (42 ряда)

<i>Модель</i>	<i>sMAPE</i>	<i>MAPE</i>	<i>RMSE</i>	<i>MSE</i>
<i>Helformer</i>	20.404000	32.902000	2526.963000	43419699.998000
<i>ARIMA</i>	13.884000	16.022000	1076.372000	3291823.322000
<i>ARIMA auto</i>	12.178000	14.775000	1008.681000	2977100.506000
<i>ETS</i>	15.236000	17.358000	1122.513000	3225316.410000
<i>Prophet</i>	16.870000	18.163000	1191.215000	2459068.686000

Таблица 2.3.3 — Средние метрики на M3 Yearly (42 ряда)

<i>Модель</i>	<i>sMAPE</i>	<i>MAPE</i>	<i>RMSE</i>	<i>MSE</i>
<i>Helformer</i>	14.370000	19.048000	1283.264000	15215084.637000
<i>ARIMA</i>	13.086000	14.636000	838.323000	1936986.732000
<i>ARIMA auto</i>	13.672000	16.487000	880.335000	2145612.344000
<i>ETS</i>	13.832000	14.517000	825.084000	1834877.447000
<i>Prophet</i>	14.929000	16.125000	900.982000	1540598.713000

Таблица 2.3.4 - средние метрики на M3 (Monthly+Quarterly+Yearly, 126 рядов)

ETS и *Helformer* в среднем дают наиболее конкурентоспособные результаты на подвыборках МЗ, однако вклад моделей по типу метрик различается. По относительным ошибкам *Helformer* часто оказывается сильным (*Monthly*: лучший $sMAPE = 13.634$; *Quarterly*: лучший $sMAPE = 9.070$), что согласуется с тем, что корректировка остатка помогает точнее воспроизводить форму колебаний. При этом по абсолютным метрикам ($RMSE/MSE$) наблюдается нестабильность: на *Monthly* и особенно на *Yearly* *Helformer* заметно проигрывает конкурентам по $RMSE/MSE$ (*Yearly*: $RMSE = 2526.963$, $MSE = 43419699.998$), то есть при отдельных сериях допускает крупные промахи по масштабу, даже если относительная форма прогноза местами сохраняется. В общем зачете *Helformer* уступает по относительным метрикам лучшим статистическим моделям ($sMAPE = 14.370$, $MAPE = 19.048$), и особенно выделяется повышенными $RMSE/MSE$ ($RMSE = 1283.264$, $MSE = 15215084.637$), что указывает на чувствительность гибридного механизма к рядам с доминирующим трендом и слабой/отсутствующей сезонностью.

2.3.3 Качественный анализ прогнозов на отдельных рядах

Для иллюстрации поведения моделей ниже приведены примеры прогнозов на трёх рядах из разных частот МЗ.

На месячном ряде заметны выраженные сезонные колебания с изменяющейся амплитудой и локальными всплесками. В таких условиях модели, склонные к сильному сглаживанию, часто выдают плоский прогноз и не попадают в резкие движения тестового отрезка. *ETS* и *Helformer*, как правило, лучше удерживают форму ряда на горизонте и дают более реалистичную динамику, тогда как часть статистических моделей может либо запаздывать, либо недооценивать амплитуду.

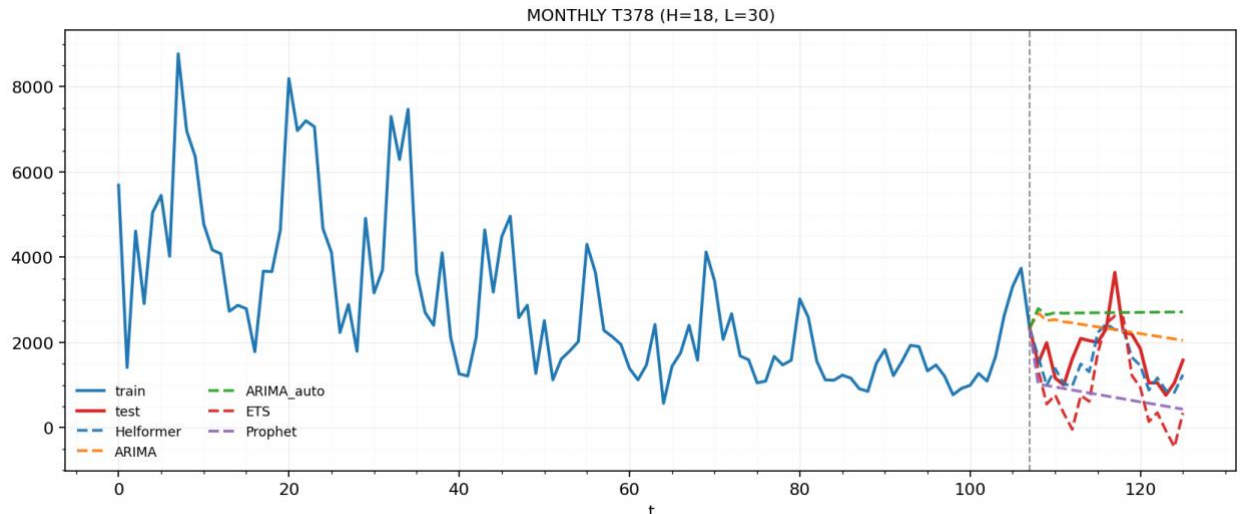


Рисунок 2.3.1. Прогнозы моделей на МЗ Monthly T378 (H=18, L=30)

Квартальный ряд характеризуется устойчивой сезонной структурой и умеренным трендом, поэтому базовые модели сглаживания часто дают хорошую опору для прогноза. Визуально прогнозы *ETS* и *Helformer* лучше согласуются с ожидаемой формой ряда на тестовом интервале, тогда как альтернативные подходы могут сильнее расходиться в точках локальных экстремумов.

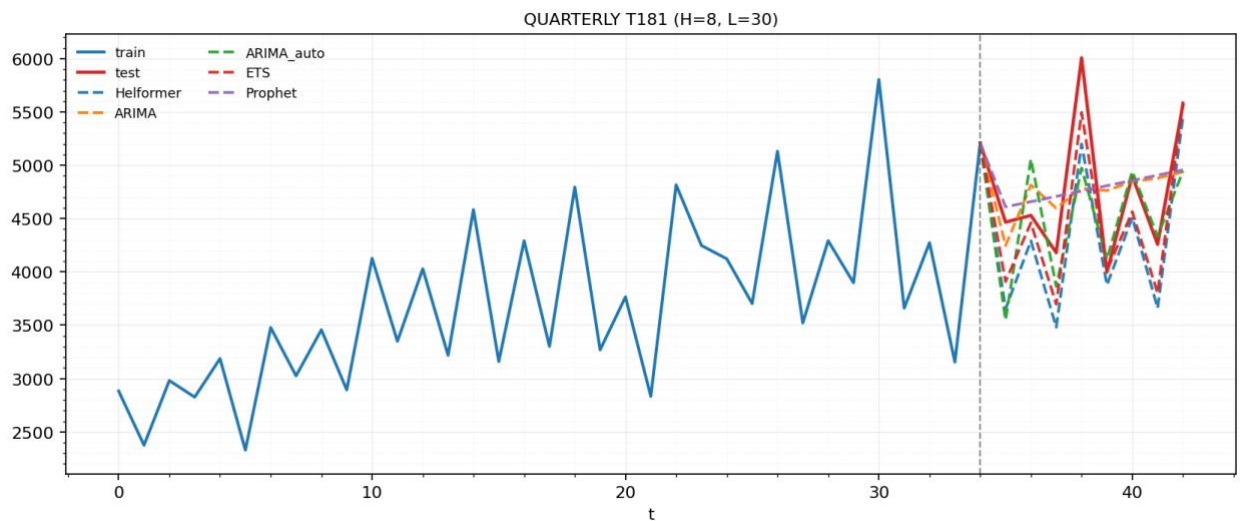


Рисунок 2.3.2. Прогнозы моделей на M3 Quarterly T181 (H=8, L=30)

Годовой ряд демонстрирует доминирующий тренд при практически отсутствии сезонности. В этом случае ключевым фактором становится корректная экстраполяция тренда и устойчивость к единичным изломам траектории перед границей прогноза. Prophet и ARIMA-подходы могут давать более консервативные траектории (недооценка роста), тогда как гибридные решения при определённых настройках способны сильнее проталкивать тренд вперёд. Для годовых рядов особенно важно избегать систематического смещения прогноза относительно тестовой части.

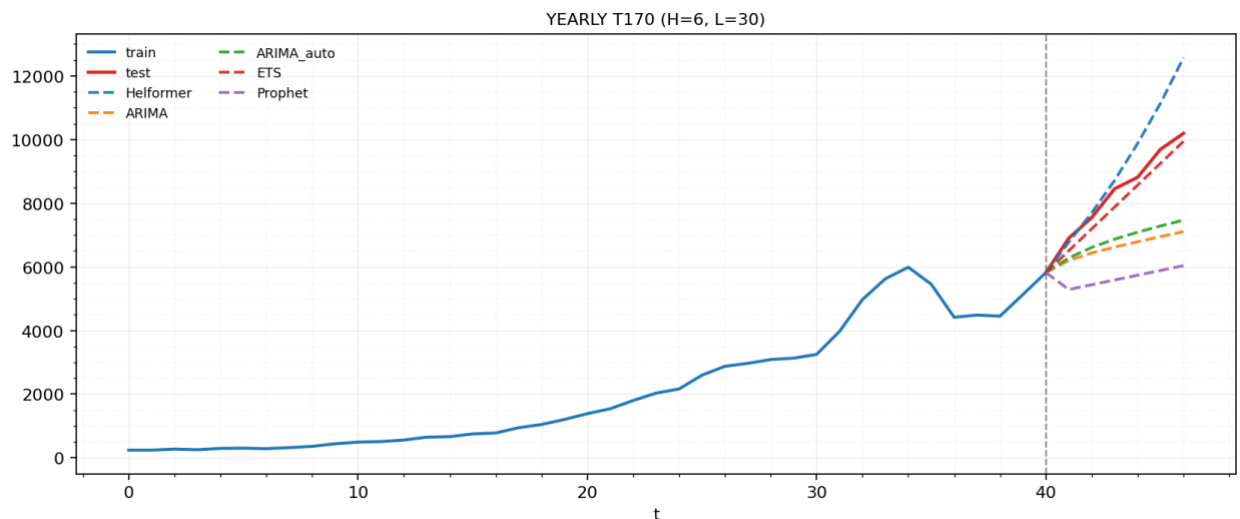


Рисунок 2.3.3. Прогнозы моделей на M3 Yearly T170 (H=6, L=30)

2.3.4 Анализ разложения Helformer (роль базовой компоненты и остатка)

Дополнительно рассмотрим разложения, используемые в *Helformer*, для тех же трёх рядов. На каждом графике показаны: исходный ряд и базовая *Holt-Winters* аппроксимация, оценка трендоподобной компоненты, сезонная компонента (если применима), а также остаток, который затем поступает в нейросетевой блок.

Сезонная компонента имеет стабильный период (12) и выраженную амплитуду, а трендоподобная часть отражает долгосрочное снижение уровня ряда. Остаток в целом колеблется вокруг единицы, но содержит всплески, которые базовая модель сглаживания не объясняет полностью - именно эта часть является целью для нейросетевой корректировки.

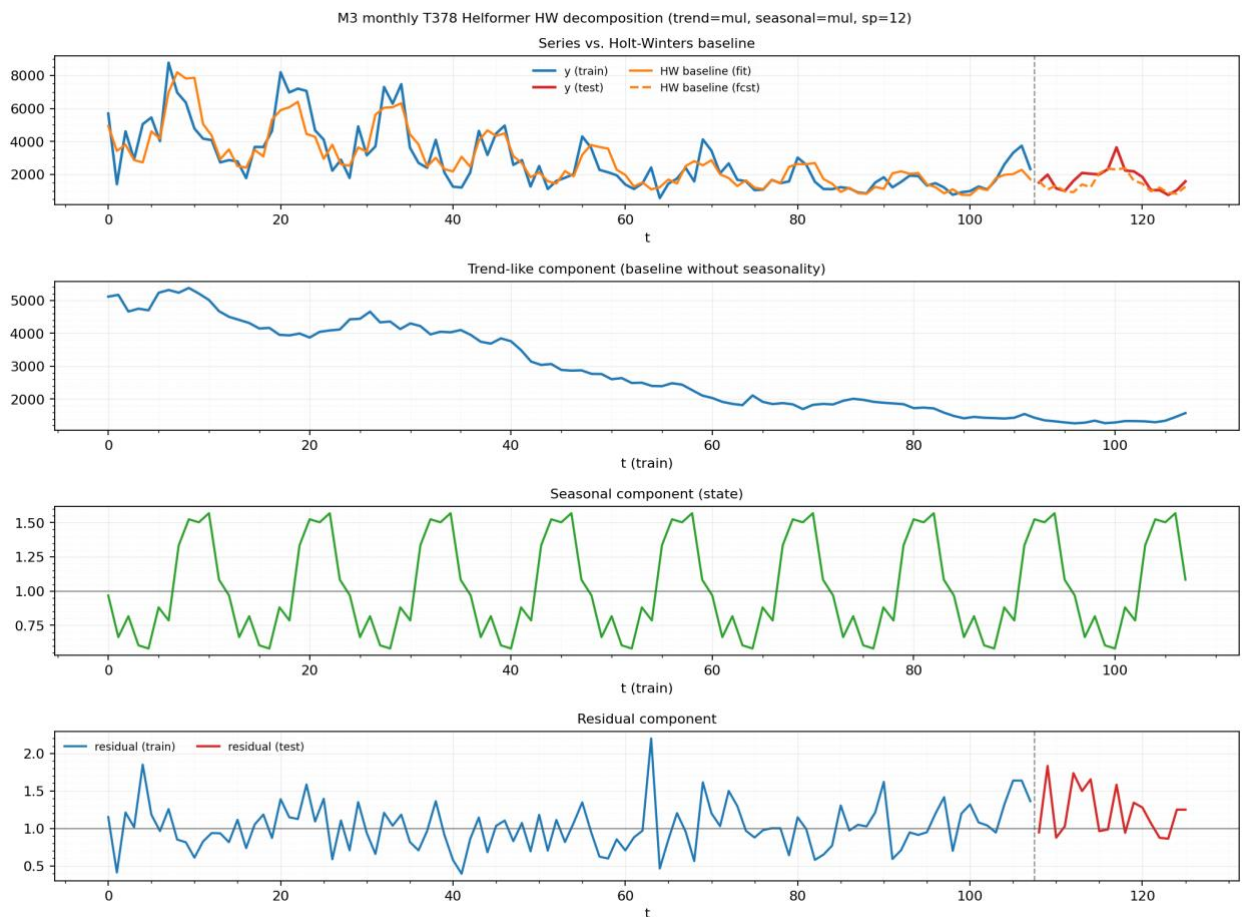


Рисунок 2.3.4. Разложение *Helformer* для M3 Monthly T378

Для квартального ряда сезонность выражена слабее, но остаётся регулярной (период 4). Базовая Holt–Winters компонента хорошо описывает основную структуру, а остаток остаётся умеренным по амплитуде. В таких условиях *Helformer* обычно не перестраивает прогноз целиком, а вносит небольшую поправку поверх сильной статистической базы.

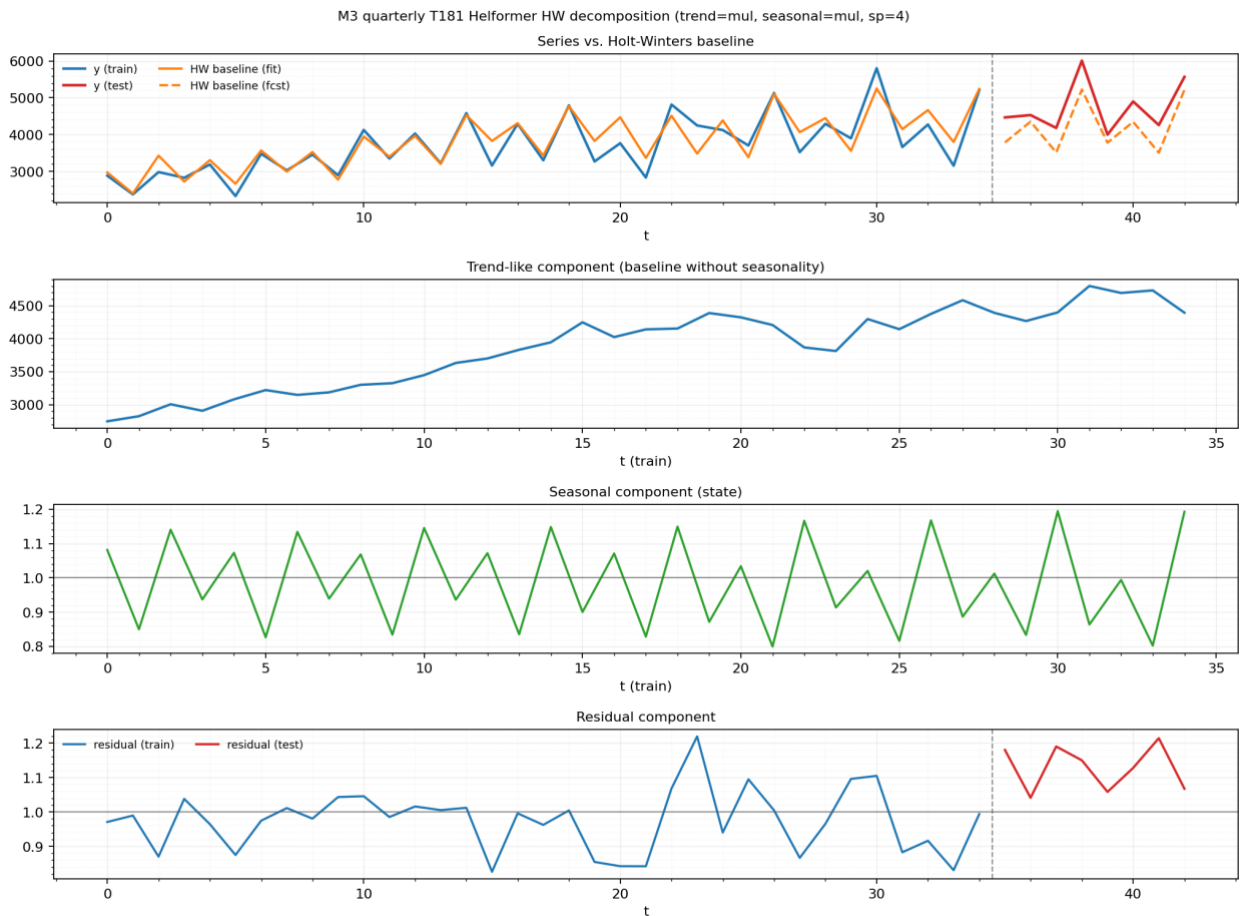


Рисунок 2.3.5. Разложение *Helformer* для M3 Quarterly T181

Для годового ряда сезонная компонента отсутствует, а динамика определяется трендом. В разложении видно, что базовая модель даёт основную траекторию роста, а остаток отражает нерегулярные отклонения относительно тренда. Это объясняет, почему на Yearly-сериях итоговое качество сильнее зависит от устойчивости оценивания тренда и корректности работы с последними наблюдениями перед границей прогноза.

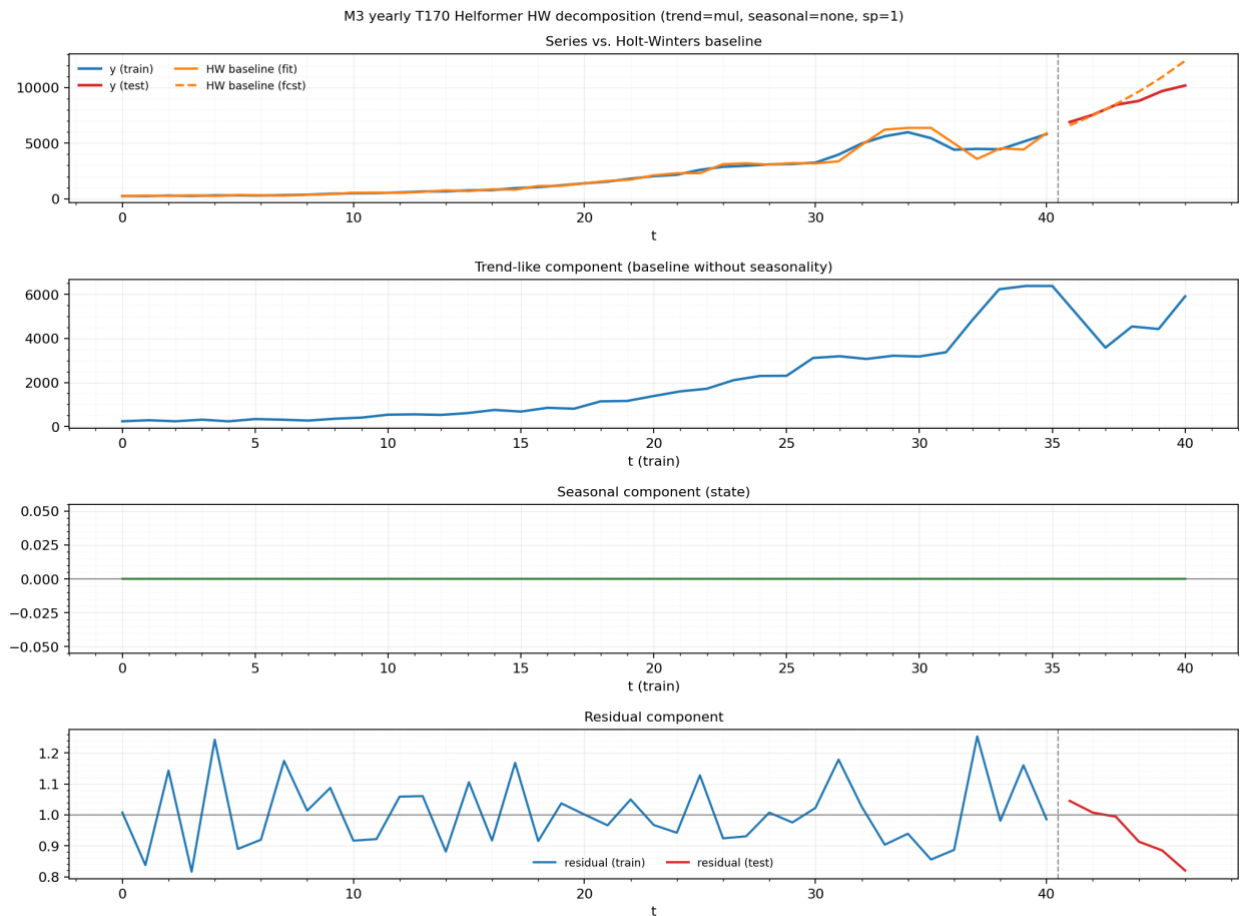


Рисунок 2.3.6. Разложение *Helformer* для M3 Yearly T170

2.3.5 Вывод по эксперименту M3

Итог по эксперименту M3: на сезонных рядах (*Monthly/Quarterly*) *Helformer* демонстрирует сильную сторону именно в относительной точности (лучший *sMAPE* в обеих категориях), но его ключевой недостаток - риск крупных абсолютных ошибок, наиболее явно проявляющийся на годовых рядах, что приводит к завышенным *RMSE/MSE* и ухудшает общий зачет по абсолютным метрикам. Поэтому *Helformer* целесообразно рассматривать как метод, улучшающий форму прогноза (*sMAPE/MAPE*) на сезонных частотах, но требующий осторожности по устойчивости масштаба ошибки (*RMSE/MSE*), особенно на *Yearly*-сериях.

2.4. Тестирование моделей на наборе данных M4

Набор данных *M4* используется для проверки методов прогнозирования на существенно более разнообразных рядах по сравнению с синтетикой и *M3*: в нём присутствуют разные масштабы значений, разные уровни шума, неоднородные тренды и сезонности, а также нерегулярные «переломы» динамики. Поэтому *M4* удобно рассматривать как стресс-тест на устойчивость моделей и их способность переноситься на реальные данные.

2.4.1. Протокол эксперимента

Тестирование проводилось по стандартной схеме *M4*: для каждого временного ряда последняя часть длины (H) выделяется под тест, на оставшемся префиксе обучаются модели и строится прогноз на горизонт (H). Горизонт выбирается по частоте данных (для *quarterly* в *M4* используется ($H=8$)).

Оценка качества выполнялась по метрикам *sMAPE*, *MAPE*, *RMSE*, *MSE*, рассчитанным на тестовом отрезке. Далее метрики агрегировались усреднением по всем рядам рассматриваемой подвыборки.

<i>Модель</i>	<i>sMAPE</i>	<i>MAPE</i>	<i>RMSE</i>	<i>MSE</i>
<i>Naive</i>	10.734017	11.271954	203.474159	49778.637658
<i>Helformer</i>	6.296734	6.081387	107.690268	13360.861833
<i>ARIMA</i>	10.039316	11.478423	170.497651	50275.421228
<i>ARIMA auto</i>	5.192852	5.402355	97.787364	11695.128808
<i>ETS</i>	9.746515	10.777175	172.670585	46889.459077

Таблица 2.4.1 — Средние метрики на M4 Quarterly

<i>Модель</i>	<i>sMAPE</i>	<i>MAPE</i>	<i>RMSE</i>	<i>MSE</i>
<i>Naive</i>	17.124461	19.433712	326.460151	106576.230475
<i>Helformer</i>	12.026974	11.650037	166.972594	27879.847034
<i>ARIMA</i>	23.798211	28.231816	376.115364	141462.767140
<i>ARIMA auto</i>	9.408795	10.142474	162.319219	26347.528903
<i>ETS</i>	22.401646	25.732533	357.056859	127489.600765

Таблица 2.4.2 — Метрики на M4 Quarterly Q15710 (H=8)

<i>Модель</i>	<i>sMAPE</i>	<i>MAPE</i>	<i>RMSE</i>	<i>MSE</i>
<i>Naive</i>	5.534422	5.366456	176.926701	31303.057500
<i>Helformer</i>	2.168475	2.134355	81.164630	6587.697234
<i>ARIMA</i>	2.157155	2.125857	77.687566	6035.357956
<i>ARIMA auto</i>	2.021653	1.993135	74.231786	5510.358032
<i>ETS</i>	2.506318	2.463174	91.096706	8298.609887

Таблица 2.4.3 — Метрики на M4 Quarterly Q18575 (H=8)

<i>Модель</i>	<i>sMAPE</i>	<i>MAPE</i>	<i>RMSE</i>	<i>MSE</i>
<i>Naïve</i>	9.543168	9.015694	107.035625	11456.625000
<i>Helformer</i>	4.694752	4.459769	74.933579	5615.041232
<i>ARIMA</i>	4.162583	4.077594	57.690022	3328.138588
<i>ARIMA auto</i>	4.148109	4.071456	56.811086	3227.499488
<i>ETS</i>	4.331581	4.135817	69.858189	4880.166580

Таблица 2.4.4 — Метрики на M4 Quarterly Q2142 (H=8)

2.4.2. Количественные результаты и их интерпретация

По итоговым таблицам для M4 целесообразно разделять выводы по типу метрик:

- По **относительным ошибкам (*sMAPE/MAPE*)** *Helformer* действительно показывает сильную сторону: в среднем по подвыборке (табл. 2.4.1) он заметно лучше “чистых” статистических базовых моделей *ARIMA* и *ETS* (*sMAPE*=6.297 против 10.039 и 9.747; *MAPE*=6.081 против 11.478 и 10.777). Это означает, что гибридная корректировка помогает точнее воспроизводить относительные колебания на тестовом горизонте.
- Однако по **абсолютным метрикам (*RMSE/MSE*)** *Helformer* уступает лучшей модели на данной подвыборке — *ARIMA_auto* (табл. 2.4.1: *RMSE*=107.690 и *MSE*=13360.862 у *Helformer* против *RMSE*=97.787 и *MSE*=11695.129 у *ARIMA_auto*). На отдельных рядах это проявляется сильнее: например, на *Q2142* *Helformer* даёт *RMSE*=74.934, тогда как *ARIMA_auto* — *RMSE*=56.811 (табл. 2.4.4), то есть при близких процентных ошибках возможны более крупные промахи в абсолютных единицах.

Helformer уверенно улучшает *ETS/ARIMA* по *sMAPE/MAPE*, но по *RMSE/MSE* не достигает качества *ARIMA_auto*, что указывает на риск более крупных абсолютных отклонений на части серий.

2.4.3. Качественный анализ (примеры рядов quarterly)

Ниже приведены примеры рядов quarterly, иллюстрирующие поведение *Helformer*.

Пример 1 — регулярная сезонность и умеренный шум (*Q15710*). На *Q15710* гибридная схема заметно снижает процентную ошибку относительно *ETS/ARIMA* (табл. 2.4.2: *sMAPE*=12.027 у *Helformer* против 22.402 у *ETS* и 23.798 у *ARIMA*), что согласуется с идеей коррекции остатка поверх *Holt–Winters*. При этом на этом же ряде по абсолютным метрикам *Helformer* лишь немного уступает *ARIMA_auto* (*RMSE*=166.973 против 162.319), то есть улучшение формы прогноза не гарантирует лидерства по *RMSE/MSE*.

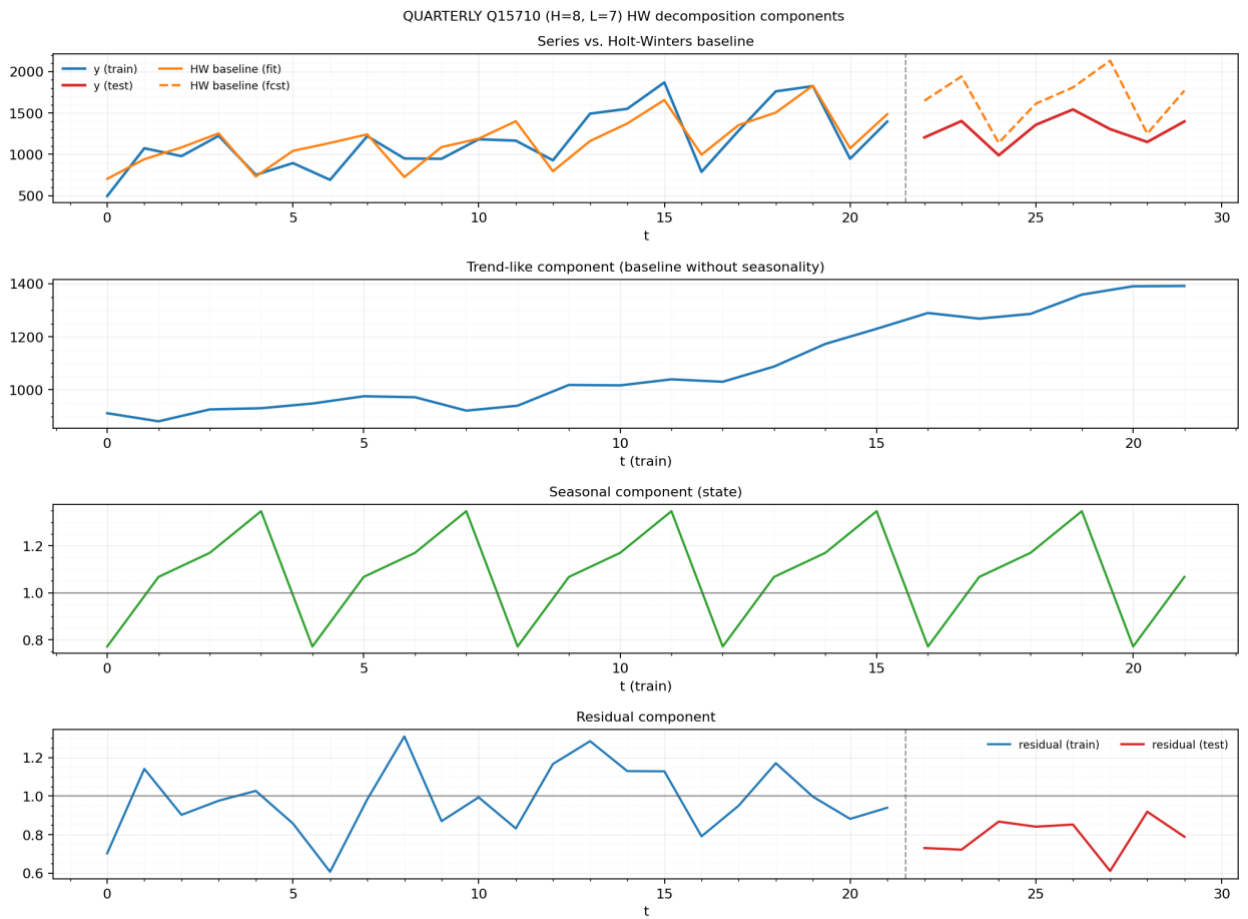


Рисунок 2.4.1 - *QUARTERLY Q15710*: разложение *Helformer*

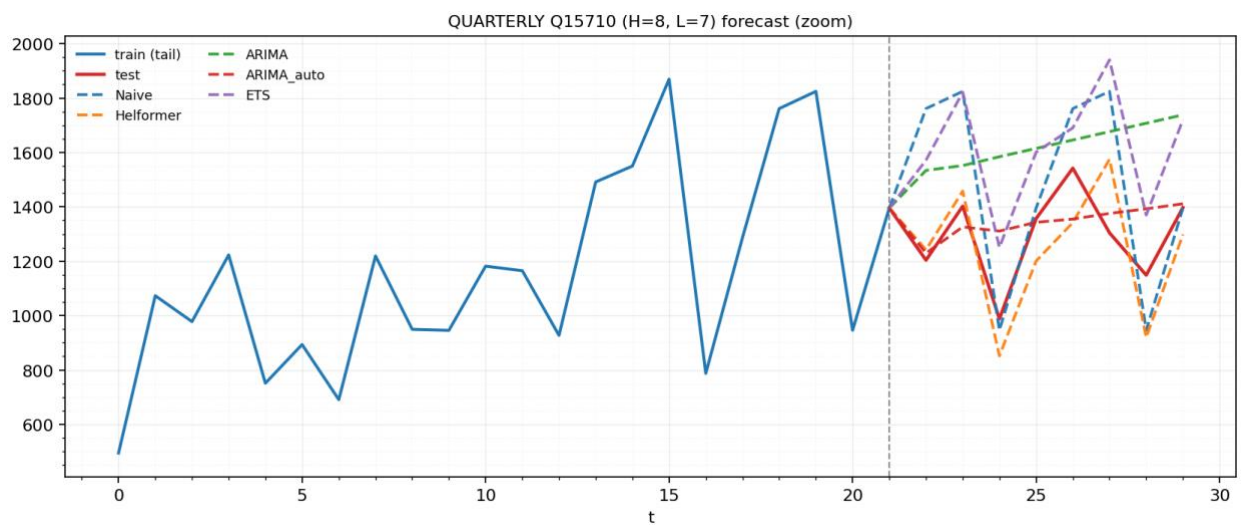


Рисунок 2.4.2 - *QUARTERLY Q15710*: сравнение прогнозов моделей на горизонте (H=8)

Пример 2 - смена режима/скачок уровня (*Q2142*).

На *Q2142* наиболее точные абсолютные прогнозы дают *ARIMA*-подходы (табл. 2.4.4: $RMSE \approx 56.8$ – 57.7 у *ARIMA_auto/ARIMA*). *Helformer* остаётся конкурентным по процентным метрикам ($sMAPE=4.695$ лучше $ETS=4.332$ не получилось, но близко), однако проигрывает по $RMSE/MSE$, что соответствует ситуации, когда итоговая траектория по форме в целом близка, но есть промах по уровню/амплитуде на отдельных шагах.

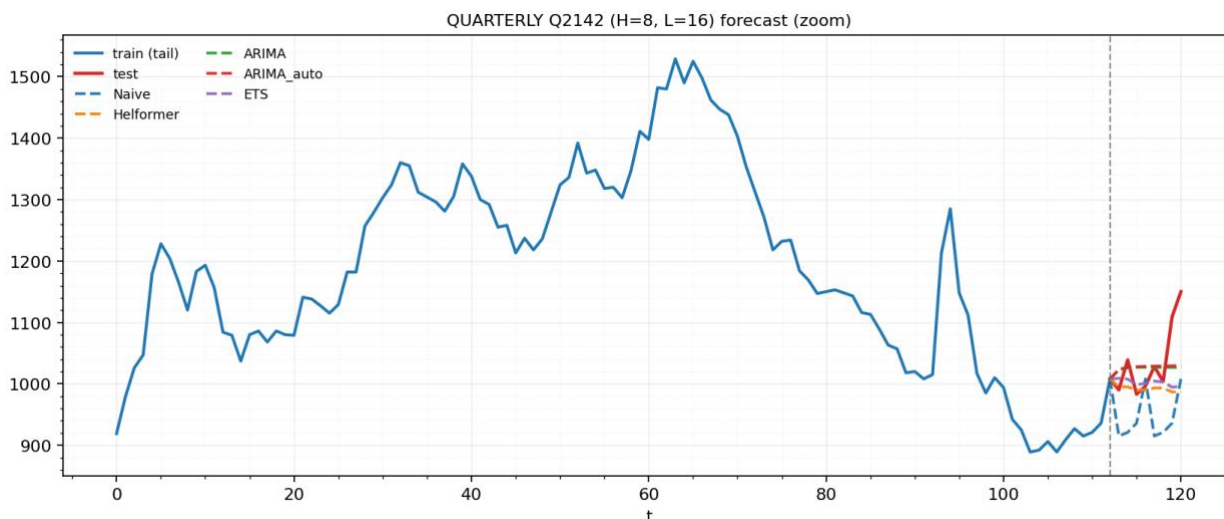


Рисунок 2.4.3 - *QUARTERLY Q2142*: сравнение прогнозов как пример роста $RMSE/MSE$ при смене режима

2.4.4 Вывод по тестированию на M4

Тестирование на *quarterly*-подвыборке *M4* показывает, что *Helformer* стабильно снижает **относительные ошибки ($sMAPE/MAPE$) по сравнению с *ETS* и *ARIMA***, т.е. хорошо работает как механизм «доводки формы» прогноза поверх *Holt–Winters* (табл. 2.4.1: $sMAPE=6.297$ против 9.747 – 10.039). Одновременно **по $RMSE/MSE$ *Helformer* не является лучшим**: на этой подвыборке лидирует *ARIMA_auto* ($RMSE=97.787$ против 107.690 у *Helformer*), а на ряде со сложной динамикой (*Q2142*) разрыв по $RMSE$ становится заметным (74.934 против 56.811). Практически это означает: *Helformer* стоит рассматривать как метод, который улучшает процентные метрики и локальную динамику, но при постановках, где критичны большие промахи в абсолютных единицах, необходимо отдельно контролировать $RMSE/MSE$ и анализировать случаи, где гибридная корректировка не успевает адаптироваться к смене режима.

Заключение

В рамках выполненной НИР были рассмотрены и сопоставлены классические и современные методы прогнозирования одномерных временных рядов (*ARIMA/ARIMA_auto*, *ETS/Holt-Winters*, *Prophet*), а также исследован гибридный подход *Helformer*, в котором статистическая база *Holt-Winters* дополняется нейросетевой корректировкой остатка.

Практическая часть включала реализацию единого конвейера экспериментов (разбиение на *train/test* по горизонту H , обучение моделей, построение прогноза, расчёт *sMAPE*, *MAPE*, *RMSE* и *MSE*) и проведение сравнительного тестирования на синтетических профилях и на подвыборках реальных наборов *M3* и *M4*.

Эксперименты на синтетических рядах показали, что методы экспоненциального сглаживания дают устойчивую базу качества, а гибридизация полезна прежде всего тогда, когда в данных присутствуют нерегулярные колебания поверх выраженной трендово-сезонной структуры: нейросетевая поправка позволяет точнее воспроизводить форму колебаний без разрушения базового прогноза.

На подвыборках *M3* проявляется компромисс между относительными и абсолютными метриками. Для *Monthly* и *Quarterly Helformer* оказался лучшим по относительным ошибкам (*Monthly*: *sMAPE*=13.634, *MAPE*=13.923; *Quarterly*: *sMAPE*=9.070, *MAPE*=10.318), что согласуется с его назначением - корректировать локальные отклонения. При этом для *Yearly* качество *Helformer* резко деградирует (*sMAPE*=20.404, *MAPE*=32.902) и сопровождается крупными абсолютными промахами (*RMSE*=2526.963). В *overall* по *M3* (126 рядов) *Helformer* остаётся близок к базовым по *sMAPE* (14.370), но имеет наибольшие *RMSE*=1283.264 и *MSE*=15215084.637; для сравнения, *ETS* достигает *RMSE*=825.084 (*MAPE*=14.517), а минимальный *sMAPE* даёт *ARIMA* (13.086). Высокие *RMSE/MSE* у *Helformer* указывают на наличие редких крупных промахов, которые квадратично штрафуются в *MSE*.

Тестирование на *quarterly*-подвыборке *M4* подтверждает, что *Helformer* конкурентоспособен по процентным метрикам и заметно превосходит «чистые» *ARIMA* и *ETS*, но не всегда становится лидером: в среднем по *M4 Quarterly* лучшая модель - *ARIMA_auto* (*sMAPE*=9.409, *MAPE*=10.142, *RMSE*=162.319), тогда как *Helformer* показывает *sMAPE*=12.027 при близком *RMSE*=166.973. Таким образом, при наличии сильной автонастройки сезонности и тренда статистические методы могут выигрывать, а вклад нейросетевой корректировки проявляется прежде всего на рядах с выраженными локальными колебаниями.

В целом результаты подтверждают практическую ценность *Helformer* как метода доводки прогноза: он чаще улучшает относительную точность и визуальное совпадение траекторий (*sMAPE/MAPE*), однако требует контроля по *RMSE/MSE* и анализа ситуаций смены режима или сдвига уровня, где возможны редкие, но крупные промахи.

Список литературы

1. Kehinde T. O. Helformer: an attention-based deep learning model for cryptocurrency price forecasting [Электронный ресурс] / T. O. Kehinde, O. J. Adedokun, A. Joseph, K. M. Kabirat, H. A. Akano, O. A. Olanrewaju // Journal of Big Data. — 2025. — Vol. 12. — Art. 81. — DOI: 10.1186/s40537-025-01135-4. — Режим доступа: <https://link.springer.com/article/10.1186/s40537-025-01135-4> (дата обращения: 23.12.2025), свободный. (link.springer.com)
2. Zhou T. FEDformer: Frequency Enhanced Decomposed Transformer for Long-term Series Forecasting [Электронный ресурс] / T. Zhou, Z. Ma, Q. Wen, X. Wang, L. Sun, R. Jin // Proceedings of the 39th International Conference on Machine Learning. — PMLR, 2022. — Vol. 162. — P. 27268–27286. — Режим доступа: <https://proceedings.mlr.press/v162/zhou22g.html> (дата обращения: 23.12.2025), свободный. ([Proceedings of Machine Learning Research](https://proceedings.mlr.press/v162/zhou22g.html))
3. Percival D. B. Wavelet Methods for Time Series Analysis / D. B. Percival, A. T. Walden. — Cambridge : Cambridge University Press, 2000. — 594 p. (books.google.com)
4. Percival D. P. On estimation of the wavelet variance // Biometrika. — 1995. — Vol. 82, № 3. — P. 619–631. — DOI: 10.1093/biomet/82.3.619. ([OUP Academic](https://academic.oup.com/biomet))
5. Box J. Time series analysis; forecasting and control / J. Box, M. Jenkins. — Сан-Франциско : Holden-Day, 1970. — 712 с.
6. Доугарти К. Введение в эконометрику / К. Доугарти ; пер. с англ. — Москва : ИНФРА-М, 2009. — 435 с.
7. Айвазян С. А. Прикладная статистика и основы эконометрики / С. А. Айвазян, В. С. Мхитарян. — Москва : ЮНИТИ-ДАНА, 1998. — 1006 с.

Приложение

Полный код представлен в репозитории «<https://github.com/sheeelby/Hybrid-models-for-TS-processing>».

```
"""Helformer inference helpers (TensorFlow/Keras)."""
from __future__ import annotations

from dataclasses import dataclass
from pathlib import Path
from typing import Any, Iterable, Optional, Tuple

import numpy as np
import json

try: # pragma: no cover - optional dependency
    import tensorflow as tf
    from tensorflow.keras.models import load_model
except Exception: # pragma: no cover - fallback when TF is missing
    tf = None
    load_model = None

try: # pragma: no cover - optional dependency
    from statsmodels.tsa.holtwinters import ExponentialSmoothing
except Exception: # pragma: no cover - fallback when statsmodels is missing
    ExponentialSmoothing = None

try: # pragma: no cover - optional dependency
    import h5py
except Exception: # pragma: no cover - fallback when h5py is missing
    h5py = None

def mish(x):
    if tf is None: # pragma: no cover - TF required for inference
        raise RuntimeError("TensorFlow is required for mish activation")
    return x * tf.math.tanh(tf.math.softplus(x))

def _extract_helformer_config(weights_path: Path) -> dict:
    if h5py is None:
        raise RuntimeError("h5py is required to inspect Helformer weights")
    with h5py.File(weights_path, "r") as handle:
```

```

        model_config = handle.attrs.get("model_config")
    if model_config is None:
        raise RuntimeError("model_config not found in Helformer weights")
    if isinstance(model_config, (bytes, bytearray)):
        model_config = model_config.decode("utf-8")
    cfg = json.loads(model_config)
    layers = cfg.get("config", {}).get("layers", [])
    input_layer = next((layer for layer in layers if layer.get("class_name") == "InputLayer"), None)
    if input_layer is None:
        raise RuntimeError("InputLayer not found in Helformer config")
    batch_shape = input_layer.get("config", {}).get("batch_shape", [None, None, None])
    lookback = int(batch_shape[1]) if len(batch_shape) > 1 and batch_shape[1] is not None else 30

    mha_layers = [layer for layer in layers if layer.get("class_name") == "MultiHeadAttention"]
    if not mha_layers:
        raise RuntimeError("MultiHeadAttention layers not found in Helformer config")
    mha_cfg = mha_layers[0].get("config", {})
    num_blocks = len(mha_layers)
    num_heads = int(mha_cfg.get("num_heads", 4))
    head_size = int(mha_cfg.get("key_dim", 32))
    dropout_rate = float(mha_cfg.get("dropout", 0.0))

    lstm_layer = next((layer for layer in layers if layer.get("class_name") == "LSTM"), None)
    if lstm_layer is None:
        raise RuntimeError("LSTM layer not found in Helformer config")
    units = int(lstm_layer.get("config", {}).get("units", 32))

    return {
        "lookback": lookback,
        "num_blocks": num_blocks,
        "num_heads": num_heads,
        "head_size": head_size,
        "dropout_rate": dropout_rate,
        "units": units,
    }

def create_helformer_model(
    *,
    lookback: int,
    num_blocks: int,
    num_heads: int,

```

```

    head_size: int,
    dropout_rate: float,
    units: int,
):
    if tf is None: # pragma: no cover - TF required for inference
        raise RuntimeError("TensorFlow is required for Helformer inference")

    lookback = int(lookback)
    num_blocks = int(num_blocks)
    num_heads = int(num_heads)
    head_size = int(head_size)
    dropout_rate = float(dropout_rate)
    units = int(units)

    inputs = tf.keras.layers.Input(shape=(lookback, 1))
    x = inputs
    for _ in range(num_blocks):
        x_norm1 = tf.keras.layers.LayerNormalization(epsilon=1e-6)(x)
        attention = tf.keras.layers.MultiHeadAttention(
            num_heads=num_heads,
            key_dim=head_size,
            dropout=dropout_rate,
        )(x_norm1, x_norm1)
        x = tf.keras.layers.Add()([x, attention])
        x_norm2 = tf.keras.layers.LayerNormalization(epsilon=1e-6)(x)
        x = tf.keras.layers.Add()([x, x_norm2])
    x = tf.keras.layers.LSTM(units, activation=mish, return_sequences=False)(x)
    outputs = tf.keras.layers.Dense(1)(x)
    return tf.keras.Model(inputs=inputs, outputs=outputs)

def load_helformer_model(weights_path: Path | str):
    if load_model is None: # pragma: no cover - TF required for inference
        raise RuntimeError("TensorFlow is required to load the Helformer model")
    weights_path = Path(weights_path)
    if not weights_path.exists():
        raise FileNotFoundError(f"Helformer weights not found: {weights_path}")
    try:
        return load_model(weights_path, custom_objects={"mish": mish}, compile=False)
    except Exception:
        cfg = _extract_helformer_config(weights_path)
        model = create_helformer_model(**cfg)
        model.load_weights(weights_path)

```

```

        return model

def _minmax_fit(series: np.ndarray) -> Tuple[float, float]:
    if series.size == 0:
        return 0.0, 1.0
    vmin = float(np.nanmin(series))
    vmax = float(np.nanmax(series))
    scale = vmax - vmin
    if not np.isfinite(scale) or abs(scale) < 1e-8:
        scale = 1.0
    return vmin, float(scale)

def _minmax_transform(series: np.ndarray, vmin: float, scale: float) -> np.ndarray:
    return (series - vmin) / scale

def _minmax_inverse(series: np.ndarray, vmin: float, scale: float) -> np.ndarray:
    return series * scale + vmin

def _prepare_window(history: Iterable[float], lookback: int) -> np.ndarray:
    if lookback <= 0:
        raise ValueError("lookback must be positive")
    hist = list(history)
    if not hist:
        return np.zeros(lookback, dtype=np.float32)
    if len(hist) >= lookback:
        window = hist[-lookback:]
    else:
        pad_val = hist[-1]
        window = [pad_val] * (lookback - len(hist)) + hist
    return np.asarray(window, dtype=np.float32)

def _holt_winters_baseline(
    y_tr: np.ndarray,
    horizon: int,
    seasonal_period: Optional[int],
    trend: str,
    seasonal: str,
) -> Optional[Tuple[np.ndarray, np.ndarray]]:

```



```

if ExponentialSmoothing is None:
    return None
data = np.asarray(y_tr, float).ravel()
if data.size < 4:
    return None
seasonal_periods = seasonal_period if seasonal_period and seasonal_period > 1 else None
seasonal = seasonal if seasonal_periods else None
trend_mode = trend
seasonal_mode = seasonal
if (trend_mode == "mul" or seasonal_mode == "mul") and np.any(data <= 0):
    if trend_mode == "mul":
        trend_mode = "add"
    if seasonal_mode == "mul":
        seasonal_mode = "add"
try:
    model = ExponentialSmoothing(
        data,
        trend=trend_mode,
        seasonal=seasonal_mode,
        seasonal_periods=seasonal_periods,
    )
    fitted = model.fit(optimized=True, use_brute=True)
    base_train = np.asarray(fitted.fittedvalues, float)
    if base_train.size != data.size:
        base_train = np.resize(base_train, data.size)
    base_train = np.where(np.isfinite(base_train), base_train, data)
    base_test = np.asarray(fitted.forecast(horizon), float)
    if base_test.size != horizon:
        base_test = np.resize(base_test, horizon)
    last_val = base_train[-1] if base_train.size else (data[-1] if data.size else 0.0)
    base_test = np.where(np.isfinite(base_test), base_test, last_val)
    return base_train, base_test
except Exception:
    return None

def _holt_winters_fit(
    y_tr: np.ndarray,
    *,
    seasonal_period: Optional[int],
    trend: str,
    seasonal: str,

```

```

) -> Optional[Tuple[Any, str, str, Optional[int]]]:
    if ExponentialSmoothing is None:
        return None
    data = np.asarray(y_tr, float).ravel()
    if data.size < 4:
        return None
    seasonal_periods = seasonal_period if seasonal_period and seasonal_period > 1 else None
    seasonal_mode = seasonal if seasonal_periods else None
    trend_mode = trend
    if (trend_mode == "mul" or seasonal_mode == "mul") and np.any(data <= 0):
        if trend_mode == "mul":
            trend_mode = "add"
        if seasonal_mode == "mul":
            seasonal_mode = "add"
    try:
        model = ExponentialSmoothing(
            data,
            trend=trend_mode,
            seasonal=seasonal_mode,
            seasonal_periods=seasonal_periods,
        )
        fitted = model.fit(optimized=True, use_brute=True)
    except Exception:
        return None
    return fitted, trend_mode, (seasonal_mode or "none"), seasonal_periods

def helformer_forecast(
    y_tr: Iterable[float],
    horizon: int,
    *,
    model,
    lookback: int = 30,
    seasonal_period: Optional[int] = None,
    use_hw: bool = False,
    hw_trend: str = "mul",
    hw_seasonal: str = "mul",
) -> np.ndarray:
    y_tr = np.asarray(list(y_tr), dtype=float).ravel()
    if horizon <= 0:
        raise ValueError("horizon must be positive")
    if y_tr.size == 0:

```

```

return np.zeros(horizon, dtype=float)

series = y_tr
base_test = None
if use_hw:
    baseline = _holt_winters_baseline(
        y_tr=y_tr,
        horizon=horizon,
        seasonal_period=seasonal_period,
        trend=hw_trend,
        seasonal=hw_seasonal,
    )
    if baseline is not None:
        base_train, base_test = baseline
        denom = np.where(np.abs(base_train) < 1e-8, 1.0, base_train)
        series = y_tr / denom
    else:
        use_hw = False

series = np.where(np.isfinite(series), series, 0.0)
vmin, scale = _minmax_fit(series)
scaled_series = _minmax_transform(series, vmin, scale)
history = scaled_series.tolist()

preds_scaled: list[float] = []
for _ in range(horizon):
    window = _prepare_window(history, lookback).reshape(1, lookback, 1)
    pred = model.predict(window, verbose=0)
    pred_val = float(np.ravel(pred)[0])
    if not np.isfinite(pred_val):
        pred_val = history[-1] if history else 0.0
    preds_scaled.append(pred_val)
    history.append(pred_val)

preds = _minmax_inverse(np.asarray(preds_scaled, float), vmin, scale)
if use_hw and base_test is not None:
    base_test = np.asarray(base_test, float).ravel()
    if base_test.size < horizon:
        pad_val = base_test[-1] if base_test.size else 1.0
        base_test = np.pad(base_test, (0, horizon - base_test.size), constant_values=pad_val)
    elif base_test.size > horizon:

```

```

        base_test = base_test[:horizon]

        preds = preds * base_test
    return preds

@dataclass(frozen=True)
class HelformerHWComponents:
    base_train: np.ndarray
    base_test: np.ndarray
    ratio_train: np.ndarray
    ratio_test: np.ndarray
    level: Optional[np.ndarray]
    trend: Optional[np.ndarray]
    season: Optional[np.ndarray]
    trend_mode: str
    seasonal_mode: str
    seasonal_periods: Optional[int]

def helformer_hw_decompose(
    y_tr: Iterable[float],
    y_te: Iterable[float],
    *,
    seasonal_period: Optional[int] = None,
    hw_trend: str = "mul",
    hw_seasonal: str = "mul",
) -> Optional[Tuple[np.ndarray, np.ndarray, np.ndarray, np.ndarray]]:
    """Compute the Holt-Winters ratio decomposition used by Helformer.

    Returns base (train fitted, test forecast) and multiplicative ratios
    y/base for both train and test segments. Returns None if the baseline
    cannot be fit (e.g., statsmodels missing or convergence issues).
    """
    y_tr_arr = np.asarray(list(y_tr), dtype=float).ravel()
    y_te_arr = np.asarray(list(y_te), dtype=float).ravel()
    horizon = int(y_te_arr.size)
    baseline = _holt_winters_baseline(
        y_tr=y_tr_arr,
        horizon=horizon,
        seasonal_period=seasonal_period,
        trend=hw_trend,
        seasonal=hw_seasonal,

```

```

)

if baseline is None:
    return None

base_train, base_test = baseline
denom_tr = np.where(np.abs(base_train) < 1e-8, 1.0, base_train)
ratio_train = y_tr_arr / denom_tr
denom_te = np.where(np.abs(base_test) < 1e-8, 1.0, base_test)
ratio_test = y_te_arr / denom_te
ratio_train = np.where(np.isfinite(ratio_train), ratio_train, 1.0)
ratio_test = np.where(np.isfinite(ratio_test), ratio_test, 1.0)
return base_train, base_test, ratio_train, ratio_test

def helformer_hw_components(
    y_tr: Iterable[float],
    y_te: Iterable[float],
    *,
    seasonal_period: Optional[int] = None,
    hw_trend: str = "mul",
    hw_seasonal: str = "mul",
) -> Optional[HelformerHWComponents]:
    y_tr_arr = np.asarray(list(y_tr), dtype=float).ravel()
    y_te_arr = np.asarray(list(y_te), dtype=float).ravel()
    horizon = int(y_te_arr.size)
    fit = _holt_winters_fit(
        y_tr_arr,
        seasonal_period=seasonal_period,
        trend=hw_trend,
        seasonal=hw_seasonal,
    )
    if fit is None:
        return None
    fitted, trend_mode, seasonal_mode, seasonal_periods = fit

    base_train = np.asarray(fitted.fittedvalues, float)
    if base_train.size != y_tr_arr.size:
        base_train = np.resize(base_train, y_tr_arr.size)
    base_train = np.where(np.isfinite(base_train), base_train, y_tr_arr)

    base_test = np.asarray(fitted.forecast(horizon), float) if horizon > 0 else np.zeros(0, dtype=float)
    if base_test.size != horizon:
        base_test = np.resize(base_test, horizon)

```

```

last_val = base_train[-1] if base_train.size else (y_tr_arr[-1] if y_tr_arr.size else 0.0)
base_test = np.where(np.isfinite(base_test), base_test, last_val)

denom_tr = np.where(np.abs(base_train) < 1e-8, 1.0, base_train)
ratio_train = y_tr_arr / denom_tr
denom_te = np.where(np.abs(base_test) < 1e-8, 1.0, base_test)
ratio_test = y_te_arr / denom_te
ratio_train = np.where(np.isfinite(ratio_train), ratio_train, 1.0)
ratio_test = np.where(np.isfinite(ratio_test), ratio_test, 1.0)

level = np.asarray(getattr(fitted, "level", None), float) if hasattr(fitted, "level") else None
trend = np.asarray(getattr(fitted, "trend", None), float) if hasattr(fitted, "trend") else None
season = np.asarray(getattr(fitted, "season", None), float) if hasattr(fitted, "season") else None

if level is not None and level.size != y_tr_arr.size:
    level = np.resize(level, y_tr_arr.size)
if trend is not None and trend.size != y_tr_arr.size:
    trend = np.resize(trend, y_tr_arr.size)
if season is not None and season.size != y_tr_arr.size:
    season = np.resize(season, y_tr_arr.size)

return HelformerHWComponents(
    base_train=base_train,
    base_test=base_test,
    ratio_train=ratio_train,
    ratio_test=ratio_test,
    level=level,
    trend=trend,
    season=season,
    trend_mode=trend_mode,
    seasonal_mode=seasonal_mode,
    seasonal_periods=seasonal_periods,
)

__all__ = [
    "create_helformer_model",
    "helformer_forecast",
    "helformer_hw_decompose",
    "helformer_hw_components",
    "load_helformer_model",
]

```

```
from __future__ import annotations

from contextlib import nullcontext
from typing import Iterable, Tuple

import numpy as np

try:
    from statsmodels.tsa.arima.model import ARIMA
    from statsmodels.tsa.holtwinters import ExponentialSmoothing
except Exception:
    ARIMA = None
    ExponentialSmoothing = None

try:
    from statsmodels.tsa.stattools import acf, pacf
except Exception:
    acf = None
    pacf = None

try:
    import pandas as pd
    from prophet import Prophet

    try:
        from cmdstanpy.utils import disable_logging as
_disable_cmdstanpy_logging
    except Exception:
        _disable_cmdstanpy_logging = None
except Exception:
    pd = None
    Prophet = None
    _disable_cmdstanpy_logging = None

def _repeat_last(y: np.ndarray, horizon: int) -> np.ndarray:
    if horizon <= 0:
        raise ValueError("horizon must be positive")
    if y.size == 0:
        return np.zeros(horizon, dtype=float)
    return np.repeat(float(y[-1]), horizon).astype(float)

def _clean_series(y: Iterable[float]) -> np.ndarray:
    data = np.asarray(list(y), dtype=float)
    if data.size == 0:
        return data
    mask = np.isfinite(data)
    if not np.all(mask):
```

```

        data = data[mask]
        return data.astype(float, copy=False)

def _linear_extrapolation(y: np.ndarray, horizon: int, window: int = 32) ->
np.ndarray:
    if horizon <= 0:
        raise ValueError("horizon must be positive")
    if y.size == 0:
        return np.zeros(horizon, dtype=float)
    n = int(y.size)
    w = int(min(max(2, window), n))
    t = np.arange(w, dtype=float)
    y_w = y[-w:].astype(float, copy=False)
    y_mean = float(np.mean(y_w))
    y_centered = y_w - y_mean
    slope, intercept = np.polyfit(t, y_centered, deg=1)
    future_t = np.arange(w, w + horizon, dtype=float)
    return (slope * future_t + intercept + y_mean).astype(float)

def _arma_trend_for_d(d: int) -> str:
    return "t" if int(d) > 0 else "c"

def _fit_forecast_arma(data: np.ndarray, horizon: int, order: Tuple[int, int,
int]) -> np.ndarray:
    if ARIMA is None:
        raise RuntimeError("statsmodels не установлен, ARIMA недоступна")
    p, d, q = (int(order[0]), int(order[1]), int(order[2]))
    trend = _arma_trend_for_d(d)
    model = ARIMA(
        data,
        order=(p, d, q),
        trend=trend,
        enforce_stationarity=False,
        enforce_invertibility=False,
    )
    fitted = model.fit(method_kwargs={"warn_convergence": False, "maxiter":
200})
    forecast = fitted.forecast(steps=horizon)
    return np.asarray(forecast, dtype=float)

def _auto_arma_order(
    data: np.ndarray,
    d: int = 1,
    max_lag: int | None = None,
    max_p: int = 3,
    max_q: int = 3,
) -> Tuple[int, int, int]:

```



```

n = int(data.size)
if n < 10 or acf is None or pacf is None: # type: ignore[truthy-function]
    return (1, d, 0)
max_lag = int(max_lag or min(24, n // 2))
max_lag = max(1, max_lag)
try:
    acf_vals = acf(data, nlags=max_lag, fft=False)
    pacf_vals = pacf(data, nlags=max_lag, method="ywunbiased")
    crit = 1.96 / np.sqrt(n)
    p_candidates = [lag for lag in range(1, max_lag + 1) if
abs(pacf_vals[lag]) > crit]
    q_candidates = [lag for lag in range(1, max_lag + 1) if
abs(acf_vals[lag]) > crit]
    p = min(max(p_candidates) if p_candidates else 0, max_p)
    q = min(max(q_candidates) if q_candidates else 0, max_q)
    return (int(p), int(d), int(q))
except Exception:
    return (1, d, 0)

def arima_forecast(
    y: Iterable[float],
    horizon: int,
    order: Tuple[int, int, int] = (1, 1, 1),
) -> np.ndarray:

    if ARIMA is None:
        raise RuntimeError("statsmodels не установлен, ARIMA недоступна")

    data = _clean_series(y)
    if data.size < 6:
        return _linear_extrapolation(data, horizon)
    try:
        forecast = _fit_forecast_arima(data, horizon, order=order)
        if np.nanstd(forecast) < 1e-12 and np.nanstd(np.diff(data[-min(16,
data.size) :])) > 1e-12:
            return _linear_extrapolation(data, horizon)
        return forecast
    except Exception:
        for alt in ((0, 1, 1), (1, 1, 0), (0, 1, 0), (1, 0, 1)):
            try:
                return _fit_forecast_arima(data, horizon, order=alt)
            except Exception:
                continue
        return _linear_extrapolation(data, horizon)

def ets_forecast(
    y: Iterable[float],
    horizon: int,
    seasonal_periods: int | None = None,

```

```

trend: str | None = "add",
seasonal: str | None = "add",
) -> np.ndarray:
    if ExponentialSmoothing is None:
        raise RuntimeError("statsmodels не установлен, ETS недоступна")
    data = np.asarray(list(y), dtype=float)
    if data.size < 4:
        return _repeat_last(data, horizon)
    seasonal_periods = seasonal_periods if seasonal_periods and
seasonal_periods > 1 else None
    seasonal = seasonal if seasonal_periods else None
    try:
        model = ExponentialSmoothing(
            data,
            trend=trend,
            seasonal=seasonal,
            seasonal_periods=seasonal_periods,
        )
        fitted = model.fit(optimized=True, use_brute=True)
        forecast = fitted.forecast(horizon)
        return np.asarray(forecast, dtype=float)
    except Exception:
        return _repeat_last(data, horizon)

def auto_arma_forecast(
    y: Iterable[float],
    horizon: int,
) -> np.ndarray:
    if ARIMA is None:
        raise RuntimeError("statsmodels не установлен, ARIMA недоступна")
    data = _clean_series(y)
    if data.size < 6:
        return _linear_extrapolation(data, horizon)
    try:
        p, d, q = _auto_arma_order(data)
        forecast = _fit_forecast_arma(data, horizon, order=(p, d, q))
        if np.nanstd(forecast) < 1e-12 and np.nanstd(np.diff(data[-min(16,
data.size) :])) > 1e-12:
            return _linear_extrapolation(data, horizon)
        return forecast
    except Exception:
        best_aic = np.inf
        best_order: Tuple[int, int, int] | None = None
        for d in (0, 1):
            for p in range(0, 3):
                for q in range(0, 3):
                    if p == 0 and q == 0 and d == 0:
                        continue
                    try:
                        model = ARIMA(

```

```

        data,
        order=(p, d, q),
        trend=_arima_trend_for_d(d),
        enforce_stationarity=False,
        enforce_invertibility=False,
    )
    fitted = model.fit(method_kwargs={"warn_convergence":
False, "maxiter": 100})
    aic = float(getattr(fitted, "aic", np.inf))
    if np.isfinite(aic) and aic < best_aic:
        best_aic = aic
        best_order = (p, d, q)
    except Exception:
        continue
    if best_order is not None:
        try:
            return _fit_forecast_arima(data, horizon, order=best_order)
        except Exception:
            pass
    return _linear_extrapolation(data, horizon)

def prophet_forecast(
    y: Iterable[float],
    horizon: int,
    freq: str = "D",
) -> np.ndarray:
    if Prophet is None or pd is None:
        raise RuntimeError("prophet не установлен, Prophet недоступен")
    data = np.asarray(list(y), dtype=float)
    if data.size < 2:
        return _repeat_last(data, horizon)
    context = (
        _disable_cmdstanpy_logging()
        if _disable_cmdstanpy_logging is not None
        else nullcontext()
    )
    try:
        start = pd.Timestamp("2000-01-01")
        idx = pd.date_range(start=start, periods=data.size, freq=freq)
        df = pd.DataFrame({"ds": idx, "y": data})
        m = Prophet(
            seasonality_mode="additive",
            yearly_seasonality=False,
            weekly_seasonality=False,
            daily_seasonality=False,
        )
        with context:
            m.fit(df)
            future = m.make_future_dataframe(periods=horizon, freq=freq,
include_history=False)

```

```

        forecast = m.predict(future)["yhat"].to_numpy()
        return np.asarray(forecast, dtype=float)
    except Exception:
        return _repeat_last(data, horizon)

__all__ = ["arima_forecast", "ets_forecast", "auto_arima_forecast",
"prophet_forecast"]

```

Листинг 2. Архитектура классических статистических моделей.

```

"""Evaluation helpers for M3 Helformer experiments."""
from __future__ import annotations

import warnings
from pathlib import Path
from typing import Dict, Iterable, List, Mapping, Sequence

import numpy as np
import pandas as pd

warnings.filterwarnings("ignore", message=".*np.object.*", category=FutureWarning)

try: # pragma: no cover - optional dependency
    import tensorflow as tf
except Exception: # pragma: no cover - fallback when TF is missing
    tf = None

try: # pragma: no cover - optional dependency
    from statsmodels.tsa.holtwinters import ExponentialSmoothing
except Exception: # pragma: no cover - fallback when statsmodels is missing
    ExponentialSmoothing = None

try: # pragma: no cover - optional dependency
    from statsmodels.tools.sm_exceptions import ConvergenceWarning
except Exception: # pragma: no cover - fallback when statsmodels is missing
    ConvergenceWarning = None

try: # pragma: no cover - optional dependency
    from tqdm.auto import tqdm
except Exception: # pragma: no cover - best-effort fallback
    tqdm = None

```

```

from ..config.settings import settings
from ..data import (
    M3_H,
    M3_P,
    ensure_m3_csv,
    load_train_tsts,
    plot_forecast,
    seasonal_naive,
    smape,
    mape,
    mse,
    rmse,
)
from ..models import arima_forecast, auto_arima_forecast, ets_forecast, create_helformer_model,
prophet_forecast
from ..viz import save_series_viz_bundle_basic

def _progress(iterable, **kwargs):
    if tqdm is None:
        return iterable
    return tqdm(iterable, **kwargs)

def _ensure_min_length(series: np.ndarray, min_len: int) -> np.ndarray:
    series = np.asarray(series, float).ravel()
    if series.size == 0:
        return np.zeros(min_len, dtype=float)
    if series.size >= min_len:
        return series
    pad_len = min_len - series.size
    pad_val = float(series[0])
    pad = np.full(pad_len, pad_val, dtype=float)
    return np.concatenate([pad, series])

def _minmax_fit(series: np.ndarray) -> tuple[float, float]:
    series = np.asarray(series, float).ravel()
    if series.size == 0:
        return 0.0, 1.0
    vmin = float(np.nanmin(series))
    vmax = float(np.nanmax(series))

```

```

scale = vmax - vmin
if not np.isfinite(scale) or abs(scale) < 1e-8:
    scale = 1.0
return vmin, float(scale)

def _minmax_transform(series: np.ndarray, vmin: float, scale: float) -> np.ndarray:
    return (np.asarray(series, float) - vmin) / scale

def _minmax_inverse(series: np.ndarray, vmin: float, scale: float) -> np.ndarray:
    return np.asarray(series, float) * scale + vmin

def _build_xy(series: np.ndarray, lookback: int) -> tuple[np.ndarray, np.ndarray]:
    series = np.asarray(series, float).ravel()
    if series.size <= lookback:
        series = _ensure_min_length(series, lookback + 1)
    X, Y = [], []
    for i in range(lookback, len(series)):
        X.append(series[i - lookback : i])
        Y.append(series[i])
    X = np.asarray(X, np.float32).reshape(-1, lookback, 1)
    Y = np.asarray(Y, np.float32).reshape(-1, 1)
    return X, Y

def _build_x(series: np.ndarray, lookback: int) -> np.ndarray:
    series = np.asarray(series, float).ravel()
    if series.size <= lookback:
        series = _ensure_min_length(series, lookback + 1)
    X = []
    for i in range(lookback, len(series)):
        X.append(series[i - lookback : i])
    return np.asarray(X, np.float32).reshape(-1, lookback, 1)

def _hw_baseline(
    y_tr: np.ndarray,
    horizon: int,
    seasonal_period: int | None,
    trend: str,
    seasonal: str,

```

```

) -> tuple[np.ndarray, np.ndarray] | None:
    if ExponentialSmoothing is None:
        return None
    data = np.asarray(y_tr, float).ravel()
    if data.size < 4:
        return None
    seasonal_periods = seasonal_period if seasonal_period and seasonal_period > 1 else None
    seasonal_mode = seasonal if seasonal_periods else None
    trend_mode = trend
    if (trend_mode == "mul" or seasonal_mode == "mul") and np.any(data <= 0):
        if trend_mode == "mul":
            trend_mode = "add"
        if seasonal_mode == "mul":
            seasonal_mode = "add"
    try:
        model = ExponentialSmoothing(
            data,
            trend=trend_mode,
            seasonal=seasonal_mode,
            seasonal_periods=seasonal_periods,
        )
        with warnings.catch_warnings():
            if ConvergenceWarning is not None:
                warnings.filterwarnings("ignore", category=ConvergenceWarning)
            fitted = model.fit(optimized=True, use_brute=True)
        base_train = np.asarray(fitted.fittedvalues, float)
        if base_train.size != data.size:
            base_train = np.resize(base_train, data.size)
        base_train = np.where(np.isfinite(base_train), base_train, data)
        base_test = np.asarray(fitted.forecast(horizon), float)
        if base_test.size != horizon:
            base_test = np.resize(base_test, horizon)
        last_val = base_train[-1] if base_train.size else (data[-1] if data.size else 0.0)
        base_test = np.where(np.isfinite(base_test), base_test, last_val)
        return base_train, base_test
    except Exception:
        return None

def _predict_test_windows(model, x_test: np.ndarray) -> np.ndarray:
    if x_test.size == 0:
        return np.zeros(0, dtype=float)

```

```

if tf is None:
    raise RuntimeError("TensorFlow is required to run Helformer predictions")
x_tensor = tf.convert_to_tensor(x_test, dtype=tf.float32)
preds = model(x_tensor, training=False)
return np.asarray(preds.numpy(), float).reshape(-1)

def evaluate_m3_helformer(
    categories: Iterable[str] = ("yearly", "quarterly", "monthly"),
    n_per_cat: int | None = None,
    pick: str = "random",
    seed: int = 42,
    helformer_epochs: int = 40,
    helformer_batch_size: int = 32,
    helformer_lr: float = 8e-4,
    helformer_num_blocks: int = 4,
    helformer_num_heads: int = 4,
    helformer_head_size: int = 56,
    helformer_dropout: float = 0.11266524308240201,
    helformer_units: int = 25,
    helformer_lookback: int = 30,
    helformer_verbose: int = 0,
    helformer_use_hw: bool = True,
    helformer_hw_trend: str = "mul",
    helformer_hw_seasonal: str = "mul",
    csv_dir: Path | None = None,
    tsf_dir: Path | None = None,
    out_prefix: Path | None = None,
    force_rebuild_csv: bool = False,
    visualize: bool = False,
    series_override: Mapping[str, Sequence[str]] | None = None,
) -> pd.DataFrame:
    if tf is None:
        raise RuntimeError("TensorFlow is required to train Helformer")

    csv_dir = Path(csv_dir or settings.m3_csv_dir)
    tsf_dir = Path(tsf_dir or settings.m3_tsf_dir)
    out_dir = Path(out_prefix or (settings.outputs_dir / "m3_eval"))
    out_dir.mkdir(parents=True, exist_ok=True)

    ensure_m3_csv(csv_dir=csv_dir, tsf_dir=tsf_dir, force_rebuild=force_rebuild_csv)

```



```

rng = np.random.default_rng(seed)
np.random.seed(seed)
tf.random.set_seed(seed)

rows: List[Dict] = []
categories = tuple(categories)
freq_map = {"yearly": "YE", "quarterly": "QE", "monthly": "ME"}
for cat in _progress(categories, desc="Categories"):
    H = M3_H[cat]
    per = M3_P[cat]
    pairs = load_train_tsts(cat, csv_dir=csv_dir)
    if not pairs:
        print(f"[{cat}] no pairs found in CSV dir: {csv_dir}")
        continue
    selected_list: List[tuple[str, np.ndarray, np.ndarray]]
    if series_override and cat in series_override:
        wanted = set(series_override[cat])
        selected_list = [triple for triple in pairs if triple[0] in wanted]
        if n_per_cat and n_per_cat > 0:
            selected_list = selected_list[: min(len(selected_list), n_per_cat)]
    elif n_per_cat is None or n_per_cat <= 0:
        selected_list = pairs
    elif pick == "first":
        selected_list = pairs[:n_per_cat]
    elif pick == "last":
        selected_list = pairs[-n_per_cat:]
    else:
        count = min(n_per_cat, len(pairs))
        idx = rng.choice(len(pairs), size=count, replace=False)
        selected_list = [pairs[int(i)] for i in idx]

    for sid, y_tr, y_te in _progress(selected_list, desc=f"{cat} series", leave=False):
        forecasts: Dict[str, np.ndarray] = {}
        try:
            tf.keras.backend.clear_session()
            model = create_helformer_model(
                lookback=int(helformer_lookback),
                num_blocks=int(helformer_num_blocks),
                num_heads=int(helformer_num_heads),
                head_size=int(helformer_head_size),
                dropout_rate=float(helformer_dropout),

```

```

        units=int(helformer_units),
    )
    model.compile(
        optimizer=tf.keras.optimizers.Adam(learning_rate=float(helformer_lr)),
        loss="mean_squared_error",
    )
    y_tr_arr = np.asarray(y_tr, float).ravel()
    y_te_arr = np.asarray(y_te, float).ravel()
    ratio_train = y_tr_arr.copy()
    ratio_test = y_te_arr.copy()
    base_train = None
    base_test = None
    if helformer_use_hw:
        baseline = _hw_baseline(
            y_tr_arr,
            H,
            seasonal_period=per,
            trend=helformer_hw_trend,
            seasonal=helformer_hw_seasonal,
        )
        if baseline is not None:
            base_train, base_test = baseline
            denom = np.where(np.abs(base_train) < 1e-8, 1.0, base_train)
            ratio_train = y_tr_arr / denom
            denom_te = np.where(np.abs(base_test) < 1e-8, 1.0, base_test)
            ratio_test = y_te_arr / denom_te
        else:
            print(f"[{cat}:{sid}] HW baseline unavailable; using raw series")

    ratio_train = _ensure_min_length(ratio_train, int(helformer_lookback) + 1)
    train_values = ratio_train
    vmin, scale = _minmax_fit(train_values)
    scaled_train = _minmax_transform(train_values, vmin, scale)
    X, Y = _build_xy(scaled_train, int(helformer_lookback))
    if X.shape[0] == 0:
        raise ValueError("not enough data for Helformer windows")
    model.fit(
        X,
        Y,
        batch_size=int(helformer_batch_size),
        epochs=int(helformer_epochs),

```

```

        verbose=int(helformer_verbose),
    )
    ratio_test = _ensure_min_length(ratio_test, 1)
    test_values = np.concatenate([ratio_train[-int(helformer_lookback) :], ratio_test])
    scaled_test = _minmax_transform(test_values, vmin, scale)
    x_test = _build_x(scaled_test, int(helformer_lookback))
    pred_ratio_scaled = _predict_test_windows(model, x_test)
    pred_ratio = _minmax_inverse(pred_ratio_scaled, vmin, scale)
    if base_test is not None:
        pred = pred_ratio * base_test
    else:
        pred = pred_ratio
    forecasts["Helformer"] = pred[:H]
except Exception as exc:
    print(f"[{cat}:{sid}] Helformer failed: {exc}")
finally:
    try:
        tf.keras.backend.clear_session()
    except Exception:
        pass
try:
    forecasts["ARIMA"] = arima_forecast(y_tr, H)
except Exception as exc:
    print(f"[{cat}:{sid}] ARIMA failed: {exc}")
try:
    forecasts["ARIMA_auto"] = auto_arima_forecast(y_tr, H)
except Exception as exc:
    print(f"[{cat}:{sid}] ARIMA_auto failed: {exc}")
try:
    forecasts["ETS"] = ets_forecast(y_tr, H, seasonal_periods=per)
except Exception as exc:
    print(f"[{cat}:{sid}] ETS failed: {exc}")
try:
    freq = freq_map.get(cat, "D")
    forecasts["Prophet"] = prophet_forecast(y_tr, H, freq=freq)
except Exception as exc:
    print(f"[{cat}:{sid}] Prophet failed: {exc}")
if not forecasts:
    forecasts["Naive"] = seasonal_naive(y_tr, H, per)

rec = {"category": cat, "series_id": sid}

```

```

for name, pred in forecasts.items():
    key = name.replace(" ", "_")
    rec[f"{key}_sMAPE"] = smape(y_te, pred)
    rec[f"{key}_MAPE"] = mape(y_te, pred)
    rec[f"{key}_RMSE"] = rmse(y_te, pred)
    rec[f"{key}_MSE"] = mse(y_te, pred)
rows.append(rec)

title = f"{cat.upper()} {sid} (H={H}, L={helformer_lookback})"
if visualize:
    series_key = f"{cat}_{sid}"
    save_series_viz_bundle_basic(
        out_dir=out_dir / "viz",
        series_key=series_key,
        title_prefix=title,
        y_tr=y_tr,
        y_te=y_te,
        forecasts=forecasts,
    )
else:
    save_png = out_dir / f"{cat}_{sid}.png"
    plot_forecast(title, y_tr, y_te, forecasts, save_path=save_png)

df = pd.DataFrame(rows)
metrics_csv = out_dir / "metrics.csv"
df.to_csv(metrics_csv, index=False)
print(f"[saved] metrics: {metrics_csv}")
if not df.empty:
    metric_suffixes = {
        "sMAPE": "_sMAPE",
        "MAPE": "_MAPE",
        "RMSE": "_RMSE",
        "MSE": "_MSE",
    }
    for metric, suffix in metric_suffixes.items():
        cols = [c for c in df.columns if c.endswith(suffix)]
        if not cols:
            continue
        print(f"[{metric}] mean by category")
        print(df.groupby("category")[cols].mean(numeric_only=True).round(3))
        overall = df[cols].mean(numeric_only=True)

```

```

        print(f"[{metric} overall]")
        print(overall.round(3))
    else:
        print("No results generated — check CSV/logs.")
    return df

__all__ = ["evaluate_m3_helformer"]

```

Листинг 3. Логика обработки датасета *M3*.

```

"""Evaluation helpers for M4 hybrid experiments."""
from __future__ import annotations

from pathlib import Path
from typing import Any, Dict, Iterable, List, Mapping, Sequence
import warnings

import numpy as np
import pandas as pd
import torch

warnings.filterwarnings("ignore", message=".*np.object.*", category=FutureWarning)

try: # pragma: no cover - optional dependency
    from tqdm.auto import tqdm
except Exception: # pragma: no cover - best-effort fallback
    tqdm = None

try: # pragma: no cover - optional dependency
    import tensorflow as tf
except Exception: # pragma: no cover - fallback when TF is missing
    tf = None

try: # pragma: no cover - optional dependency
    import optuna
    from optuna.integration import TFKerasPruningCallback
except Exception: # pragma: no cover - fallback when Optuna is missing
    optuna = None

```

```

TFKerasPruningCallback = None

try: # pragma: no cover - optional dependency
    from statsmodels.tsa.holtwinters import ExponentialSmoothing
except Exception: # pragma: no cover - fallback when statsmodels is missing
    ExponentialSmoothing = None

try: # pragma: no cover - optional dependency
    from statsmodels.tools.sm_exceptions import ConvergenceWarning
except Exception: # pragma: no cover - fallback when statsmodels is missing
    ConvergenceWarning = None

from ..config.settings import settings
from ..data import (
    M4_H,
    M4_P,
    best_L,
    ensure_m4_csv,
    load_m4_train_test,
    plot_forecast,
    plot_forecast_test_only,
    seasonal_naive,
    smape,
    mape,
    mse,
    rmse,
)
from ..hybrids import HybridComponent, HybridPlus, VWHybridMixed, build_global_hybrid_components
from ..hybrids.modwt_hybrid import modwt_decompose_with_boundary
from ..models import (
    arima_forecast,
    auto_arima_forecast,
    create_helformer_model,
    ets_forecast,
    make_model,
    prophet_forecast,
)
from ..training import TrainConfig
from ..viz import (
    save_component_forecast_plot,
    save_component_forecast_test_only_plot,
    save_series_viz_bundle,

```

```

save_series_viz_bundle_basic,
save_series_viz_bundle_helformer_hw,
save_simulation_full_plot,
save_simulation_train_plot,
)

def _fitted_one_step_series(
    component: HybridComponent,
    comp_tr: np.ndarray,
    *,
    device: str,
    total_len: int | None = None,
    mode: str = "rollout",
) -> np.ndarray:
    comp_tr = np.asarray(comp_tr, float).ravel()
    n = int(comp_tr.size)
    if n == 0:
        return comp_tr
    if total_len is None:
        total_len = n
    total_len = int(max(1, total_len))
    lookback = int(component.lookback or 0)
    if component.model is None or lookback <= 0 or n <= 2:
        out = np.empty(total_len, dtype=float)
        init_len = min(n, total_len)
        out[:init_len] = comp_tr[:init_len]
        for t in range(1, init_len):
            out[t] = out[t - 1]
        for t in range(init_len, total_len):
            out[t] = out[t - 1]
        return out

    if bool(getattr(component, "per_series_scaling", False)):
        mu = float(np.mean(comp_tr))
        sd = float(np.std(comp_tr) + 1e-8)
    else:
        mu = float(component.mu)
        sd = float(component.sd + 1e-8)

    lookback = min(lookback, max(1, n - 1))
    out = np.empty(total_len, dtype=float)

```

```

init_len = min(lookback, n, total_len)
out[:init_len] = comp_tr[:init_len]
if init_len < lookback:
    # pad seed if series shorter than lookback
    for t in range(init_len, min(lookback, total_len)):
        out[t] = out[t - 1]
    init_len = min(lookback, total_len)
model = component.model
model.eval()
mode = str(mode or "rollout").lower()
for t in range(init_len, total_len):
    if mode == "fitted" and t < n:
        window = comp_tr[max(0, t - lookback) : t]
        if window.size < lookback:
            pad = np.repeat(window[0] if window.size else out[0], lookback - window.size)
            window = np.concatenate([pad, window])
        else:
            window = out[t - lookback : t]
        xb = ((window - mu) / sd).astype(np.float32).reshape(1, 1, -1)
        with torch.no_grad():
            pred = model(torch.from_numpy(xb).to(device)).detach().cpu().numpy().ravel()
        if pred.size <= 0:
            out[t] = out[t - 1]
        else:
            out[t] = float(pred[0]) * sd + mu
    return out

def _progress(iterable, **kwargs):
    if tqdm is None:
        return iterable
    return tqdm(iterable, **kwargs)

def _ensure_min_length(series: np.ndarray, min_len: int) -> np.ndarray:
    series = np.asarray(series, float).ravel()
    if series.size == 0:
        return np.zeros(min_len, dtype=float)
    if series.size >= min_len:
        return series
    pad_len = min_len - series.size
    pad_val = float(series[0])

```



```

pad = np.full(pad_len, pad_val, dtype=float)
return np.concatenate([pad, series])

def _minmax_fit(series: np.ndarray) -> tuple[float, float]:
    series = np.asarray(series, float).ravel()
    if series.size == 0:
        return 0.0, 1.0
    vmin = float(np.nanmin(series))
    vmax = float(np.nanmax(series))
    scale = vmax - vmin
    if not np.isfinite(scale) or abs(scale) < 1e-8:
        scale = 1.0
    return vmin, float(scale)

def _minmax_transform(series: np.ndarray, vmin: float, scale: float) -> np.ndarray:
    return (np.asarray(series, float) - vmin) / scale

def _minmax_inverse(series: np.ndarray, vmin: float, scale: float) -> np.ndarray:
    return np.asarray(series, float) * scale + vmin

def _build_xy(series: np.ndarray, lookback: int) -> tuple[np.ndarray, np.ndarray]:
    series = np.asarray(series, float).ravel()
    if series.size <= lookback:
        series = _ensure_min_length(series, lookback + 1)
    X, Y = [], []
    for i in range(lookback, len(series)):
        X.append(series[i - lookback : i])
        Y.append(series[i])
    X = np.asarray(X, np.float32).reshape(-1, lookback, 1)
    Y = np.asarray(Y, np.float32).reshape(-1, 1)
    return X, Y

def _build_x(series: np.ndarray, lookback: int) -> np.ndarray:
    series = np.asarray(series, float).ravel()
    if series.size <= lookback:
        series = _ensure_min_length(series, lookback + 1)
    X = []
    for i in range(lookback, len(series)):

```

```

        X.append(series[i - lookback : i])
    return np.asarray(X, np.float32).reshape(-1, lookback, 1)

def _hw_baseline(
    y_tr: np.ndarray,
    horizon: int,
    seasonal_period: int | None,
    trend: str,
    seasonal: str,
) -> tuple[np.ndarray, np.ndarray] | None:
    if ExponentialSmoothing is None:
        return None
    data = np.asarray(y_tr, float).ravel()
    if data.size < 4:
        return None
    seasonal_periods = seasonal_period if seasonal_period and seasonal_period > 1 else None
    seasonal_mode = seasonal if seasonal_periods else None
    trend_mode = trend
    if (trend_mode == "mul" or seasonal_mode == "mul") and np.any(data <= 0):
        if trend_mode == "mul":
            trend_mode = "add"
        if seasonal_mode == "mul":
            seasonal_mode = "add"
    try:
        model = ExponentialSmoothing(
            data,
            trend=trend_mode,
            seasonal=seasonal_mode,
            seasonal_periods=seasonal_periods,
        )
    with warnings.catch_warnings():
        if ConvergenceWarning is not None:
            warnings.filterwarnings("ignore", category=ConvergenceWarning)
        warnings.filterwarnings("ignore", category=RuntimeWarning)
        fitted = model.fit(optimized=True, use_brute=True)
    base_train = np.asarray(fitted.fittedvalues, float)
    if base_train.size != data.size:
        base_train = np.resize(base_train, data.size)
    base_train = np.where(np.isfinite(base_train), base_train, data)
    base_test = np.asarray(fitted.forecast(horizon), float)
    if base_test.size != horizon:

```

```

        base_test = np.resize(base_test, horizon)
        last_val = base_train[-1] if base_train.size else (data[-1] if data.size else 0.0)
        base_test = np.where(np.isfinite(base_test), base_test, last_val)
        return base_train, base_test
    except Exception:
        return None

def _predict_test_windows(model, x_test: np.ndarray) -> np.ndarray:
    if x_test.size == 0:
        return np.zeros(0, dtype=float)
    if tf is None:
        raise RuntimeError("TensorFlow is required to run Helformer predictions")
    x_tensor = tf.convert_to_tensor(x_test, dtype=tf.float32)
    preds = model(x_tensor, training=False)
    return np.asarray(preds.numpy(), float).reshape(-1)

def _predict_rollout(
    model,
    history: list[float],
    lookback: int,
    horizon: int,
) -> np.ndarray:
    if horizon <= 0:
        return np.zeros(0, dtype=float)
    if lookback <= 0:
        raise ValueError("lookback must be positive")
    hist = list(history)
    if not hist:
        hist = [0.0] * lookback
    preds: list[float] = []
    for _ in range(horizon):
        if len(hist) >= lookback:
            window = hist[-lookback:]
        else:
            pad_val = hist[0]
            window = [pad_val] * (lookback - len(hist)) + hist
        x_step = np.asarray(window, np.float32).reshape(1, lookback, 1)
        pred = _predict_test_windows(model, x_step)
        pred_val = float(pred[-1]) if pred.size else (hist[-1] if hist else 0.0)
        if not np.isfinite(pred_val):

```

```

        pred_val = hist[-1] if hist else 0.0
        preds.append(pred_val)
        hist.append(pred_val)
    return np.asarray(preds, float)

def _predict_rollout_full(
    model,
    series_scaled: np.ndarray,
    lookback: int,
    total_len: int,
) -> np.ndarray:
    if total_len <= 0:
        return np.zeros(0, dtype=float)
    if lookback <= 0:
        raise ValueError("lookback must be positive")
    series_scaled = np.asarray(series_scaled, float).ravel().tolist()
    if not series_scaled:
        seed = [0.0] * lookback
    elif len(series_scaled) >= lookback:
        seed = series_scaled[:lookback]
    else:
        pad_val = series_scaled[0]
        seed = series_scaled + [pad_val] * (lookback - len(series_scaled))
    if total_len <= lookback:
        return np.asarray(seed[:total_len], float)
    preds = _predict_rollout(model, seed, lookback, total_len - lookback)
    return np.asarray(seed + preds.tolist(), float)

def _ratio_std_score(
    y_tr: np.ndarray,
    horizon: int,
    seasonal_period: int | None,
    trend: str,
    seasonal: str,
) -> float:
    baseline = _hw_baseline(
        y_tr,
        horizon,
        seasonal_period=seasonal_period,
        trend=trend,

```

```

        seasonal=seasonal,
    )
    if baseline is None:
        return float("nan")
    base_train, _ = baseline
    denom = np.where(np.abs(base_train) < 1e-8, 1.0, base_train)
    ratio = np.asarray(y_tr, float) / denom
    if ratio.size == 0:
        return float("nan")
    return float(np.nanstd(ratio))

def _select_by_ratio_std(
    pairs: Sequence[tuple[str, np.ndarray, np.ndarray]],
    *,
    horizon: int,
    seasonal_period: int | None,
    trend: str,
    seasonal: str,
    min_std: float | None,
    top_k: int | None,
) -> list[tuple[str, np.ndarray, np.ndarray]]:
    scored: list[tuple[float, tuple[str, np.ndarray, np.ndarray]]] = []
    for sid, y_tr, y_te in pairs:
        score = _ratio_std_score(
            y_tr,
            horizon,
            seasonal_period=seasonal_period,
            trend=trend,
            seasonal=seasonal,
        )
        if not np.isfinite(score):
            continue
        if min_std is not None and score < float(min_std):
            continue
        scored.append((score, (sid, y_tr, y_te)))
    if not scored:
        return []
    scored.sort(key=lambda item: item[0], reverse=True)
    if top_k is not None and top_k > 0:
        scored = scored[:top_k]
    picked = [item[1] for item in scored]

```

```

sample = ", ".join(f"{sid} (std={score:.4f})" for score, (sid, _, _) in scored[:3])
print(f"[m4] selected by ratio std: {sample}")
return picked

def _run_helformer_optuna(
    X: np.ndarray,
    Y: np.ndarray,
    lookback: int,
    *,
    n_trials: int,
    timeout: float | None,
    validation_split: float,
) -> dict[str, Any]:
    if optuna is None:
        raise RuntimeError("Optuna is required for Helformer tuning")
    if tf is None:
        raise RuntimeError("TensorFlow is required for Helformer tuning")
    if X.size == 0 or Y.size == 0:
        raise ValueError("not enough data for Helformer tuning")
    if not (0.0 < validation_split < 1.0):
        raise ValueError("validation_split must be between 0 and 1")
    n_trials = int(n_trials)
    if n_trials <= 0:
        raise ValueError("n_trials must be positive")

    def objective(trial: optuna.Trial) -> float:
        learning_rate = trial.suggest_float("learning_rate", 0.0001, 0.01, log=True)
        units = trial.suggest_int("units", 20, 50, step=5)
        dropout_rate = trial.suggest_float("dropout_rate", 0.0, 0.3)
        batch_size = trial.suggest_categorical("batch_size", [16, 32, 64, 128])
        epochs = trial.suggest_int("epochs", 50, 150, step=10)
        num_blocks = trial.suggest_int("num_blocks", 1, 4)
        num_heads = trial.suggest_int("num_heads", 2, 10, step=2)
        head_size = trial.suggest_int("head_size", 8, 64, step=8)

        tf.keras.backend.clear_session()
        try:
            model = create_helformer_model(
                lookback=lookback,
                num_blocks=num_blocks,
                num_heads=num_heads,

```

```

        head_size=head_size,
        dropout_rate=dropout_rate,
        units=units,
    )
    model.compile(
        optimizer=tf.keras.optimizers.Adam(learning_rate=float(learning_rate)),
        loss="mean_squared_error",
    )
    callbacks = []
    if TFKerasPruningCallback is not None:
        callbacks = [TFKerasPruningCallback(trial, "val_loss")]
    history = model.fit(
        X,
        Y,
        batch_size=int(batch_size),
        epochs=int(epochs),
        validation_split=float(validation_split),
        verbose=0,
        callbacks=callbacks,
    )
    val_loss = history.history.get("val_loss")
    if not val_loss:
        return float("inf")
    return float(min(val_loss))
finally:
    try:
        tf.keras.backend.clear_session()
    except Exception:
        pass

study = optuna.create_study(direction="minimize")
study.optimize(objective, n_trials=n_trials, timeout=timeout)
return dict(study.best_params)

MODEL_LABELS = {
    "timesnet": "TimesNet+",
    "nbeats": "N-BEATS Full",
    "vw_timesnet_ets": "VW + TimesNet + ETS",
    "vw_timesnet_arima_auto": "VW + TimesNet + Arima_auto",
    "vw_nbeats_ets": "VW + N-Beats + ETS",
    "vw_nbeats_arima_auto": "VW + N-Beats + Arima_auto",
}

```

```

}

def _base_factory(name: str, params: Mapping[str, Any] | None = None):
    def _fn(cfg: TrainConfig):
        return make_model(name, cfg, params=params)

    return _fn

def _effective_model_params(
    model_name: str,
    *,
    base_model_name: str | None = None,
    seasonal_period: int | None,
    model_params: Mapping[str, Mapping[str, Any]] | None,
) -> Mapping[str, Any] | None:
    base_name = (base_model_name or model_name).lower()
    primary = model_name.lower()
    params_raw = None
    if model_params:
        params_raw = model_params.get(primary)
        if params_raw is None and base_name != primary:
            params_raw = model_params.get(base_name)
    params = dict(params_raw) if params_raw else None
    if base_name == "nbeats" and (seasonal_period is None or seasonal_period <= 1):
        if params is None:
            params = {}
        params.setdefault("use_seasonality", False)
    return params

def _cat_level(cat: str, base_level: int) -> int:
    base_level = int(base_level)
    suggested = {
        "yearly": base_level,
        "quarterly": max(base_level, 2),
        "monthly": max(base_level, 3),
        "weekly": max(base_level, 3),
        "daily": max(base_level, 4),
        "hourly": max(base_level, 5),
    }

```



```

return int(suggested.get(str(cat).lower(), base_level))

def evaluate_m4_hybrids(
    categories: Iterable[str] = ("yearly", "quarterly", "monthly", "weekly", "daily", "hourly"),
    n_per_cat: int | None = None,
    pick: str = "random",
    seed: int = 42,
    epochs: int = 8,
    base_models: Iterable[str] | None = None,
    csv_dir: Path | None = None,
    raw_dir: Path | None = None,
    out_prefix: Path | None = None,
    wavelet: str = "db4",
    level: int = 1,
    boundary: str = "wrap",
    force_rebuild_csv: bool = False,
    force_rebuild_global_components: bool = False,
    series_override: Mapping[str, Sequence[str]] | None = None,
    visualize: bool = False,
    plot_test_only: bool = True,
    use_full_modwt_components: bool = True,
    simulate_full_series: bool = False,
    simulation_mode: str = "rollout",
    simulation_train_only_plot: bool = False,
    use_helformer: bool = False,
    helformer_epochs: int = 40,
    helformer_batch_size: int = 32,
    helformer_lr: float = 8e-4,
    helformer_num_blocks: int = 4,
    helformer_num_heads: int = 4,
    helformer_head_size: int = 56,
    helformer_dropout: float = 0.11266524308240201,
    helformer_units: int = 25,
    helformer_lookback: int = 30,
    helformer_verbose: int = 0,
    helformer_use_hw: bool = True,
    helformer_hw_trend: str = "mul",
    helformer_hw_seasonal: str = "mul",
    helformer_optuna: bool = False,
    helformer_optuna_trials: int = 50,
    helformer_optuna_timeout: float | None = None,

```

```

helformer_optuna_validation_split: float = 0.2,
helformer_pick_by_ratio_std: bool = False,
helformer_ratio_std_min: float | None = None,
helformer_forecast_mode: str = "rollout",
model_params: Mapping[str, Mapping[str, Any]] | None = None,
) -> pd.DataFrame:
    if base_models is None:
        base_models = ("timesnet", "nbeats")
    base_models = tuple((m.lower() for m in base_models))
    label_map = {name: MODEL_LABELS.get(name, f"{name.title()}+") for name in base_models}
    hybrid_models = base_models

    csv_dir = Path(csv_dir or settings.m4_csv_dir)
    out_dir = Path(out_prefix or (settings.outputs_dir / "m4_eval"))
    out_dir.mkdir(parents=True, exist_ok=True)

    ensure_m4_csv(
        csv_dir=csv_dir,
        raw_dir=raw_dir,
        categories=categories,
        force_rebuild=force_rebuild_csv,
    )

    rng = np.random.default_rng(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
    if use_helformer:
        if tf is None:
            raise RuntimeError("TensorFlow is required to train Helformer")
        tf.random.set_seed(seed)
        # Helformer is always used with HW decomposition in M4 experiments.
        helformer_use_hw = True
        if helformer_optuna and optuna is None:
            raise RuntimeError("Optuna is required when helformer_optuna is enabled")

    rows: List[Dict] = []
    categories = tuple(categories)
    freq_map = {
        "yearly": "YE",
        "quarterly": "QE",
        "monthly": "ME",

```

```

    "weekly": "W",
    "daily": "D",
    "hourly": "H",
}

for cat in _progress(categories, desc="Categories"):
    cat = str(cat).lower()
    if cat not in M4_H:
        print(f"[m4:{cat}] unknown category; skipping")
        continue
    H = M4_H[cat]
    per = M4_P[cat]
    cat_level = _cat_level(cat, level)
    pairs = load_m4_train_test(cat, csv_dir=csv_dir)
    if not pairs:
        print(f"[m4:{cat}] no pairs found in CSV dir: {csv_dir}")
        continue

    selected_list: List[tuple[str, np.ndarray, np.ndarray]] | None = None
    if series_override and cat in series_override:
        wanted = set(series_override[cat])
        selected_list = [triple for triple in pairs if triple[0] in wanted]
        if n_per_cat and n_per_cat > 0:
            selected_list = selected_list[: min(len(selected_list), n_per_cat)]
    elif use_helformer and helformer_pick_by_ratio_std:
        selected_list = _select_by_ratio_std(
            pairs,
            horizon=H,
            seasonal_period=per,
            trend=helformer_hw_trend,
            seasonal=helformer_hw_seasonal,
            min_std=helformer_ratio_std_min,
            top_k=n_per_cat if n_per_cat and n_per_cat > 0 else None,
        )
    if not selected_list:
        print(f"[m4:{cat}] ratio-std selection empty; falling back to pick={pick}")
        selected_list = None

    if selected_list is None:
        if n_per_cat is None or n_per_cat <= 0:
            selected_list = list(pairs)
        elif pick == "first":

```

```

        selected_list = list(pairs[:n_per_cat])
    elif pick == "last":
        selected_list = list(pairs[-n_per_cat:])
    else:
        count = min(n_per_cat, len(pairs))
        idx = rng.choice(len(pairs), size=count, replace=False)
        selected_list = [pairs[int(i)] for i in idx]

# Global hybrid components for neural base models (TimesNet / N-BEATS).
# For M4 we optionally use MODWT components computed on the full series
# (train+test) to avoid boundary mismatch; in that mode, global pretraining
# is disabled to keep training consistent.
global_hybrid_components: Dict[str, List[HybridComponent]] = {}
if not use_full_modwt_components:
    for model_name in hybrid_models:
        if model_name not in {"timesnet", "nbeats"}:
            continue
        params = _effective_model_params(
            model_name,
            seasonal_period=(per if per and per > 1 else None),
            model_params=model_params,
        )
        hybrid_ckpt = out_dir / f"{cat}_{model_name}_hybrid_global.pt"
        if hybrid_ckpt.exists() and not force_rebuild_global_components:
            try:
                ckpt = torch.load(hybrid_ckpt, map_location="cpu")
                if "horizon" in ckpt and int(ckpt.get("horizon", -1)) != int(H):
                    raise ValueError("checkpoint params mismatch")
                if "wavelet" in ckpt and ckpt.get("wavelet") != wavelet:
                    raise ValueError("checkpoint params mismatch")
                if "level" in ckpt and int(ckpt.get("level", -1)) != int(cat_level):
                    raise ValueError("checkpoint params mismatch")
                if "boundary" in ckpt and str(ckpt.get("boundary", "wrap")).lower() != str(boundary).lower():
                    raise ValueError("checkpoint params mismatch")
                comps_meta = list(ckpt.get("components", []))
                if any(("per_series_scaling" not in item) for item in comps_meta):
                    raise ValueError("checkpoint too old (missing per_series_scaling)")
                if any(not bool(item.get("per_series_scaling", False)) for item in comps_meta):
                    raise ValueError("checkpoint too old (global scaling)")
                comps: List[HybridComponent] = []
                for item in ckpt.get("components", []):

```

```

state_dict = item.get("state_dict")
lookback = item.get("lookback")
mu = float(item.get("mu", 0.0))
sd = float(item.get("sd", 1.0))
model = None
if state_dict is not None and lookback is not None:
    cfg_global = TrainConfig(
        lookback=int(lookback),
        horizon=H,
        epochs=0,
        batch_size=128,
        lr=1e-3,
        weight_decay=1e-4,
        clip=1.0,
    )
    model = _base_factory(model_name, params=params)(cfg_global)
    model.load_state_dict(state_dict)
    model.to(cfg_global.device)
    model.eval()
    comps.append(
        HybridComponent(
            model=model,
            mu=mu,
            sd=sd,
            lookback=lookback,
            per_series_scaling=bool(item.get("per_series_scaling", True)),
        )
    )
if comps:
    global_hybrid_components[model_name] = comps
    continue
except Exception as exc:
    print(f"[m4:{cat}] failed to load hybrid components for {model_name}: {exc}")

hybrid_cfg = TrainConfig(
    lookback=max(16, min(256, max(32, 2 * H, 3 * per))),
    horizon=H,
    epochs=max(int(epochs), 2),
    batch_size=128,
    lr=3e-4,
    weight_decay=1e-4,

```

```

        clip=1.0,
    )
    comps = build_global_hybrid_components(
        selected_list,
        hybrid_cfg,
        base_model_fn=_base_factory(model_name, params=params),
        wavelet=wavelet,
        level=cat_level,
        boundary=boundary,
    )
    global_hybrid_components[model_name] = comps
    try:
        payload = {
            "category": cat,
            "model_name": model_name,
            "horizon": H,
            "wavelet": wavelet,
            "level": cat_level,
            "boundary": boundary,
            "components": [],
        }
        for comp in comps:
            state = comp.model.state_dict() if comp.model is not None else None
            payload["components"].append(
                {
                    "state_dict": state,
                    "mu": comp.mu,
                    "sd": comp.sd,
                    "lookback": comp.lookback,
                    "per_series_scaling": bool(getattr(comp, "per_series_scaling", False)),
                }
            )
        torch.save(payload, hybrid_ckpt)
    except Exception as exc:
        print(f"[m4:{cat}] failed to save hybrid components for {model_name}: {exc}")

for sid, y_tr, y_te in _progress(selected_list, desc=f"{cat} series", leave=False):
    L = best_L(y_tr, H, per)
    comps_override: list[np.ndarray] | None = None
    comps_full: list[np.ndarray] | None = None
    if use_full_modwt_components:

```

```

y_full = np.concatenate([np.asarray(y_tr, float), np.asarray(y_te, float)], axis=0)
A_full, D_full = modwt_decompose_with_boundary(
    y_full, wavelet=wavelet, level=cat_level, boundary=boundary, check=True
)
comps_full = [A_full] + D_full if len(D_full) else [A_full]
comps_override = [np.asarray(c[: len(y_tr)], float) for c in comps_full]
total_len = int(len(y_tr) + H)
cfg = TrainConfig(
    lookback=L,
    horizon=H,
    epochs=epochs,
    batch_size=64,
    lr=3e-4,
    weight_decay=2e-4,
    clip=0.5,
)
naive = seasonal_naive(y_tr, H, per)
forecasts: Dict[str, np.ndarray] = {"Naive": naive.copy()}
component_forecasts: Dict[str, Dict[str, np.ndarray]] = {}
simulations: Dict[str, np.ndarray] = {}
for model_name in hybrid_models:
    label = label_map[model_name]
    try:
        per_eff = per if per and per > 1 else None
        if model_name in {"timesnet", "nbeats"}:
            params = _effective_model_params(
                model_name,
                seasonal_period=per_eff,
                model_params=model_params,
            )
            model = HybridPlus(
                base_model_fn=_base_factory(model_name, params=params),
                cfg=cfg,
                wavelet=wavelet,
                level=cat_level,
                boundary=boundary,
                pretrained_components=None if use_full_modwt_components else
global_hybrid_components.get(model_name),
                seasonal_period=per_eff,
            ).fit(y_tr, components_override=comps_override)
            forecasts[label] = model.forecast(y_tr, components_override=comps_override)

```

```

        component_forecasts[label] = model.forecast_components(y_tr,
components_override=comps_override)

        if simulate_full_series and comps_override is not None and comps_full is not None:
            fitted_components = []
            for comp_obj, comp_full_arr in zip(model.components, comps_full):
                comp_tr_arr = np.asarray(comp_full_arr[: len(y_tr)], float)
                fitted_components.append(
                    _fitted_one_step_series(
                        comp_obj,
                        comp_tr_arr,
                        device=cfg.device,
                        total_len=total_len,
                        mode=simulation_mode,
                    )
                )
            y_sim = np.sum(np.stack(fitted_components, 0), axis=0)
            # If we simulated beyond train+H (shouldn't), trim.
            simulations[label] = np.asarray(y_sim[:total_len], float)
    elif model_name in {"vw_timesnet_ets", "vw_timesnet_arima_auto"}:
        detail = "ets" if model_name.endswith("_ets") else "arima_auto"
        aj_params = _effective_model_params(
            model_name,
            base_model_name="timesnet",
            seasonal_period=per_eff,
            model_params=model_params,
        )
        model = VWHybridMixed(
            aj_model_fn=_base_factory("timesnet", params=aj_params),
            detail_method=detail,
            cfg=cfg,
            wavelet=wavelet,
            level=cat_level,
            seasonal_period=per_eff,
        ).fit(y_tr, components_override=comps_override)
        forecasts[label] = model.forecast(y_tr, components_override=comps_override)
        component_forecasts[label] = model.forecast_components(y_tr,
components_override=comps_override)

        if simulate_full_series and comps_full is not None and model.aj_component is not None:
            A_tr_arr = np.asarray(comps_full[0][: len(y_tr)], float)
            Aj_sim = _fitted_one_step_series(
                model.aj_component,

```



```

        A_tr_arr,
        device=cfg.device,
        total_len=total_len,
        mode=simulation_mode,
    )
    details_sum = np.zeros(total_len, dtype=float)
    # Use true details on train, predicted details on test (H).
    comp_map = component_forecasts.get(label, {})
    for j, dj_full in enumerate(comps_full[1:], start=1):
        dj_tr = np.asarray(dj_full[: len(y_tr)], float)
        dj_pred = np.asarray(comp_map.get(f"D_{j}"), np.zeros(H), float).ravel()
        if dj_pred.size != H:
            dj_pred = np.pad(dj_pred, (0, max(0, H - dj_pred.size)), mode="edge")[:H]
        dj_series = np.concatenate([dj_tr, dj_pred], axis=0)
        if dj_series.size < total_len:
            dj_series = np.pad(dj_series, (0, total_len - dj_series.size), mode="edge")
        details_sum += dj_series[:total_len]
    simulations[label] = (Aj_sim + details_sum)[:total_len]
elif model_name in {"vw_nbeats_ets", "vw_nbeats_arima_auto"}:
    detail = "ets" if model_name.endswith("_ets") else "arima_auto"
    aj_params = _effective_model_params(
        model_name,
        base_model_name="nbeats",
        seasonal_period=per_eff,
        model_params=model_params,
    )
    model = VWHybridMixed(
        aj_model_fn=_base_factory("nbeats", params=aj_params),
        detail_method=detail,
        cfg=cfg,
        wavelet=wavelet,
        level=cat_level,
        seasonal_period=per_eff,
    ).fit(y_tr, components_override=comps_override)
    forecasts[label] = model.forecast(y_tr, components_override=comps_override)
    component_forecasts[label] = model.forecast_components(y_tr,
components_override=comps_override)
    if simulate_full_series and comps_full is not None and model.aj_component is not None:
        A_tr_arr = np.asarray(comps_full[0][: len(y_tr)], float)
        Aj_sim = _fitted_one_step_series(
            model.aj_component,

```

```

        A_tr_arr,
        device=cfg.device,
        total_len=total_len,
        mode=simulation_mode,
    )
    details_sum = np.zeros(total_len, dtype=float)
    comp_map = component_forecasts.get(label, {})
    for j, dj_full in enumerate(comps_full[1:], start=1):
        dj_tr = np.asarray(dj_full[: len(y_tr)], float)
        dj_pred = np.asarray(comp_map.get(f"D_{j}"), np.zeros(H)), float).ravel()
        if dj_pred.size != H:
            dj_pred = np.pad(dj_pred, (0, max(0, H - dj_pred.size)), mode="edge")[:H]
        dj_series = np.concatenate([dj_tr, dj_pred], axis=0)
        if dj_series.size < total_len:
            dj_series = np.pad(dj_series, (0, total_len - dj_series.size), mode="edge")
        details_sum += dj_series[:total_len]
        simulations[label] = (Aj_sim + details_sum)[:total_len]
    else:
        raise ValueError(f"Unknown hybrid model '{model_name}'")
except Exception as exc:
    print(f"[m4-:{cat}]:{sid}] {label} failed: {exc}")

if use_helformer:
    try:
        tf.keras.backend.clear_session()
        y_tr_arr = np.asarray(y_tr, float).ravel()
        y_te_arr = np.asarray(y_te, float).ravel()
        ratio_train = y_tr_arr.copy()
        ratio_test = y_te_arr.copy()
        base_train = None
        base_test = None
        if helformer_use_hw:
            baseline = _hw_baseline(
                y_tr_arr,
                H,
                seasonal_period=per,
                trend=helformer_hw_trend,
                seasonal=helformer_hw_seasonal,
            )
            if baseline is not None:
                base_train, base_test = baseline

```

```

        denom = np.where(np.abs(base_train) < 1e-8, 1.0, base_train)
        ratio_train = y_tr_arr / denom
        denom_te = np.where(np.abs(base_test) < 1e-8, 1.0, base_test)
        ratio_test = y_te_arr / denom_te
    else:
        print(
            f"[m4:{cat}:{sid}] Helformer HW baseline failed; "
            "using raw series."
        )
    n_train = int(ratio_train.size)
    min_windows = 8
    max_lookback = max(1, n_train - min_windows)
    effective_lookback = min(int(helformer_lookback), max_lookback)
    if n_train <= effective_lookback:
        effective_lookback = max(1, n_train - 1)
    if effective_lookback != int(helformer_lookback):
        print(
            f"[m4:{cat}:{sid}] Helformer lookback adjusted "
            f"{int(helformer_lookback)} -> {effective_lookback} "
            f"(train_len={n_train})"
        )
    ratio_train = _ensure_min_length(ratio_train, effective_lookback + 1)
    vmin, scale = _minmax_fit(ratio_train)
    scaled_train = _minmax_transform(ratio_train, vmin, scale)
    X, Y = _build_xy(scaled_train, effective_lookback)
    if X.shape[0] == 0:
        raise ValueError("not enough data for Helformer windows")
    best_params = None
    if helformer_optuna:
        try:
            best_params = _run_helformer_optuna(
                X,
                Y,
                lookback=effective_lookback,
                n_trials=int(helformer_optuna_trials),
                timeout=helformer_optuna_timeout,
                validation_split=float(helformer_optuna_validation_split),
            )
        print(
            f"[m4:{cat}:{sid}] Helformer Optuna best params: "
            f"{best_params}"

```

```

    )
    except Exception as exc:
        print(f"[m4:{cat}:{sid}] Helformer Optuna failed: {exc}")
        best_params = None

if best_params:
    lr = float(best_params["learning_rate"])
    units = int(best_params["units"])
    dropout_rate = float(best_params["dropout_rate"])
    batch_size = int(best_params["batch_size"])
    epochs_fit = int(best_params["epochs"])
    num_blocks = int(best_params["num_blocks"])
    num_heads = int(best_params["num_heads"])
    head_size = int(best_params["head_size"])
else:
    lr = float(helformer_lr)
    units = int(helformer_units)
    dropout_rate = float(helformer_dropout)
    batch_size = int(helformer_batch_size)
    epochs_fit = int(helformer_epochs)
    num_blocks = int(helformer_num_blocks)
    num_heads = int(helformer_num_heads)
    head_size = int(helformer_head_size)

model = create_helformer_model(
    lookback=effective_lookback,
    num_blocks=num_blocks,
    num_heads=num_heads,
    head_size=head_size,
    dropout_rate=dropout_rate,
    units=units,
)
model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=lr),
    loss="mean_squared_error",
)
model.fit(
    X,
    Y,
    batch_size=batch_size,
    epochs=epochs_fit,

```

```

        verbose=int(helformer_verbose),
    )
    mode = str(helformer_forecast_mode or "rollout").lower()
    pred_ratio_scaled_full = None
    if mode in {"rollout_full", "full", "full_rollout"}:
        pred_ratio_scaled_full = _predict_rollout_full(
            model,
            scaled_train,
            effective_lookback,
            total_len,
        )
        pred_ratio_scaled = pred_ratio_scaled_full[:-H:]
    elif mode in {"rollout", "iterative", "recursive"}:
        scaled_history = scaled_train.tolist()
        pred_ratio_scaled = _predict_rollout(
            model,
            scaled_history,
            effective_lookback,
            H,
        )
    elif mode in {"test_windows", "teacher", "direct"}:
        ratio_test = _ensure_min_length(ratio_test, 1)
        test_values = np.concatenate([ratio_train[-effective_lookback:], ratio_test])
        scaled_test = _minmax_transform(test_values, vmin, scale)
        x_test = _build_x(scaled_test, effective_lookback)
        pred_ratio_scaled = _predict_test_windows(model, x_test)
    else:
        raise ValueError(f"Unknown Helformer forecast mode '{helformer_forecast_mode}'")
    pred_ratio = _minmax_inverse(pred_ratio_scaled, vmin, scale)
    if base_test is not None:
        pred = pred_ratio * base_test
    else:
        pred = pred_ratio
    forecasts["Helformer"] = pred[:H]
    if simulate_full_series:
        if pred_ratio_scaled_full is None:
            pred_ratio_scaled_full = _predict_rollout_full(
                model,
                scaled_train,
                effective_lookback,
                total_len,

```

```

    )
    pred_ratio_full = _minmax_inverse(pred_ratio_scaled_full, vmin, scale)
    if base_test is not None and base_train is not None:
        base_full = np.concatenate([base_train, base_test])
        base_full = np.resize(base_full, total_len)
        pred_full = pred_ratio_full[:total_len] * base_full
    else:
        pred_full = pred_ratio_full[:total_len]
    simulations["Helformer"] = np.asarray(pred_full, float)
except Exception as exc:
    print(f"[m4:{cat}:{sid}] Helformer failed: {exc}")
finally:
    try:
        tf.keras.backend.clear_session()
    except Exception:
        pass

# Classical baselines
try:
    forecasts["ARIMA"] = arima_forecast(y_tr, H)
except Exception as exc:
    print(f"[m4:{cat}:{sid}] ARIMA failed: {exc}")
try:
    forecasts["ARIMA_auto"] = auto_arima_forecast(y_tr, H)
except Exception as exc:
    print(f"[m4:{cat}:{sid}] ARIMA_auto failed: {exc}")
try:
    forecasts["ETS"] = ets_forecast(y_tr, H, seasonal_periods=per)
except Exception as exc:
    print(f"[m4:{cat}:{sid}] ETS failed: {exc}")
try:
    freq = freq_map.get(cat, "D")
    forecasts["Prophet"] = prophet_forecast(y_tr, H, freq=freq)
except Exception as exc:
    print(f"[m4:{cat}:{sid}] Prophet failed: {exc}")
for model_name in base_models:
    label = label_map[model_name]
    forecasts.setdefault(label, naive.copy())

rec = {"category": cat, "series_id": sid}
for name, pred in forecasts.items():

```

```

key = name.replace(" ", "_")
rec[f"{key}_sMAPE"] = smape(y_te, pred)
rec[f"{key}_MAPE"] = mape(y_te, pred)
rec[f"{key}_RMSE"] = rmse(y_te, pred)
rec[f"{key}_MSE"] = mse(y_te, pred)
rows.append(rec)

title = f"{cat.upper()} {sid} (H={H}, L={L})"
if visualize:
    series_key = f"{cat}_{sid}"
    viz_dir = out_dir / "viz"
    if use_helformer and not base_models:
        hw = None
        try:
            from ..models.helformer import helformer_hw_components

            hw = helformer_hw_components(
                y_tr,
                y_te,
                seasonal_period=per,
                hw_trend=helformer_hw_trend,
                hw_seasonal=helformer_hw_seasonal,
            )
        except Exception as exc:
            print(f"[m4:{cat}:{sid}] HW decomposition failed: {exc}")
            hw = None
    save_series_viz_bundle_helformer_hw(
        out_dir=viz_dir,
        series_key=series_key,
        title_prefix=title,
        y_tr=np.asarray(y_tr, float),
        y_te=np.asarray(y_te, float),
        forecasts=forecasts,
        base_train=None if hw is None else hw.base_train,
        base_test=None if hw is None else hw.base_test,
        season=None if hw is None else hw.season,
        seasonal_mode="mul" if hw is None else hw.seasonal_mode,
    )
    elif not base_models:
        save_series_viz_bundle_basic(
            out_dir=viz_dir,

```

```

        series_key=series_key,
        title_prefix=title,
        y_tr=np.asarray(y_tr, float),
        y_te=np.asarray(y_te, float),
        forecasts=forecasts,
    )
else:
    save_series_viz_bundle(
        out_dir=viz_dir,
        series_key=series_key,
        title_prefix=title,
        y_tr=y_tr,
        y_te=y_te,
        forecasts=forecasts,
        wavelet=wavelet,
        level=cat_level,
        boundary=boundary,
        component_forecasts=component_forecasts if component_forecasts else None,
    )
if simulate_full_series and simulations:
    if simulation_train_only_plot:
        save_simulation_train_plot(
            y_tr=np.asarray(y_tr, float),
            simulations=simulations,
            title=f"{title} simulation",
            save_path=(out_dir / "viz" / "09_simulation_train" / f"{series_key}.png"),
        )
    else:
        save_simulation_full_plot(
            y_tr=np.asarray(y_tr, float),
            y_te=np.asarray(y_te, float),
            simulations=simulations,
            title=f"{title} simulation (full series)",
            save_path=(out_dir / "viz" / "09_simulation_full" / f"{series_key}.png"),
        )
else:
    save_png = out_dir / f"{cat}_{sid}.png"
    if plot_test_only:
        save_png = out_dir / f"{cat}_{sid}_test_only.png"
        plot_forecast_test_only(title, y_te, forecasts, save_path=save_png)
    else:

```



```

    plot_forecast(title, y_tr, y_te, forecasts, save_path=save_png)
if component_forecasts:
    save_component_forecast_plot(
        y_tr=y_tr,
        y_te=y_te,
        component_forecasts=component_forecasts,
        wavelet=wavelet,
        level=cat_level,
        boundary=boundary,
        title=f"{title} component forecasts",
        save_path=out_dir / f"{cat}_{sid}_components.png",
    )
    save_component_forecast_test_only_plot(
        y_tr=y_tr,
        y_te=y_te,
        component_forecasts=component_forecasts,
        wavelet=wavelet,
        level=cat_level,
        boundary=boundary,
        title=f"{title} component forecasts (test only)",
        save_path=out_dir / f"{cat}_{sid}_components_test_only.png",
    )
if simulate_full_series and simulations:
    if simulation_train_only_plot:
        save_simulation_train_plot(
            y_tr=np.asarray(y_tr, float),
            simulations=simulations,
            title=f"{title} simulation (train only)",
            save_path=out_dir / f"{cat}_{sid}_simulation_train.png",
        )
    else:
        save_simulation_full_plot(
            y_tr=np.asarray(y_tr, float),
            y_te=np.asarray(y_te, float),
            simulations=simulations,
            title=f"{title} simulation (full series)",
            save_path=out_dir / f"{cat}_{sid}_simulation_full.png",
        )

df = pd.DataFrame(rows)
metrics_csv = out_dir / "metrics.csv"

```

```

df.to_csv(metrics_csv, index=False)
print(f"[saved] metrics: {metrics_csv}")
if df.empty:
    print("No results generated — check CSV/logs.")
    return df

metric_suffixes = {
    "sMAPE": "_sMAPE",
    "MAPE": "_MAPE",
    "RMSE": "_RMSE",
    "MSE": "_MSE",
}
for metric, suffix in metric_suffixes.items():
    cols = [c for c in df.columns if c.endswith(suffix)]
    if not cols:
        continue
    print(f"[{metric}] mean by category")
    print(df.groupby("category")[cols].mean(numeric_only=True).round(3))
    overall = df[cols].mean(numeric_only=True)
    print(f"[{metric} overall]")
    print(overall.round(3))
return df

__all__ = ["evaluate_m4_hybrids"]

```

Листинг 4. Логика обработки датасета *M4*.